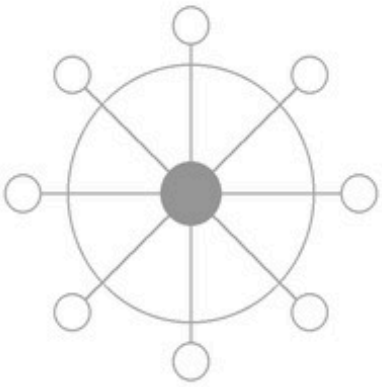
OPENSKILL IT STARTER

오픈스킬 IT 입문

Docker & Kubernetes



개념으로 시작하는 Docker & Kubernetes

OPENSKILL BOOKS
오픈스킬북스

류재성, 최주호, 김주혁 지음

오픈스킬 IT 입문 - 개념으로 시작하는 Docker & Kubernetes

초판 발행	2026년 10월 30일
지은이	류재성, 최주호, 김주혁
펴낸이	최주호
펴낸곳	오픈스킬북스
등록	제2026-000004호 (2026년 3월 20일)
주소	부산, 남구 오륙도로 85, 109동 1701호
전화	051-754-3648
이메일	getinthere@naver.com
ISBN	979-11-998558-2-3
정가	11,000원

© 2026 류재성, 최주호, 김주혁

이 책의 저작권은 지은이와 오픈스킬북스에 있습니다. 저작권법에 의해 보호를 받는 저작물이므로 무단 전재와 복제를 금합니다.

지은이 소개

류재성

전) (주)디오 ERP지원팀

현) (주)마음알지 시스템개발팀

현) 겟인데어 콘텐츠팀 프리랜서 / 메타코딩 유튜브 및 온라인 강의 운영자

'메타코딩' 출판 프로젝트 및 웹 강의 플랫폼 기획에 다수 참여

저서

AWS로 배우는 실전형 CI/CD 2판 (앤써북)

최주호

현) 겟인데어 대표

현) 유튜브 메타코딩 채널 운영

현) 지아이티아카데미 프리랜서 강사

전) 경남정보대학교 - 클라우드 학과 겸임교수

전) 패스트캠퍼스, 스파르타코딩, 인프런, 카카오캠퍼스 다수 강의 참여

전) 노마드랩 개발팀장

전) 군장병 공개 소프트웨어 역량 강화 - 국방부 장관상 수상

저서

AWS로 배우는 실전형 CI/CD 2판 (앤써북)

만들면서 배우는 플러터 앱 프로그래밍 (앤써북)

모두가 할 수 있는 플러터 UI 실전 (앤써북)

모두가 할 수 있는 플러터 UI 입문 (앤써북)

IoT 사물인터넷을 위한 라즈베리파이 5 정석 (앤써북)

JSPStudy의 JSP & Servlet 웹 프로그래밍 입문 + 활용 (앤써북)

그누위즈의 PHP&MySQL 웹 프로그래밍 입문 + 활용 (전면 개정판) (앤써북)

김주혁

전) (사)한국융합인재교육협회 교육팀장

현) (주)엠바스 마케팅&디자인 팀장

현) 겟인데어 콘텐츠팀 프리랜서 / 메타코딩 유튜브 및 온라인 강의 운영자

'메타코딩' 출판 프로젝트 및 웹 강의 플랫폼 기획에 다수 참여

머릿말

개발을 하다 보면 정상적으로 테스트가 끝난 기능이 실제 운영 환경에서는 버그나 오류를 발생시키는 경우가 종종 있습니다. 단지 서버 환경이 다르다는 이유로 예상치 못한 문제를 일으킬 때의 답답함은 개발자라면 누구나 겪어봤을 것입니다.

이러한 배포와 운영 과정의 문제를 해결하기 위해 도커와 쿠버네티스를 도입하게 되었습니다. 도커는 컨테이너 기술을 통해 내 컴퓨터와 운영 서버 간의 환경 차이를 극복하게 해주고, 쿠버네티스는 장애 발생 시 시스템이 스스로 복구되는 안정적인 환경을 만들 수 있습니다.

이 책은 방대한 이론이나 복잡한 실습을 다루기보다는, 도커와 쿠버네티스의 전체적인 구조와 흐름을 쉽게 이해할 수 있도록 구성했습니다. 처음부터 인프라의 모든 세부 기능을 완벽하게 알 필요는 없습니다. 배포와 운영을 위한 새로운 환경이 낯선 분들이 기술의 큰 그림을 파악하는 데 이 책이 도움이 되기를 바랍니다.

류재성

프롤로그

환경 차이에 부딪히다

입사 3개월 차 오픈이는 기능 하나를 손보고 있었습니다. 로컬 PC에서는 테스트가 모두 통과했고, 남은 건 운영 서버에 올리는 일뿐이었습니다.

빌드한 파일을 서버에 올리고 실행하자, 터미널이 한 박자 멈춘 뒤 빨간 글씨가 화면을 채웠습니다. 자바 버전이 맞지 않는다는 오류였습니다. 버전을 맞추자 이번엔 라이브러리가 의존하는 시스템 파일이 서버에 없었고, 그 파일을 설치하니 설정 파일의 경로가 서버의 디렉터리 구조와 어긋났습니다.

'내 PC에선 분명히 됐는데.'

하나를 해결하면 곧바로 또 다른 오류가 터졌습니다. 코드는 그대로인데 서버의 환경만 조금씩 어긋나 있었습니다. 한참을 헤매고 있을 때, 옆자리 선배가 모니터를 잠깐 들여다봤습니다.

선배 : "환경이 달라서 그래요. 도커(Docker) 한번 알아봐요."

선배의 말을 들은 오픈이는 퇴근 후 도커를 검색했습니다. 도커란 실행에 필요한 파일과 설정을 **컨테이너**라는 표준 상자에 담아, **어디서든 똑같이 실행**하게 만드는 기술이었습니다.

환경을 컨테이너에 담다

다음 날 저녁, 오픈이는 커피 한 잔을 들고 노트북 앞에 앉았습니다. 전날 본 영상으로 컨테이너와 도커가 왜 필요한지는 이해했고, 이제 남은 건 직접 손에 익히는 일이었습니다.

도커는 가상 머신과 달리 호스트의 운영체제 커널을 공유합니다. 그래서 무거운 OS 전체를 복제하지 않고도, 컨테이너마다 격리된 실행 공간을 가볍게 확보할 수 있었습니다. 먼저 Docker Hub에서 우분투 이미지를 내려받아 컨테이너로 실행하고, 그 안에서 패키지를 설치하고 웹 서버를 실행해 봤습니다.

그렇게 구성한 컨테이너의 상태도 그대로 **이미지**로 저장할 수 있었습니다. 그 이미지를 Docker Hub에 올려 두면, 누구든 어떤 서버에서든 똑같은 환경을 그대로 내려받아 실행할 수 있었습니다. **이미지 하나로 환경을 통째로 옮길 수 있게** 되면서, 환경 차이로 발생하던 문제가 해결됐습니다.

여러 컨테이너를 한 번에 실행하다

며칠 후, 화요일 오전 열 시. 회의실에 사람이 모였습니다. 팀장이 화이트보드 앞에서 이번에 만들 프로젝트를 설명했습니다. 화면을 그릴 프론트엔드, 요청을 처리할 백엔드, 회원 정보를 보관할 DB가 함께 구성된 서비스였습니다.

팀장 : "환경은 도커로 구성해봐요. 요즘 공부했잖아요."

회의실을 나서는 발걸음이 가볍지 않았습니다. 컨테이너 하나를 실행하는 건 익혔지만, 여러 개를 한꺼번에 관리해 본 적은 없었습니다.

'한 대씩은 실행해봤는데, 여러 대를 함께 구성하는 건 또 다른 이야기인데.'

자리로 돌아온 오픈이가 선배에게 물었습니다.

오픈이 : "프론트엔드·백엔드·DB까지 구성해야 하는데, 어디서부터 손대야 할지 모르겠어요."

선배 : "한꺼번에 다 하려니 막막한 거예요. 하나씩 하면 돼요. 서비스마다 필요한 걸 다 갖춘 환경을 미리 준비해 두고, 외부에서 요청을 받아 알맞은 컨테이너로 전달한 뒤, 마지막에 전체를 하나로 합쳐 한 번에 실행하면 돼요."

오픈이는 하나씩 풀어 갔습니다. 먼저 서비스마다 필요한 환경을 **Dockerfile**에 적어 두니, 누가 빌드해도 같은 이미지가 만들어졌습니다.

그다음 요청이 여러 컨테이너로 나뉘어야 할 때는 앞단에 **NGINX**를 세워, 들어온 요청을 경로에 따라 알맞은 백엔드로 분배했습니다. 여러 백엔드가 같은 세션을 공유해야 할 때는 **Redis**에 세션을 모아 두었습니다.

마지막으로 **Docker Compose** 파일 한 장에 서비스들과 연결 관계를 적고 명령어를 입력하자, 흩어져 있던 컨테이너가 명세서대로 함께 실행되며 **하나의 웹 서비스로 동작**했습니다.

완성한 프로젝트를 회의실에서 시연했습니다. 시연은 문제없이 끝났지만, 팀장이 마지막에 한 가지를 물었습니다.

팀장 : "환경 구성은 깔끔하네요. 그런데 운영 중에 컨테이너 하나라도 죽으면 누가 살리죠?"

오픈이는 그 질문에 답하지 못했습니다.

컨테이너를 자동으로 관리하다

회의실을 나선 뒤에도 그 질문이 머릿속을 떠나지 않았습니다. 도커로 컨테이너를 실행할 수는 있어도, 죽은 컨테이너를 되살리는 일까지는 도커만으로 답이 보이지 않았습니다.

점심을 먹고 돌아오는 길에 선배에게 고민을 털어놓았습니다.

오픈이 : "도커를 공부하면 끝일 줄 알았는데, 운영에서 컨테이너가 죽거나 트래픽이 몰릴 때 어떻게 해야 할지 모르겠어요."

선배 : "도커만으로는 운영까지 감당하기 어려워요. 그래서 쿠버네티스(Kubernetes)를 같이 알아야 해요. 컨테이너 관리는 쿠버네티스 담당이거든요."

쿠버네티스는 명령을 하나하나 내리는 대신, 유지할 상태를 선언해 두면 시스템이 그 상태를 알아서 유지하는 방식이었습니다.

오픈이는 **Deployment** 설정 파일에 유지할 Pod 개수를 적어 배포한 뒤, 고의로 Pod 하나를 종료해 봤습니다. 쿠버네티스는 즉시 빈자리를 감지하고 새 Pod를 생성해 원래 개수를 맞췄습니다.

새 버전을 올릴 때도 한꺼번에 바꾸지 않고 Pod를 하나씩 교체하는 **롤링 업데이트**로, 서비스가 끊기지 않았습니다. 이제 컨테이너가 종료되거나 트래픽이 몰려도, **시스템이 선언한 상태를 스스로 유지**하게 되었습니다.

외부에서 접속할 주소를 만들다

오픈이는 학습한 내용을 바탕으로 테스트 서버를 Pod로 재구성했습니다. Pod가 종료되더라도 Deployment가 즉시 새 Pod를 실행해 주니 마음이 든든했습니다.

다음 날 아침, 동료가 찾아왔습니다.

동료 : "오픈 씨, 어제 테스트 서버에 올렸다는 서비스, 접속이 안 되는데 어떻게 들어가요?"

오픈이는 당황했습니다. 로그상으로는 정상 실행 중이었지만, 정작 브라우저로 접속해 본 적은 없었습니다. Pod 목록에서 IP를 확인해 입력해 봐도 페이지는 열리지 않았고, 게다가 Pod가 재생성될 때마다 IP가 바뀌니 그 주소로는 접근할 수도 없었습니다.

'쿠버네티스 환경에서는 외부에서 Pod로 어떻게 접속해야 하지?'

막막해진 오픈이가 선배에게 묻자, 선배가 화면을 보며 설명했습니다.

선배 : "외부 요청이 실제 Pod까지 도달하려면 두 가지가 필요해요. 들어온 요청을 경로에 따라 알맞은 곳으로 보내 주는 Ingress, 그리고 수시로 바뀌는 Pod에 고정된 진입점을 제공하는 Service죠."

오픈이는 IP가 계속 바뀌는 Pod들 앞에 고정된 주소를 제공하는 **Service**를 배치했습니다. 그다음 **Ingress**를 통해 외부에서 들어온 요청을 경로에 따라 알맞은 Service로 넘겨주었습니다.

구성을 마치자 외부 브라우저의 요청이 Ingress와 Service를 거쳐 Pod 안의 애플리케이션에 정상적으로 도달했습니다. 이제 IP가 계속 바뀌어도, **외부에서 고정된 주소로 서비스에 접속**할 수 있게 되었습니다.

설정과 데이터를 따로 관리하다

며칠 뒤, 오픈이는 프로젝트를 배포하기 전 마지막으로 코드 리뷰를 하고 있었습니다. 그러던 중 Dockerfile의 환경 변수가 눈에 띄었습니다. 데이터베이스 비밀번호가 코드에 그대로 적혀 있었습니다.

'잠깐. 이대로 빌드하면 비밀번호가 이미지에 그대로 남는데.'

오픈이가 옆자리 선배에게 묻자, 선배가 의자를 돌려 오픈이를 봤습니다.

선배 : "테스트 환경에서는 괜찮지만, 실제 운영 환경에 배포할 때는 두 가지를 주의해야 해요. 비밀번호 같은 설정값은 자주 바뀌니 Dockerfile에 직접 적지 말고 이미지 밖으로 분리해서 관리하고, Pod는 언제든 종료될 수 있으니 데이터는 Pod 안이 아니라 바깥 저장소에 둬요. 설정과 데이터 모두 컨테이너 밖으로 안전하게 빼 두는 게 핵심이에요."

오픈이는 환경별 설정값을 **ConfigMap**에, 민감한 비밀번호는 **Secret**에 분리해 Pod 실행 시 외부에서 주입되도록 수정했습니다.

데이터베이스의 실제 데이터는 Pod 내부가 아닌 **PV(Persistent Volume)** 라는 외부 저장 공간을 연결해 보관했습니다. 컨테이너가 삭제되거나 재생성되어도 **설정과 데이터는 외부에서 유지**되었습니다.

프론트엔드, 백엔드, DB가 외부 설정 파일을 참조하며 동작하도록 구성을 마쳤습니다. 도커로 실행 환경을 규격화하고, 쿠버네티스로 운영까지 맡기는 구조가 완성됐습니다.

오픈이는 완성된 시스템을 선배에게 보여 주었습니다.

오픈이 : "환경의 차이를 해결하려고 시작했는데, 이제 시스템이 자동으로 운영되는 것까지 해결됐네요."

선배는 흐뭇한 표정으로 답했습니다.

선배 : "처음부터 모든 걸 알고 시작하는 사람은 없어요. 부딪히고, 배우고, 익히다 보면 되는 거예요."

목차

머릿말 1

프롤로그 2

챕터 1. 왜 컨테이너인가 7

1.1 컨테이너 - 항구에서 빌려온 이름 9

1.1.1 하역에 며칠씩 걸리던 시절 9

1.1.2 규격을 통일하다 10

1.1.3 IT로 건너온 컨테이너 10

1.2 Docker와 Kubernetes - 공장과 관제 센터 11

1.3 학습 지도 - Docker에서 Kubernetes까지 12

챕터 2. Docker 이해하기 14

2.1 가상화 - 가상머신과 컨테이너 16

2.1.1 한 주방, 네 명의 요리사 16

2.1.2 IT로 옮긴 구조 18

2.1.3 컨테이너가 만들어지는 과정 19

2.2 이미지 - 컨테이너의 설계도 20

2.2.1 이미지란? 20

2.2.2 이미지 레이어 21

2.2.3 Docker Hub 21

2.3 Docker CLI - 첫 컨테이너 띄우기 22

2.3.1 docker pull: 이미지 가져오기 22

2.3.2 docker run: 컨테이너 띄우기 23

2.3.3 docker run -d: 백그라운드로 띄우기 24

2.3.4 docker ps: 실행 중인 컨테이너 확인 25

2.3.5 자주 쓰는 명령어 25

2.4 컨테이너 통신 26

2.4.1 컨테이너 간 통신 27

2.4.2 외부에서 들어오기 - 입구 키오스크 28

2.4.3 이름으로 찾기 - 홀의 안내 지도 29

2.5 컨테이너 내부 - 격리된 작은 리눅스 30

2.5.1 자주 쓰는 리눅스 명령어 31

2.5.2 실습: 컨테이너 안에 nginx 깔아보기 32

2.5.3 vim으로 파일 편집 34

2.6 컨테이너의 메인 프로세스 36

2.6.1 메인 프로세스란? 36

2.6.2 메인 프로세스에서 빠져나오기 36

2.6.3 메인 프로세스가 유지되는 조건 37

2.6.4 메인 프로세스 바꾸기 - CMD 39

2.6.5 실행 중인 컨테이너에 들어가기 - attach와 exec 39

2.7 commit - 실행 중인 컨테이너를 이미지로 42

2.7.1 Tomcat으로 commit 실습하기 42

2.7.2 Docker Hub에 올리기 45

2.8 마운트로 데이터 보존 46

2.8.1 바인드 마운트: 호스트 폴더와 직접 연결 46

2.8.2 볼륨 마운트: Docker가 관리하는 저장소 48

챕터 3. Docker 다루기 52

3.1 Dockerfile - 환경을 자동으로 만들기 55
3.1.1 프로비저닝 55
3.1.2 Dockerfile에서 컨테이너까지의 세 단계 55
3.1.3 Dockerfile 기본 문법 56
3.1.4 WORKDIR와 COPY 57
3.1.5 CMD와 ENTRYPOINT 58
3.2 NGINX - 요청을 앞에서 받아 나눠주기 60
3.2.1 NGINX의 역할 60
3.2.2 NGINX 설정과 요청 흐름 60
3.2.3 경로 기반 라우팅 61
3.2.4 host.docker.internal과 사용자 정의 네트워크 64
3.2.5 로드밸런싱 66
3.2.6 캐싱 69
3.3 Redis - 서버 여러 대가 함께 쓰는 공용 저장소 74
3.3.1 서버 여러 대면 생기는 세션 문제 75
3.3.2 Redis - 공용 세션 저장소 75
3.3.3 실습: Redis로 세션 공유 76
3.4 MySQL - 영구 데이터 저장하기 78
3.4.1 MySQL 컨테이너 실행하기 78
3.5 Docker Compose - 여러 컨테이너를 한 번에 81
3.5.1 컨테이너를 하나씩 실행하는 한계 81
3.5.2 docker-compose.yml 기본 구조 82
3.5.3 실습: Compose로 ex01 다시 만들기 82
3.6 종합 실습 - 프론트엔드·백엔드·DB 연결하기 84
3.6.1 전체 아키텍처 84
3.6.2 Backend - Spring Boot 컨테이너 85
3.6.3 DB - MySQL 컨테이너 86
3.6.4 Frontend - 정적 페이지 + API 프록시 86
3.6.5 docker-compose.yml 86

챕터 4. Kubernetes 시작하기 90
4.1 Kubernetes - 원하는 상태를 유지하기 93
4.1.1 도커는 실행 명령, Kubernetes는 약속 유지 93
4.1.2 컨트롤 플레인과 워커 노드 94
4.1.3 Kubernetes의 동작 원리 95
4.1.4 Kubernetes 핵심 리소스 한눈에 97
4.2 Minikube - 로컬에 만드는 작은 클러스터 97
4.2.1 노트북 한 대로 클러스터 흉내 내기 97
4.2.2 클러스터 시작 98
4.2.3 자주 쓰는 Minikube 명령어 99
4.3 Pod - Kubernetes의 최소 단위 99
4.3.1 Pod란? 100
4.3.2 명령어로 Pod 만들기 100
4.3.3 YAML로 Pod 만들기 101
4.3.4 Pod 조회 102
4.3.5 자주 쓰는 kubectl 명령어 103
4.4 Deployment - 자동 복구·스케일링·무중단 배포 104
4.4.1 Pod 하나를 직접 만들면 생기는 문제 104
4.4.2 Deployment - 원하는 상태를 유지하는 매뉴얼 105
4.4.3 ReplicaSet - Pod 개수 자동 관리 108
4.4.4 롤링 업데이트 - 끊김 없는 버전 교체 110
4.4.5 Rollback - 되돌리기 112

챕터 5. Kubernetes 네트워킹 114

5.1 Service - Pod의 고정 주소 117

5.1.1 Service가 필요한 이유 117

5.1.2 Service 생성 117

5.1.3 Service 타입과 접근 범위 118

5.1.4 포트 흐름 120

5.1.5 외부에서 Service 접속해 보기 121

5.2 Ingress - 도메인 라우팅 124

5.2.1 Ingress의 역할 124

5.2.2 Ingress 컨트롤러 125

5.2.3 Ingress 적용하기 125

5.2.4 외부에서 Ingress 접속해 보기 128

5.2.5 두 리소스의 계층 분담 129

5.3 브라우저에서 Pod까지의 경로 130

5.3.1 NodePort로 클러스터에 들어오기 131

5.3.2 Ingress가 URL·도메인으로 Service 분기하기 131

5.3.3 ClusterIP가 Pod로 전달하기 132

챕터 6. Kubernetes 운영하기 135

6.1 ConfigMap·Secret - 설정과 비밀번호를 이미지 외부로 138

6.1.1 ConfigMap 138

6.1.2 Secret 140

6.1.3 ConfigMap 변경 142

6.2 Volume - 데이터의 영속성 확보 145

6.2.1 Pod의 휘발성 문제 145

6.2.2 PV와 PVC 146

6.2.3 PV 만들기 146

6.2.4 PVC 만들기 147

6.2.5 Pod에 마운트 148

6.3 통합 실습 - 쿠버네티스 위에 웹사이트 올리기 150

6.3.1 전체 그림 150

6.3.2 폴더 구조와 진행 방식 150

6.3.3 리소스 살펴보기 151

6.3.4 공통 준비 155

6.3.5 한 번에 배포하고 결과 확인 156

에필로그 159

마치며 160

CHAPTER

1

왜 컨테이너인가

환경 차이에 부딪히다

이번 챕터가 끝나면

- 환경이 달라 생기는 문제를 **컨테이너**가 어떻게 해결하는지 이해합니다.
- **도커**와 **쿠버네티스**의 역할을 이해합니다.
- Docker에서 Kubernetes까지 이어지는 학습 여정을 미리 그려 봅니다.

챕터 1. 왜 컨테이너인가

입사 3개월 차 오픈이는 기능 하나를 손보고 있었습니다. 로컬 PC에서는 테스트가 모두 통과했고, 남은 건 운영 서버에 올리는 일뿐이었습니다.

빌드한 파일을 서버에 올리고 실행 명령을 입력했습니다. 그런데 터미널이 한 박자 멈춘 뒤 빨간 글씨가 화면을 채웠습니다. 자바 버전이 맞지 않는다는 오류였습니다.

자바 버전을 서버에 맞추자 이번엔 다른 오류가 떴습니다. 라이브러리가 의존하는 시스템 파일이 서버에 없었습니다. 그 파일을 설치하니 곧바로 설정 파일의 경로가 서버의 디렉터리 구조와 어긋났습니다.

'내 PC에선 분명히 됐는데.'

하나를 해결하면 곧바로 또 다른 오류가 터졌습니다. 코드는 그대로인데 서버의 환경만 조금씩 어긋나 있었습니다.



그림 1-1. 환경 불일치로 막힌 오픈이

한참을 헤매고 있을 때, 옆자리 선배가 모니터를 잠깐 들여다봤습니다.

선배 : "환경이 달라서 그래요. **도커(Docker)** 한번 알아봐요."

선배의 말을 들은 오픈이는 퇴근 후 유튜브에 **도커**를 검색했습니다. 그중 가장 눈에 띈 영상은 도커와 컨테이너의 유래를 다룬 내용이었습니다.

1.1 컨테이너 - 항구에서 빌려온 이름

1.1.1 하역에 며칠씩 걸리던 시절

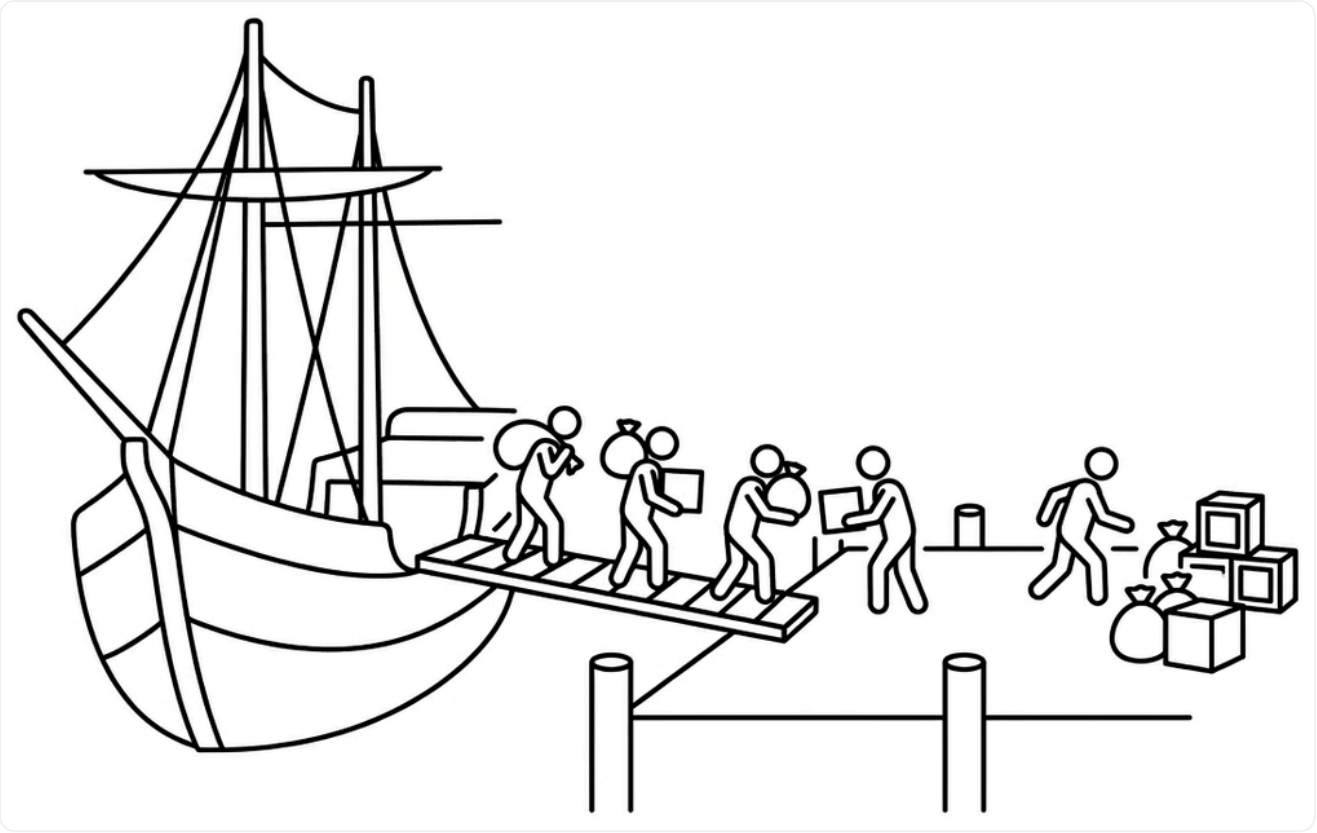


그림 1-2. 컨테이너가 없던 시절의 해상 물류

1950년대 미국 동부의 한 항구. 수십 명의 인부가 화물을 일일이 어깨에 메고 부두로 옮기고 있었습니다. 자루에 담긴 곡물, 묶음으로 된 나무 상자, 무거운 드럼통, 제각각인 길이의 철근 다발까지 화물의 모양은 천차만별이었습니다.

배를 다 비우는 데 며칠이 걸리던 시절이었습니다. 규격이 제각각이라 분류부터 트럭 적재까지 모두 사람 손을 거쳤습니다. 그 때문에 화물을 실어 가려는 트럭 운전사들은 줄 끝에서 며칠씩 차례를 기다렸습니다.

그 줄 한가운데에 트럭 운전사 **말콤 맥린**이 있었습니다. 지루한 기다림 속에서 그는 한 가지 의문을 품었습니다. **'왜 화물을 통째로 옮기지 않을까?'**

1.1.2 규격을 통일하다

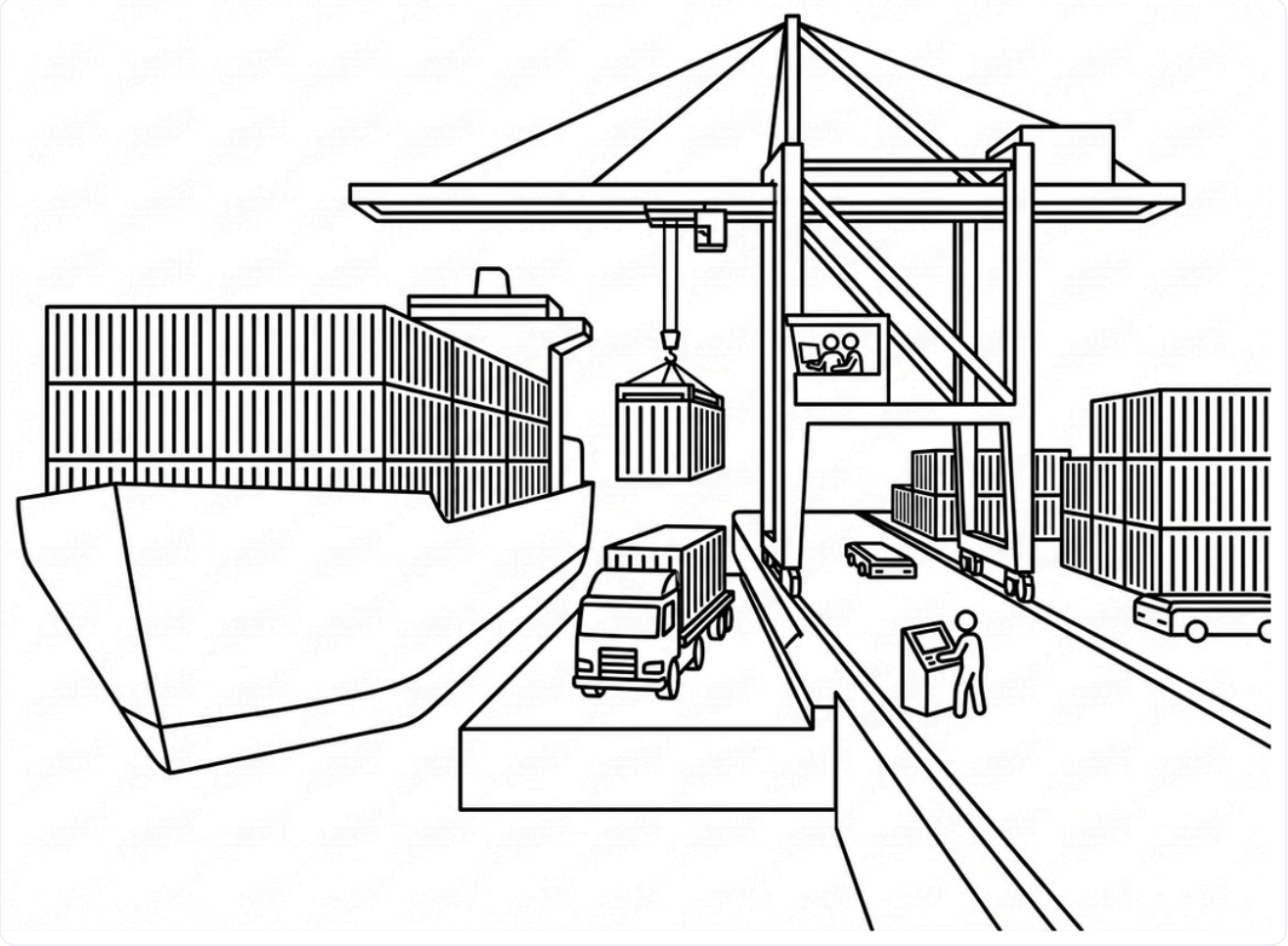


그림 1-3. 표준 컨테이너와 크레인으로 바뀐 현대 항구

맥린이 찾은 답은 **표준 규격의 상자**였습니다. 화물을 낱개로 다루지 말고, 크기와 모양이 똑같은 상자에 담아 통째로 옮기자는 것이었습니다.

규격이 생기자 항구가 달라졌습니다. 크레인이 상자를 통째로 들어 배에 싣고, 도착지에서도 같은 방식으로 내렸습니다. 안에 무엇이 들었든 **규격만 같으면 처리 과정은 동일**했습니다.

덕분에 며칠씩 걸리던 하역이 몇 시간으로 줄었습니다. 맥린은 배와 트럭을 잇는 운송 시스템까지 구축했고, 그렇게 '담는 그릇'이라는 뜻의 **컨테이너**(Container)는 비로소 **세계 물류의 표준**이 되었습니다.

1.1.3 IT로 건너온 컨테이너

컴퓨터마다 다른 환경은 소프트웨어를 옮길 때 늘 걸림돌이 됩니다. 운영체제나 라이브러리, 설정이 조금만 달라도 오류가 발생할 수 있기 때문입니다.

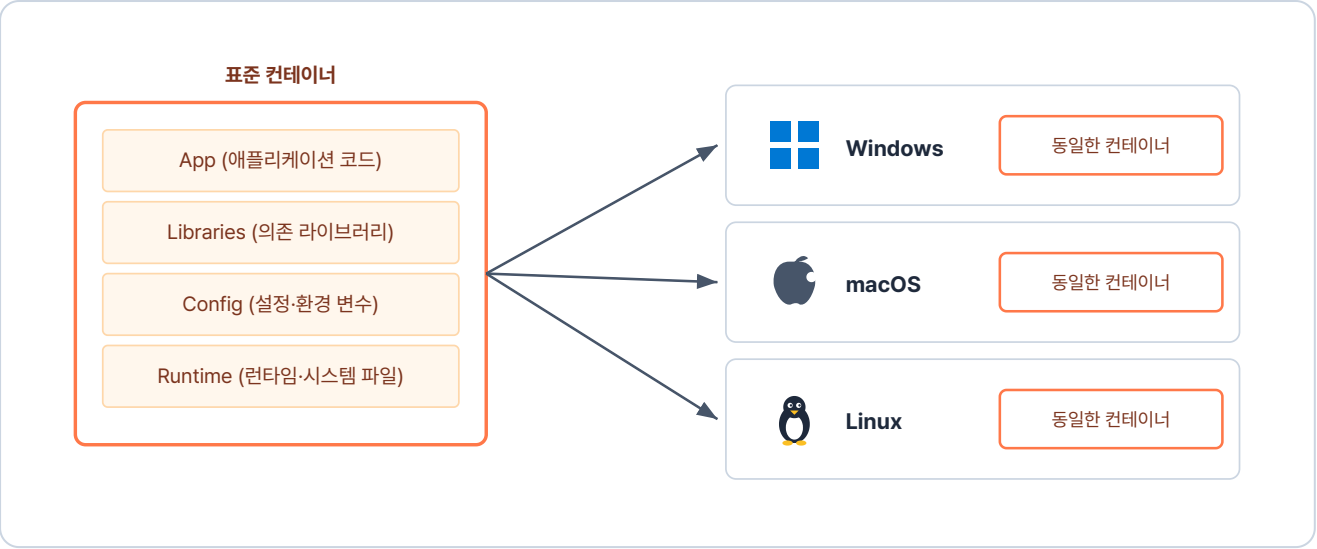


그림 1-4. 소프트웨어도 똑같이 표준 컨테이너에 담아 어디서든 같은 결과로 실행

물류가 컨테이너로 표준을 세웠듯, 소프트웨어도 **컨테이너**라는 개념으로 이 문제를 해결했습니다. 실행에 필요한 파일과 설정을 독립된 컨테이너에 담아 통째로 옮깁니다. 그러면 인프라 환경이 어떻게 바뀌든, 만든 프로그램이 늘 **같은 환경에서 안정적으로** 돌아갑니다.

> **컨테이너(Container)**: 애플리케이션과 그 애플리케이션이 필요로 하는 모든 것(라이브러리, 설정, 시스템 파일)을 하나의 상자에 담아, 어디서 실행해도 같은 결과를 내는 실행 단위입니다.

1.2 Docker와 Kubernetes - 공장과 관제 센터

컨테이너를 쓰려면 먼저 그것을 만들어낼 도구가 필요합니다. IT에서 그 역할을 하는 도구가 **Docker**입니다.

Docker는 **이미지**라는 설계도를 바탕으로 작동합니다. 이미지에 실행 환경을 미리 정의해 두면, 똑같은 환경의 컨테이너를 언제 어디서든 몇 개든 만들어낼 수 있습니다.

문제는 컨테이너가 수백, 수천 개로 늘어날 때입니다. 어느 서버에 올릴지, 문제가 생긴 컨테이너는 어떻게 교체할지 사람이 일일이 관리하기란 불가능에 가깝습니다. 항구에서 수많은 컨테이너를 조율하는 **관제 시스템**이 필요한 것과 같습니다.

소프트웨어에서 이 관제 시스템 역할을 하는 것이 **Kubernetes**입니다. "이 서비스를 항상 3개 유지해줘"처럼 **원하는 상태**를 선언해 두면, Kubernetes가 컨테이너 상태를 계속 지켜봅니다. 하나가 죽으면 자동으로 새 컨테이너를 띄우고, 필요에 따라 수를 늘리거나 줄이며 전체를 조율합니다.

'Docker가 컨테이너를 만들고, Kubernetes가 그걸 관리하는구나.'

Docker와 Kubernetes의 역할을 정리하면 다음과 같습니다.

구분	Docker	Kubernetes
역할	컨테이너 만들고 실행	컨테이너 운영 자동화
비유	표준 컨테이너를 찍는 공장	수많은 배를 관리하는 항만 관제 센터
핵심 기능	이미지 빌드, 컨테이너 실행	자동 복구, 스케일링, 무중단 배포

1.3 학습 지도 - Docker에서 Kubernetes까지

이 책의 학습 여정은 다섯 챕터로 이어집니다.



그림 1-5. 책 한 권 분량의 학습 여정. 한 단계씩 다음 챕터로 나아갑니다

각 챕터에서 다루는 내용은 다음과 같습니다.

- 챕터 2에서는 Docker로 컨테이너가 격리되는 원리를 익히고, 직접 실행해 이미지로 만듭니다.
- 챕터 3에서는 Dockerfile과 Docker Compose로 여러 컨테이너를 한 번에 다룹니다.
- 챕터 4에서는 Kubernetes의 Pod와 Deployment로 컨테이너를 자동 복구·확장합니다.

- 챕터 5에서는 Service와 Ingress로 매번 바뀌는 주소를 고정하고 외부에서 접속할 수 있게 합니다.
- 챕터 6에서는 Secret과 Volume으로 설정·데이터를 관리합니다.

이 책은 코드 작성보다 컨테이너의 전체 개념과 흐름을 이해하고, 단계별로 실습하며 익히는 것을 목표로 합니다. 한 단계씩 학습하며 Docker와 Kubernetes의 큰 흐름을 따라가 보겠습니다.

이것만은 기억하자

→ **컨테이너는 표준 상자입니다.** 말콤 맥린이 화물을 표준 컨테이너에 담아 어디든 같은 방식으로 옮긴 것처럼, IT 컨테이너도 애플리케이션과 그에 필요한 모든 것을 한 상자에 담아 어디서든 같은 결과를 냅니다.

→ **Docker는 공장, 이미지는 설계도, 컨테이너는 찍혀 나온 결과물입니다.** 이미지 하나로 같은 모양의 컨테이너를 여러 개 찍어낼 수 있습니다.

→ **Kubernetes는 항만 관제 센터입니다.** 여러 컨테이너를 자동으로 관리합니다. 사람이 "원하는 상태"만 선언하면 시스템이 그 상태를 유지합니다.

CHAPTER

2

Docker 이해하기

환경을 컨테이너에 담다

이번 챕터가 끝나면

- 컨테이너가 호스트 OS 위에서 어떻게 격리되는지 이해합니다.
- **이미지**를 받아 컨테이너를 띄우고 안으로 들어가 봅니다.
- 컨테이너 통신과 포트포워딩의 동작 흐름을 그림으로 그립니다.
- 컨테이너의 변경과 데이터를 어떻게 보존하는지 익힙니다.

챕터 2. Docker 이해하기

다음 날 저녁, 오픈이는 커피 한 잔을 들고 노트북 앞에 앉았습니다. 전날 본 영상으로 컨테이너와 도커가 왜 필요한지 알게 됐습니다. 환경이 어디서나 같아야 한다는 것, 그 답이 컨테이너라는 것까지는 이해했습니다.

이제 남은 건 직접 실습해 보는 일이었습니다. 오늘의 목표는 하나였습니다.

Docker가 무엇이고 어떻게 돌아가는지, 개념과 기본 사용법을 손에 익힌다.

지금부터 도커가 컨테이너를 어떻게 만들고, 도커를 어떻게 사용하는지 알아보겠습니다.

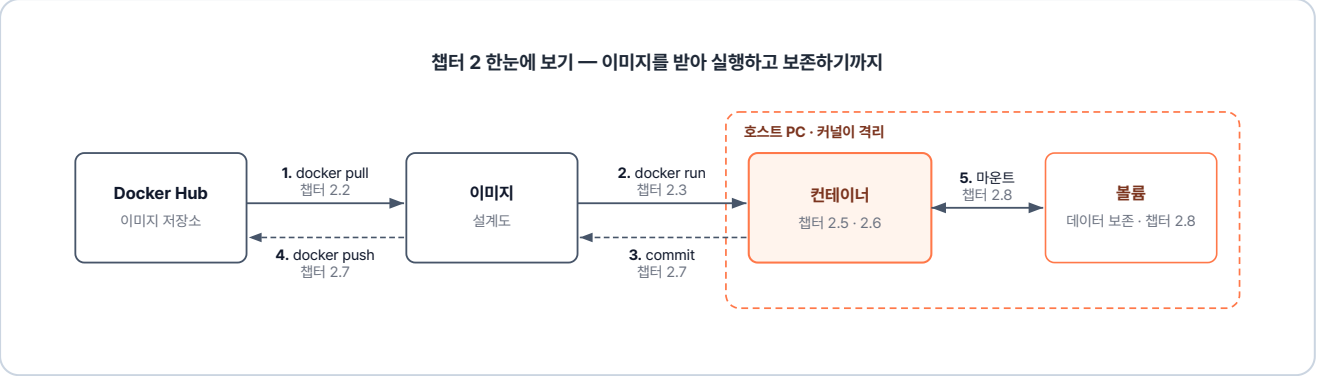


그림 2-1. 이번 챕터 실습 흐름 - 이미지를 받아 실행하고 보존하기까지

준비하기

Docker Desktop 설치

공식 사이트(https://www.docker.com/products/docker-desktop/)에서 OS에 맞는 설치 파일을 받아 설치합니다.

> **설치 전 확인 사항**
>
> - **Windows**: BIOS/UEFI에서 **하드웨어 가상화(Intel VT-x 또는 AMD-V)** 가 켜져 있어야 합니다. 자동으로 바꿀 수 없는 항목이라 설치 전에 직접 확인이 필요합니다. WSL2는 Docker Desktop 설치 과정에서 함께 자동 설치됩니다.
> - **macOS Apple Silicon(M1/M2/M3 등)**: 일부 AMD64 이미지 호환을 위해 **Rosetta 2** 설치를 권장합니다. 터미널에서 `softwareupdate --install-rosetta` 명령으로 설치할 수 있습니다.

2.1 가상화 - 가상머신과 컨테이너

2.1.1 한 주방, 네 명의 요리사

Docker와 컨테이너를 이해하려면 먼저 **가상화**를 알아야 합니다. 주방을 예로 들어 보겠습니다.

한 주방에 요리사 네 명이 있습니다. 네 명이 **동시에** 요리를 하려면 주방을 어떻게 꾸며야 할까요.

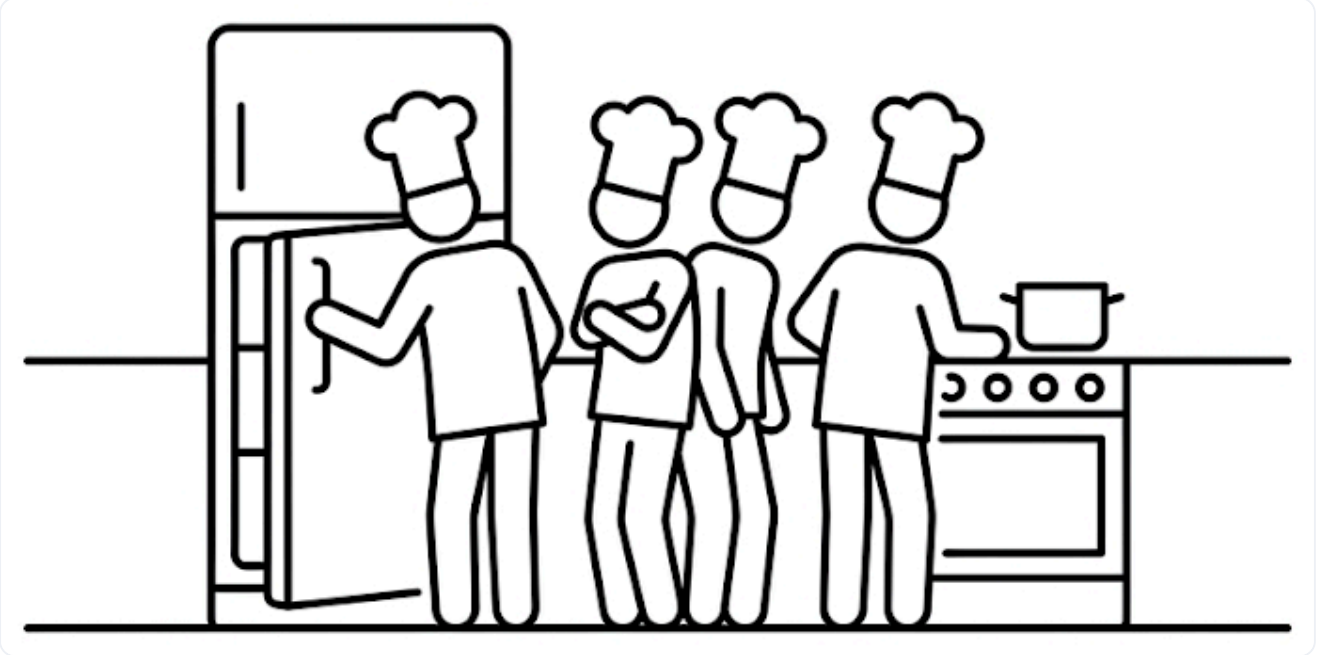


그림 2-2. 한 주방에 네 명의 요리사가 있는 상황

첫 번째 방식은 요리사마다 냉장고, 가스레인지, 조리대를 각자 한 세트씩 구매하는 방법입니다. 이렇게 하면 네 사람이 **완전히 독립된 환경**에서 요리하지만, 같은 설비를 네 번 갖춰야 하니 **비용이 많이 듭니다**.

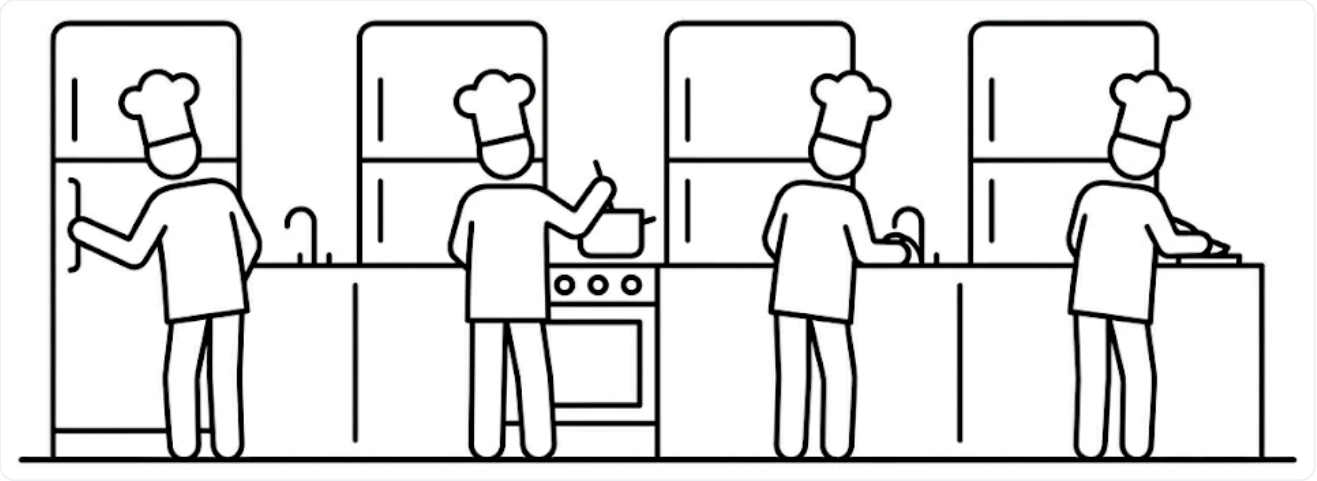


그림 2-3. 주방 설비를 통째로 복제하는 첫 번째 방식

두 번째 방식은 냉장고와 가스레인지 같은 공용 설비는 함께 쓰고, 칼·도마·조리대처럼 개인이 쓸 도구만 각자 챙기는 방법입니다. 그래서 설비를 더 늘리지 않고도 **공간을 효율적으로 나눠 쓸 수 있습니다**.

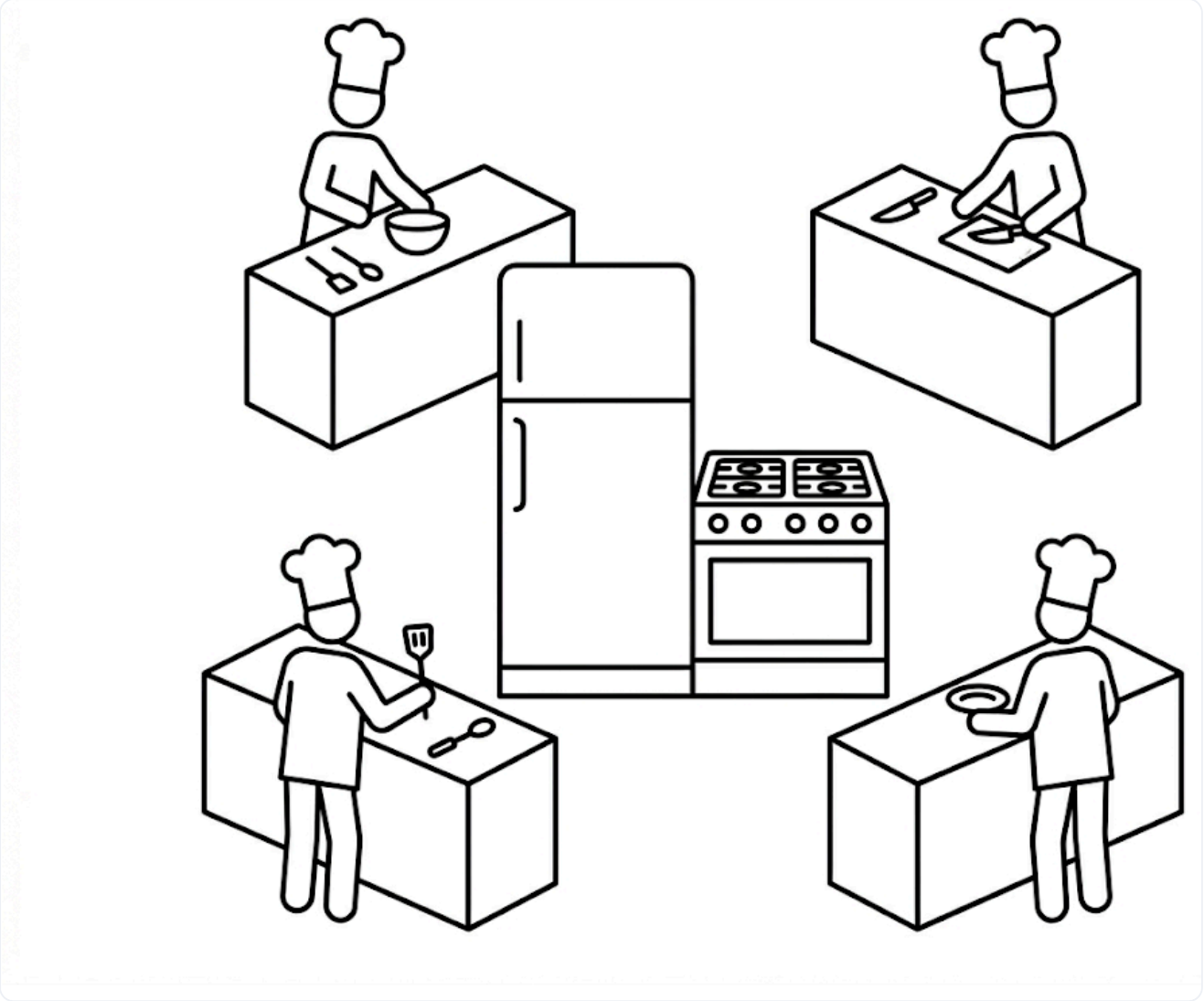


그림 2-4. 공용 설비는 공유하고 개별 공간만 쪼개는 두 번째 방식

이렇게 하나의 주방을 여러 명이 사용하는 것처럼, 하나의 컴퓨터를 여러 대처럼 나눠 쓰는 IT 기술을 **가상화(Virtualization)** 라고 부릅니다. 자원을 아끼면서도 서로 다른 환경을 안전하게 격리하는 것이 목적입니다.

가상화에는 크게 두 가지 방식이 있습니다. 환경을 통째로 복제하는 **하이퍼바이저 가상화**와, 공용 부분을 함께 쓰는 **컨테이너 가상화**입니다. **Docker는 이 컨테이너 가상화를 다루는 도구입니다.**

> **하이퍼바이저 가상화 vs 컨테이너 가상화**
> **하이퍼바이저 가상화(Hypervisor Virtualization)** 는 호스트 OS 위에 **또 다른 OS를 통째로 올립니다.** 이렇게 떠 있는 OS 한 대가 곧 **가상 머신(VM)** 입니다. 격리는 완전하지만 OS 전체가 새로 떠야 해서 무겁고 기동이 느립니다.
> 반면 **컨테이너 가상화(Container Virtualization)** 는 **호스트 OS의 커널을 공유**하고, 파일시스템·네트워크·프로세스 공간만 따로 둡니다. 그 안에서 도는 격리된 단위가 **컨테이너**입니다. 커널을 공유하므로 가볍고 빠르게 뜹니다.

'그러니까 도커는 컴퓨터 OS 하나를 여러 컨테이너가 같이 쓰면서도, 각자 독립된 공간에서 따로 돌게 만드는 거네.'

2.1.2 IT로 옮긴 구조

비유로 알아본 가상화를 실제 구조로 옮기면 다음과 같습니다.

주방의 공용 설비가 호스트 OS라면, 각자의 독립된 조리대는 컨테이너에 비유할 수 있습니다.

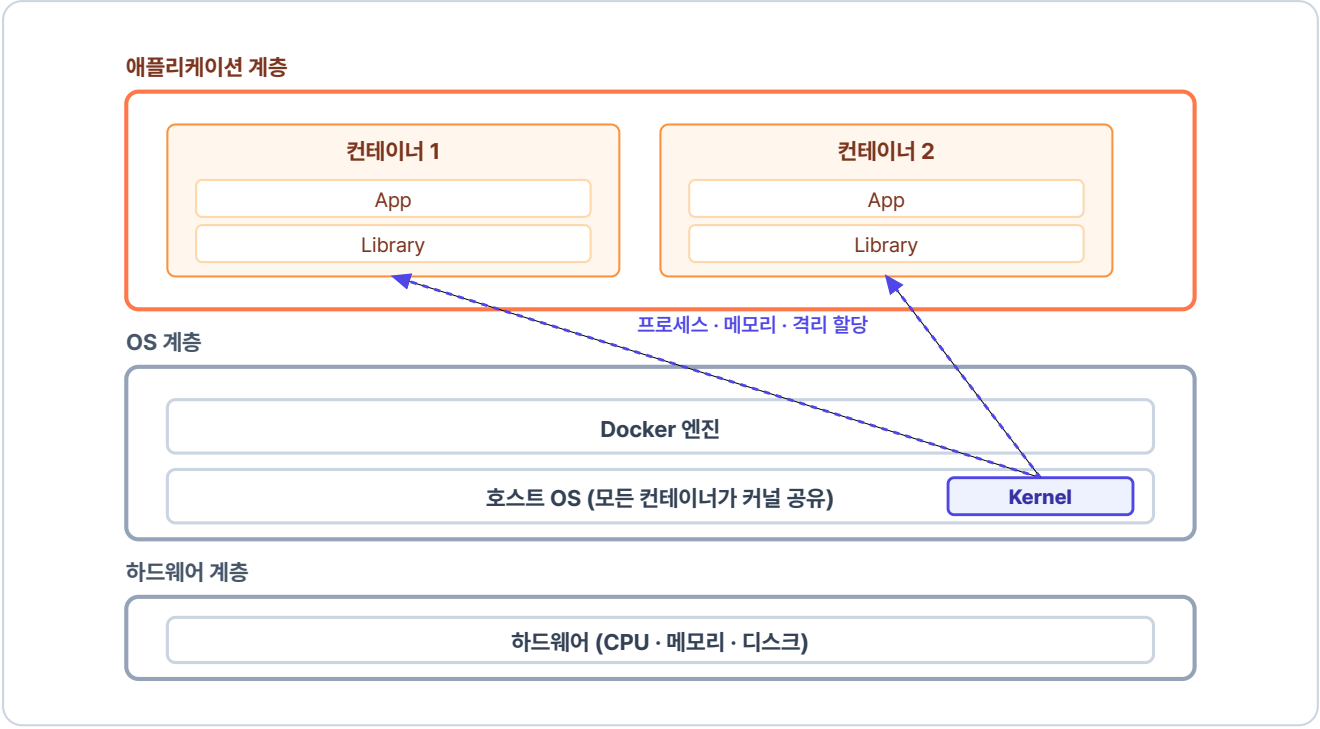


그림 2-5. 컨테이너 가상화의 전체 구조

컨테이너 가상화 구조는 크게 세 계층으로 나뉩니다. 맨 아래는 **하드웨어 계층**, 그 위는 호스트 OS와 Docker 엔진이 있는 **OS 계층**, 맨 위는 독립된 컨테이너들이 실행되는 **애플리케이션 계층**입니다. 각 컨테이너는 실행에 필요한 애플리케이션과 라이브러리를 독립적으로 갖추고 있습니다.

컨테이너마다 이런 독립을 가능하게 하는 것은 호스트 OS의 핵심인 **커널(Kernel)** 입니다. 주방장이 주방을 총괄하듯, 커널은 컨테이너마다 **독립된 프로세스**와 **격리된 메모리 공간**을 할당하고 관리합니다. 이때 호스트의 커널은 **하나뿐이고**, Docker의 모든 컨테이너가 그 **같은 리눅스 커널**을 공유합니다.

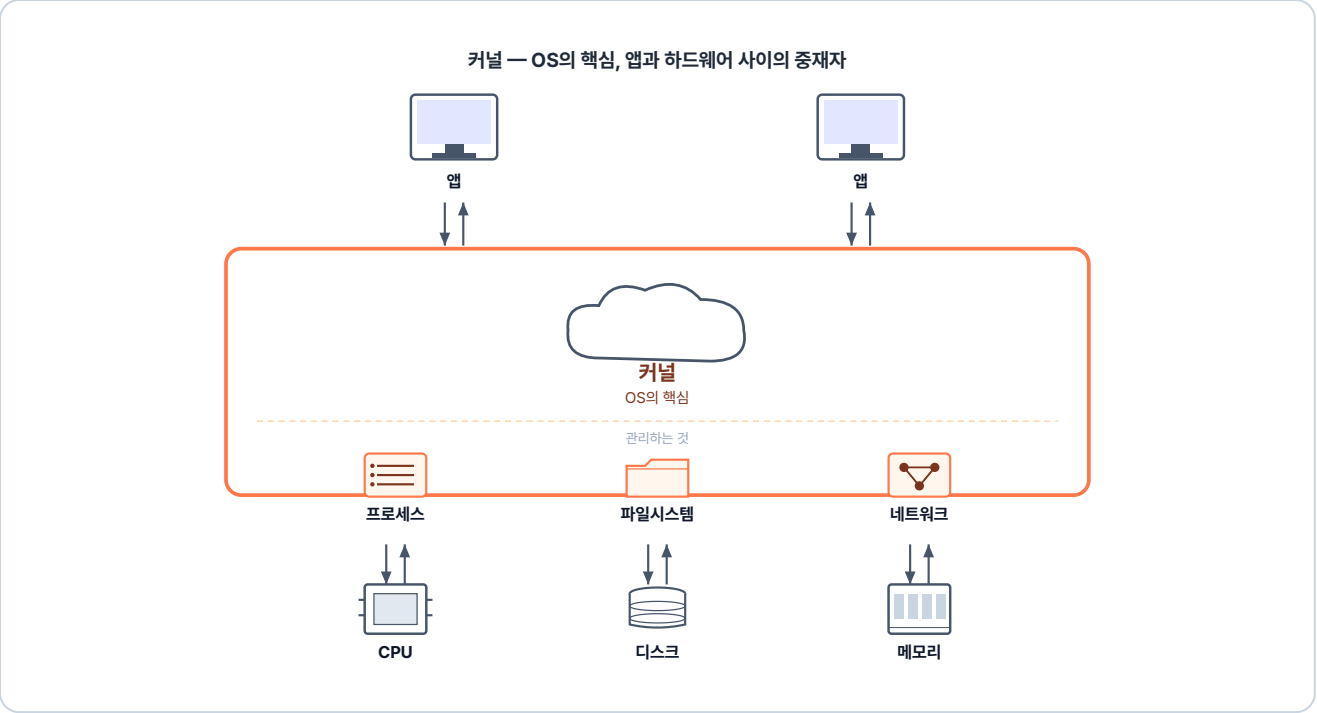


그림 2-6. 커널은 OS의 핵심으로, 앱과 하드웨어 사이에서 시스템 자원을 관리합니다

그래서 컨테이너들은 서로 다른 앱과 라이브러리를 가져도 충돌하지 않고, 한쪽이 런타임을 업그레이드해도 옆 컨테이너는 영향을 받지 않습니다.

> **컨테이너의 격리**
> 사실 컨테이너 안에서 도는 것은 처음부터 격리된 무언가가 아니라, 호스트 OS 위에서 다른 것들과 함께 도는 **평범한 프로세스**입니다. 그대로 두면 한 컨테이너의 프로세스가 다른 컨테이너의 파일을 들여다보거나, 두 컨테이너가 같은 포트를 두고 충돌할 수 있습니다. 그래서 **리눅스 커널**은 컨테이너마다 **파일시스템**(`/bin`, `/lib` 같은 폴더), **네트워크**(IP와 포트), **프로세스**(PID 번호) 세 가지 공간을 따로 만들어 줍니다. 커널이 이렇게 따로 만들어 주는 공간을 **네임스페이스(Namespace)** 라고 합니다. 컨테이너마다 자기 네임스페이스를 갖는 이 상태가 바로 **격리**입니다.

이 구조에서 **Docker 엔진**은 주문을 받는 매니저 역할을 합니다. 우리가 커널을 직접 제어하는 대신 Docker 엔진에게 명령을 내리면, 엔진이 그 명령을 커널에 전달합니다.

2.1.3 컨테이너가 만들어지는 과정

지금까지는 컨테이너가 **무엇인지**를 살펴봤습니다. 그렇다면 이 컨테이너는 **어떻게** 만들어질까요. 명령어를 실행했을 때 무엇이 어떤 순서로 일어나는지 따라가 보겠습니다.

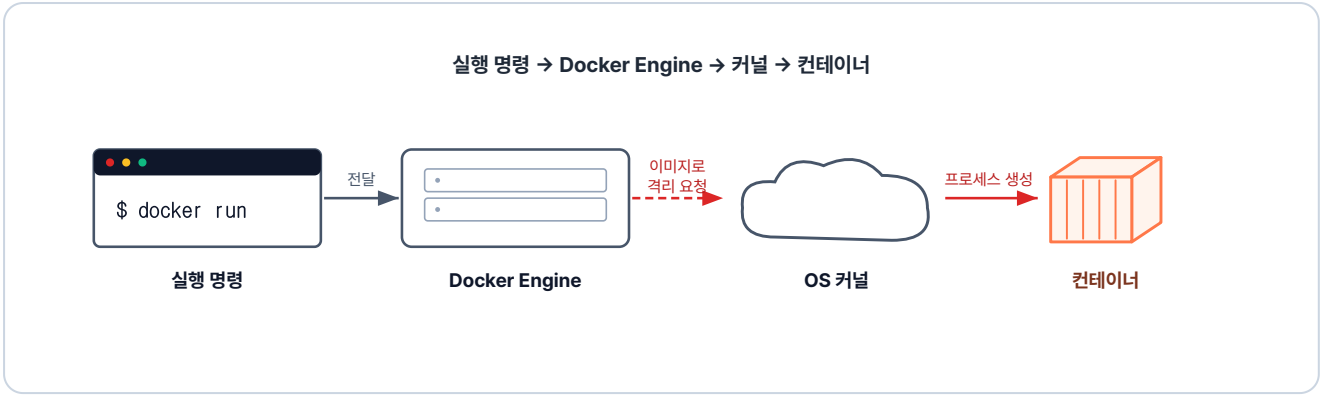


그림 2-7. 컨테이너 실행 명령이 커널의 격리 기능을 타고 컨테이너가 되는 흐름

사용자가 컨테이너 실행 명령을 내리면, **Docker 엔진**은 가장 먼저 로컬에 해당 이미지가 있는지 확인하고, 없으면 Docker Hub에서 자동으로 내려받습니다. 이미지가 준비되면 Docker 엔진은 그 이미지를 바탕으로 격리된 프로세스를 만들어 달라고 호스트 OS의 **커널**에 요청합니다. 요청을 받은 커널은 격리된 공간을 만든 뒤 그 안에서 새 프로세스를 띄웁니다.

이렇게 격리된 공간에서 실행되는 프로세스가 곧 컨테이너입니다.

'그럼 컨테이너를 만드는 재료인 이미지는 뭐지.'

2.2 이미지 - 컨테이너의 설계도

2.2.1 이미지란?

이미지(Image) 는 컨테이너가 무엇을 실행할지를 담은 **설계도**입니다. 이미지에는 무엇을 어떻게 실행할지가 모두 담겨 있고, 그 설계도를 실제로 실행한 것이 컨테이너입니다.

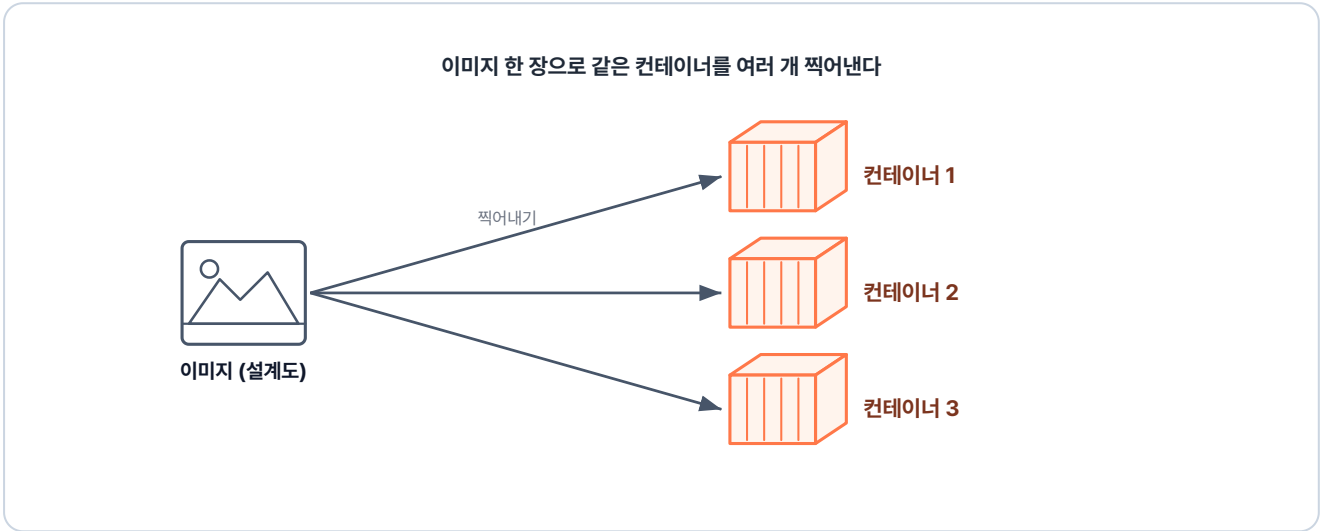


그림 2-8. 하나의 이미지로 여러 컨테이너를 찍어내는 구조

이미지는 **붕어빵 틀**과 같습니다. 붕어빵 틀 하나로 똑같은 붕어빵을 원하는 만큼 찍어내듯, 이미지 하나로 똑같은 컨테이너를 몇 개든 만들 수 있습니다. 같은 이미지로 만든 컨테이너는 어디서 실행하든 그 안의 환경이 똑같습니다. 이 덕분에 **내 PC에서 돌아가던 환경을 서버에 그대로 재현하는 일**이 가능해집니다.

2.2.2 이미지 레이어

이미지는 코드와 라이브러리, 설정을 하나로 묶은 패키지입니다. 하나의 큰 파일이 아니라 **여러 레이어가 층층이 쌓인 구조**입니다.

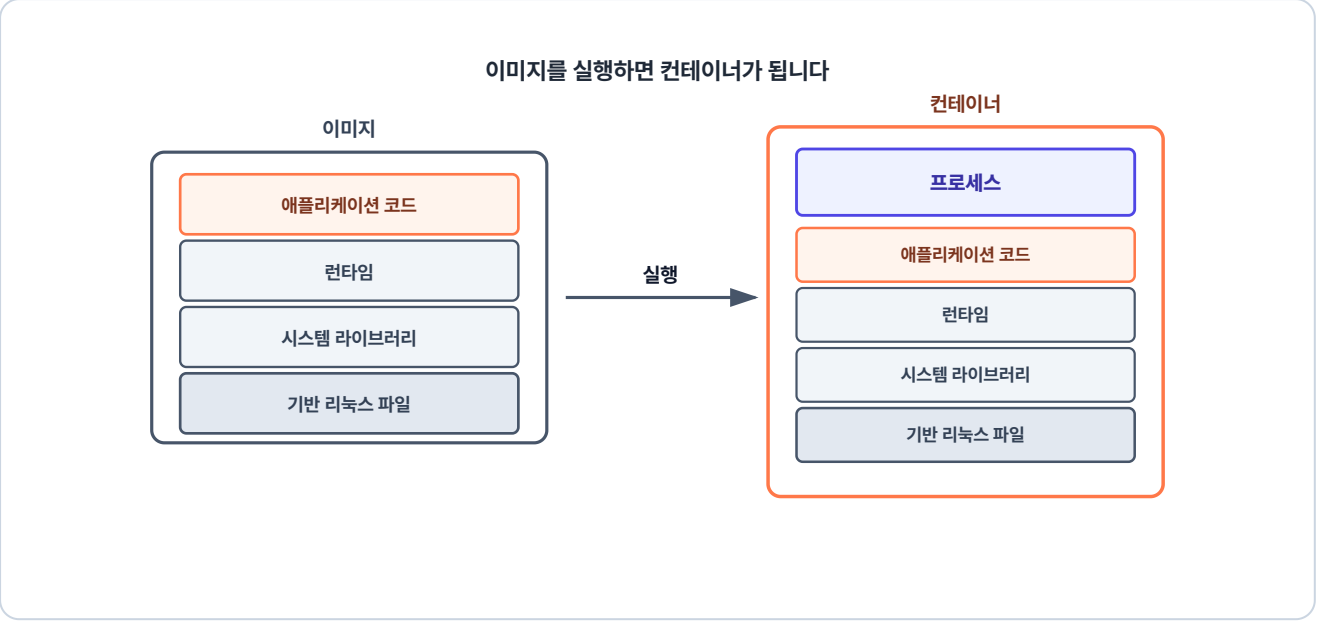


그림 2-9. 이미지를 실행하면 레이어 묶음 위에서 프로그램이 도는 컨테이너가 됩니다

맨 아래 세 레이어(**기반 리눅스 파일·시스템 라이브러리·런타임**)는 여러 앱이 공통으로 쓰는 **거의 바뀌지 않는** 부분이고, 맨 위 레이어(**애플리케이션 코드**)는 그 앱에서만 쓰는 **자주 바뀌는** 코드입니다. 그래서 코드만 바뀐 새 버전을 만들 때도 아래 세 레이어는 그대로 재사용하고, 바뀐 맨 위 레이어만 새로 만듭니다.

이미지를 실행하면, 층층이 쌓인 이 레이어들이 하나로 합쳐져 그 위에서 프로그램이 도는 **컨테이너가 됩니다**.

2.2.3 Docker Hub

이미지는 **Docker Hub**라는 공용 저장소에서 내려받을 수도 있고, 직접 만들어 올릴 수도 있습니다.

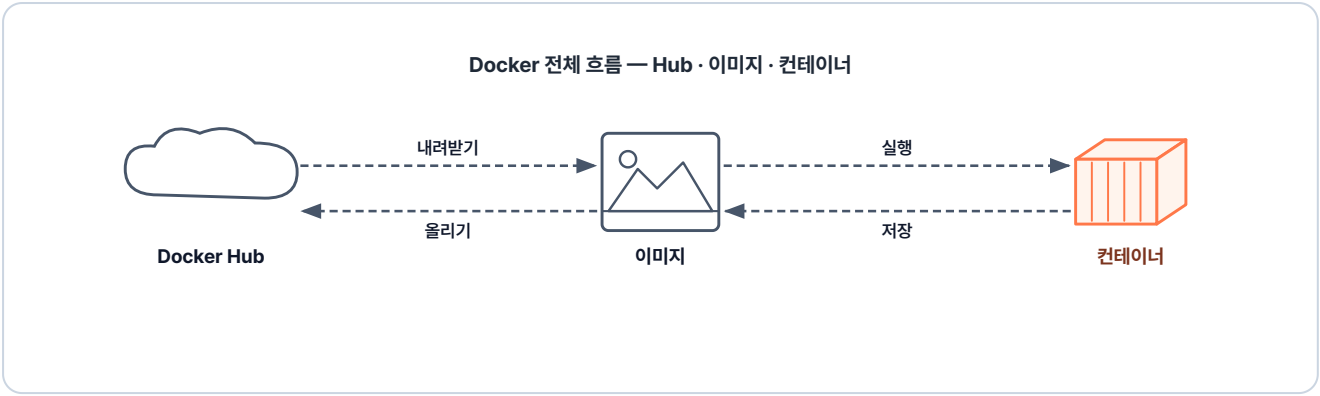


그림 2-10. Docker의 전체 흐름

Docker Hub를 활용하면 내 PC에서 세팅한 환경을 서버에서 재현하는 일도, 팀원끼리 환경을 공유하는 일도 가능합니다.

'도커 컨테이너와 이미지는 알았으니, 이제 직접 실행해 보자.'

2.3 Docker CLI - 첫 컨테이너 띄우기

이제부터는 터미널을 열고 `docker` 명령어를 직접 입력해 컨테이너를 띄워 보겠습니다.

2.3.1 docker pull: 이미지 가져오기

가장 먼저 Docker Hub에서 이미지를 내려받습니다.

터미널 **nginx 이미지 받기**

```
docker pull nginx   # nginx 이미지 다운로드
```

실행 결과

```
$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
9baba07a35b6: Pull complete
ec781dde3f47: Pull complete
0289d65812c3: Pull complete
4174e33a2c9e: Pull complete
6b40784e4837: Pull complete
980067d12da2: Pull complete
f0b77348d9b0: Pull complete
00238b7dc6b2: Pull complete
Digest: sha256:9b3a2dde20b8a1b5e9f2c4a6d8e0b3f5a7c9e1b3d5f7a9b1c3e5f7a9b1c3e5f7
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
$
```

그림 2-11. nginx 이미지 다운로드 결과

명령어 실행이 완료되면 로컬 저장소에 이미지가 저장됩니다. 다만 저장만 됐을 뿐, 아직 실행된 것은 아닙니다.

2.3.2 docker run: 컨테이너 띄우기

이어서 다운받은 이미지로 컨테이너를 실행해 봅니다.

터미널 **nginx 컨테이너 실행 (포그라운드)**

```
docker run nginx   # nginx 컨테이너 실행
```

실행 결과

```
$ docker run nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform
configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in
/etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/17 10:38:22 [notice] 1#1: using the "epoll" event method
```

그림 2-12. nginx 컨테이너 실행

명령어를 실행하자 터미널이 멈추고 아무 반응이 없습니다.

이는 컨테이너가 **포그라운드** 상태로 터미널을 점유하고 있기 때문입니다. 통화 중일 때는 다른 전화를 받을 수 없는 것과 같습니다.

> **포그라운드(Foreground)**: 프로세스가 터미널을 독점하는 상태입니다. 프로세스가 끝나거나 강제 종료될 때까지 그 터미널로 다른 명령을 내릴 수 없습니다.

우선 터미널에 `CTRL + C`를 입력해 빠져나옵니다.

2.3.3 docker run -d: 백그라운드로 띄우기

'컨테이너는 띄운 채로 터미널도 같이 쓰려면 어떻게 해야 하지?'

이 문제를 풀어 주는 게 `-d` 옵션입니다. `-d` 옵션을 사용하면 백그라운드 상태에서 프로세스를 실행할 수 있습니다.

터미널 **백그라운드로 컨테이너 띄우기**

```
docker run -d nginx   # -d: detached, 백그라운드 실행
```

실행 결과

```
$ docker run -d nginx
4932c46c66589dc651efd5e50a8773d6d077f8a1e005f177385f8e0cc115ad7d
```

그림 2-13. 백그라운드 실행 결과

이번에는 컨테이너 ID가 찍히고 곧바로 프롬프트가 돌아왔습니다. 방금 프롬프트가 돌아오지 않던 것과 달리, `-d`로 띄우면 컨테이너는 백그라운드에서 돌고 터미널은 그대로 쓸 수 있습니다.

> **백그라운드(Background)**: 프로세스가 터미널을 점유하지 않고 뒤에서 실행되는 상태입니다. 프로세스가 작동하는 중에도 같은 터미널 창에서 다른 명령어를 계속 입력하고 실행할 수 있습니다.

'터미널을 계속 쓰려면 백그라운드로 실행하면 되겠네.'

2.3.4 docker ps: 실행 중인 컨테이너 확인

백그라운드에서 돌고 있는 컨테이너가 실제로 살아 있는지 확인합니다.

터미널 **실행 중 컨테이너 확인**

```
docker ps    # 실행 중인 컨테이너 목록
```

실행 결과

```
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
4932c46c6658 nginx "/docker-entrypoint...." 1 second ago Up Less than a second 80/tcp
dazzling_carso
```

그림 2-14. 컨테이너 목록 조회

목록에서 방금 띄운 nginx를 확인할 수 있습니다.

2.3.5 자주 쓰는 명령어

다음은 Docker에서 자주 사용하는 명령어입니다.

명령어	설명
docker pull <이미지명>	Docker Hub에서 이미지 다운로드
docker images	로컬에 저장된 이미지 목록
docker logs <컨테이너ID>	컨테이너 로그 출력
docker ps -a	종료된 컨테이너 포함 전체 목록
docker stop <컨테이너ID>	실행 중인 컨테이너 종료
docker rm <컨테이너ID>	종료된 컨테이너 삭제
docker rmi <이미지ID>	이미지 삭제

지금까지 도커 명령어로 이미지를 다운로드하고 컨테이너를 직접 실행해 봤습니다. 하지만 컨테이너 내부에서 프로그램이 정상적으로 작동하고 있더라도, 외부에서 보낸 요청이 안으로 들어오지 못하면 서비스를 정상적으로 이용할 수 없습니다.

'바깥에서 보낸 요청은, 이 컨테이너 안까지 어떻게 들어오는 거지?'

컨테이너를 실제 운영 환경에서 활용하기 위해서는 외부와 데이터를 주고받는 네트워크 흐름을 이해해야 합니다.

2.4 컨테이너 통신

외부에서 보낸 요청이 컨테이너 안까지 어떻게 전달되는지, 또 컨테이너끼리 어떻게 데이터를 주고받는지 살펴보겠습니다.

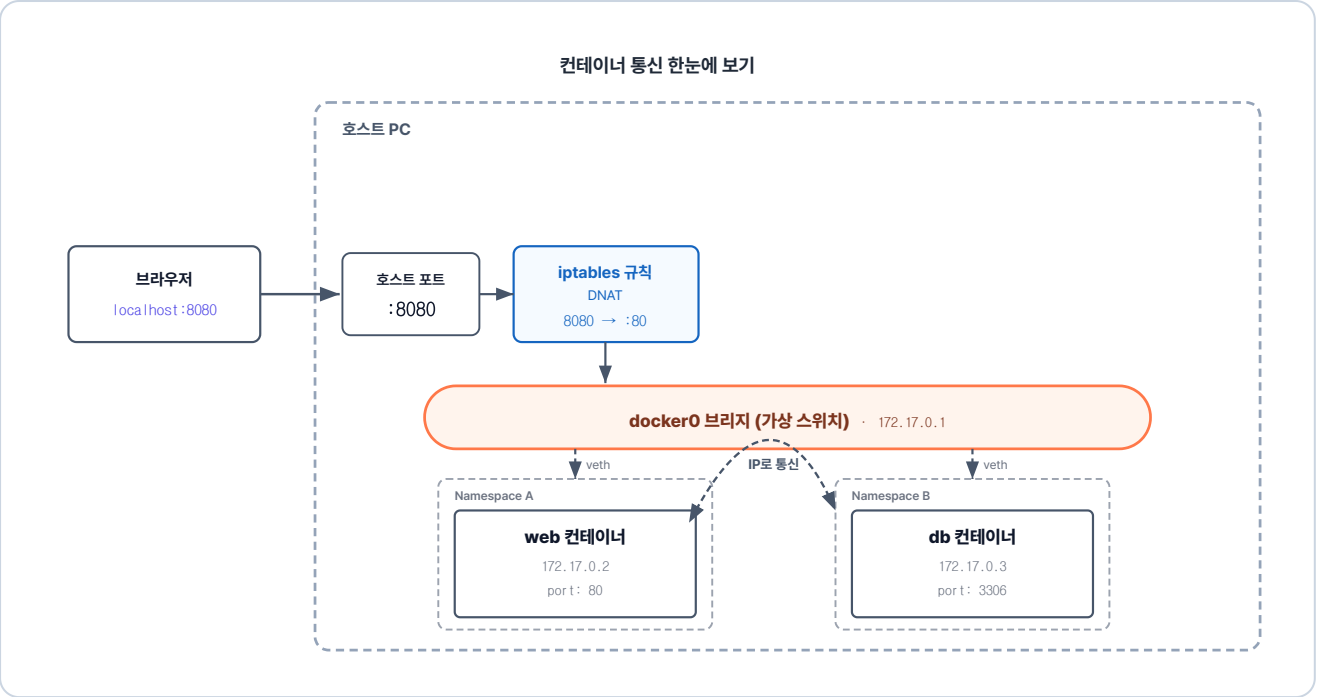


그림 2-15. 컨테이너 통신 한눈에 보기 - 외부 진입과 컨테이너끼리 IP 통신

여기서는 컨테이너 네트워크를 세 가지로 나눠 살펴보겠습니다. **컨테이너끼리 연결하는 방법, 외부 요청이 컨테이너로 들어오는 방법, 이름으로 컨테이너를 찾는 방법**입니다.

2.4.1 컨테이너 간 통신

푸드코트에 여러 식당이 있습니다. 각 식당은 서로의 주방에 들어갈 수 없습니다. 대신 각 주방의 **주방 모니터**는 **통신 케이블**로 벽에 걸린 **주문 전광판**에 연결되며, 서로의 주문 현황을 확인할 수 있습니다.

컨테이너 간 통신도 비슷합니다. 각 컨테이너는 자기만의 격리된 네트워크인 **네트워크 Namespace**를 갖고, **veth pair**라는 가상 케이블로 **docker0** 브리지에 연결됩니다. 한 컨테이너가 다른 컨테이너의 IP로 호출하면 **데이터는 docker0를 통해 전달됩니다.**

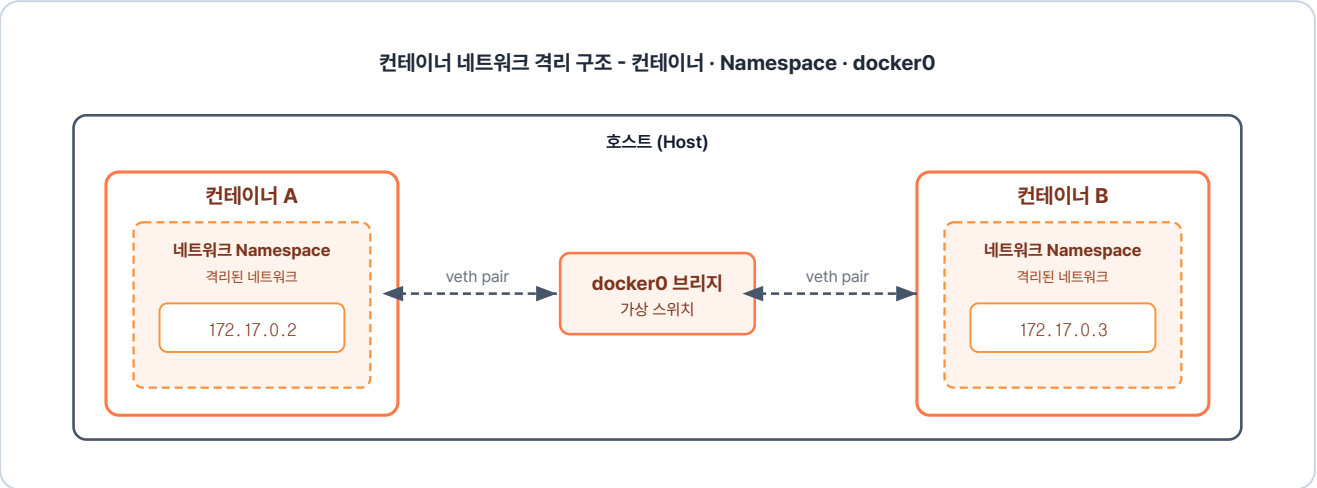


그림 2-16. 컨테이너 네트워크 격리 구조

구성 요소	하는 일
네트워크 Namespace	컨테이너마다 따로 가진, 자기 IP·포트를 쓰는 격리된 네트워크
veth pair	네트워크 Namespace를 docker0에 잇는 가상 케이블 한 쌍
docker0	같은 브리지에 묶인 컨테이너 사이로 패킷(데이터 묶음)을 중계하는 가상 브리지

2.4.2 외부에서 들어오기 - 입구 키오스크

푸드코트에서 손님은 식당으로 들어갈 수 없습니다. 대신 **키오스크**를 통해 식당에 주문을 요청합니다. 짜장면을 선택하면 A식당, 돈까스를 선택하면 B식당으로 키오스크의 규칙에 따라 주문이 전달됩니다.

외부에서 컨테이너로의 요청도 바로 접근이 불가능합니다. 그래서 키오스크의 역할을 하는 **포트포워딩**을 사용해야 합니다.

> **왜 컨테이너 IP로 바로 접근할 수 없을까**
>
> 컨테이너 IP는 실행할 때마다 새로 할당되어, 외부에서 접속할 고정된 주소가 없습니다. 게다가 Windows·macOS에서는 컨테이너가 Docker Desktop이 실행하는 리눅스 가상 머신 안에서 동작합니다. 컨테이너 IP는 이 가상 머신 내부 네트워크에만 있는 주소라, 그 바깥에 있는 호스트는 해당 IP로 도달할 수 없습니다.

예를 들어 `docker run -p 8080:80` 을 실행하면 **iptables(DNAT)** 에 규칙이 작성됩니다. 그리고 외부에서 8080 포트로 요청이 오면 이 규칙에 의해 80 포트를 가진 컨테이너로 연결됩니다.

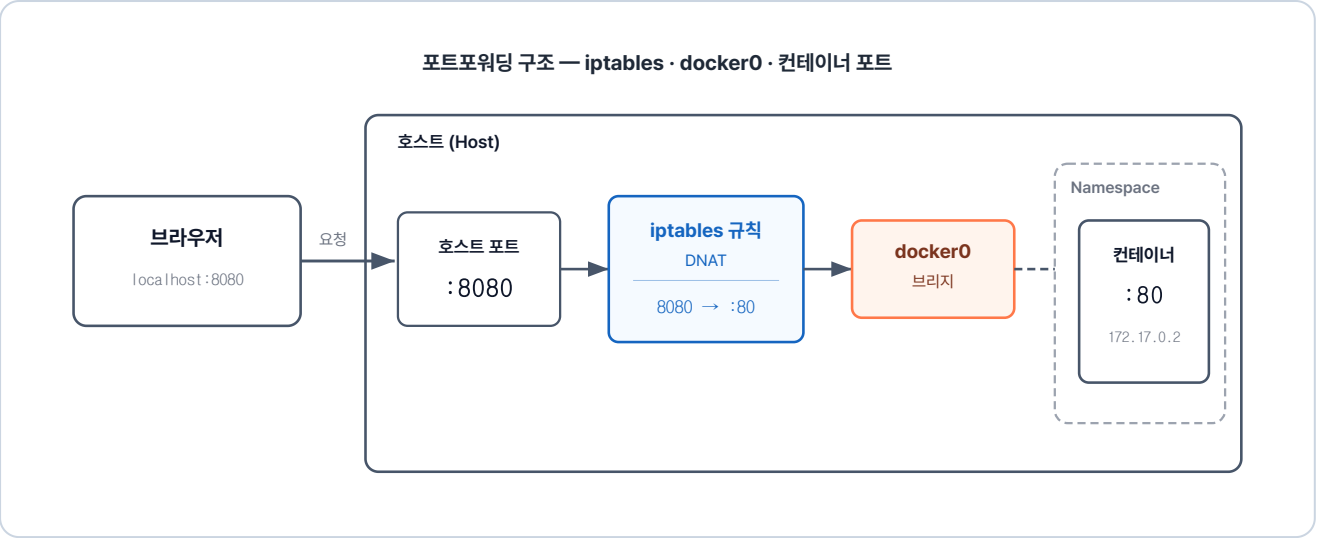


그림 2-17. 호스트 포트가 컨테이너 포트로 연결되는 구조

구성 요소	하는 일
포트포워딩	외부 공개 포트로 온 요청을 컨테이너 포트로 넘기는 메커니즘
iptables 규칙(DNAT)	외부 공개 포트로 온 요청의 목적지를 컨테이너의 IP·포트로 바꾸는 규칙
컨테이너 포트	컨테이너 안에서 앱이 듣고 있는, 요청이 최종 도착하는 포트

2.4.3 이름으로 찾기 - 홀의 안내 지도

여러 식당 중 손님은 자신이 원하는 식당의 위치를 알 수 없습니다. 이때 **안내 지도**를 보면 식당의 이름을 보고 위치를 찾을 수 있습니다.

도커에는 **Docker DNS** 기능이 있습니다. 새로운 컨테이너가 실행되면 컨테이너의 이름과 IP를 Docker DNS에 등록합니다. 그리고 **한 컨테이너가 다른 컨테이너를 이름으로 호출하면, Docker DNS가 이름에 맞는 IP를 돌려줍니다.** 이름으로 통신이 가능하기 때문에 IP가 바뀌어도 같은 이름으로 통신할 수 있습니다.

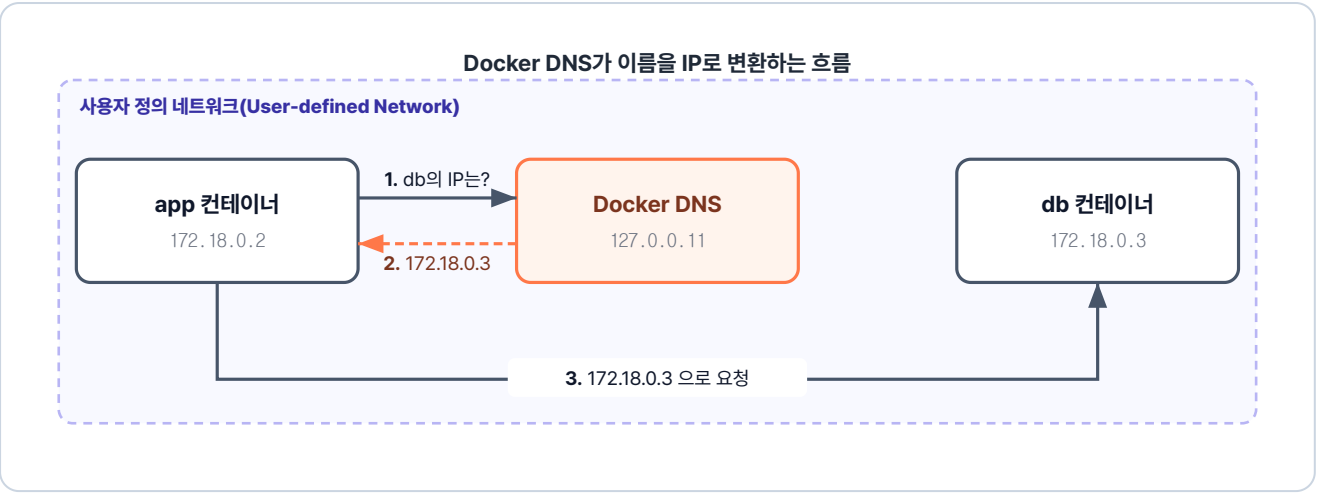


그림 2-18. Docker DNS가 이름을 IP로 변환

구성 요소	하는 일
Docker DNS	컨테이너 이름을 IP로 자동 변환해 주는 내장 DNS(Domain Name System) 서버

다만 내장 DNS는 **사용자 정의 네트워크(User-defined Network)** 에서만 동작합니다. 기본 네트워크(docker0)에서는 이름으로 찾을 수 없기 때문에, 직접 네트워크를 만들어 컨테이너들을 연결해야 합니다. 구체적인 생성 방법은 다음 챕터에서 다룹니다.

컨테이너 통신을 살펴봤으니, 이제 컨테이너 내부를 들여다볼 차례입니다. 컨테이너 안은 작은 리눅스 환경과 같아서, 가장 먼저 기본 리눅스 명령어부터 알아보겠습니다.

2.5 컨테이너 내부 - 격리된 작은 리눅스

리눅스를 연습하려면 리눅스 환경이 필요합니다. **Ubuntu**는 가장 널리 쓰이는 배포판 중 하나라 자료가 풍부하고, Docker Hub에 공식 이미지가 올라와 있어서 바로 받아 실행할 수 있습니다.

포트포워딩을 사용해 ubuntu 컨테이너 내부로 들어가 보겠습니다. 외부에서 80 포트로 요청하면 컨테이너 내부의 80 포트로 요청이 전달됩니다.

터미널 **ubuntu 컨테이너 안으로 들어가기**

```
# -i 키보드 입력 전달, -t 터미널 화면 제공, -p 80:80 포트포워딩
docker run -it -p 80:80 ubuntu
```

실행결과

```
$ docker run -it -p 80:80 ubuntu
root@c0b6dd5261d8:/#
```

그림 2-19. Ubuntu 컨테이너 안으로 들어간 상태

프롬프트가 `root@...#` 모양으로 바뀌었습니다. 방금까지 호스트의 터미널이었는데 이제는 컨테이너 안의 쉘입니다.

쉘(Shell): 사용자와 커널 사이에서 명령을 주고받는 프로그램입니다. 사용자가 쉘에 명령을 내리면 쉘이 커널에 그 명령을 전달하고 커널이 프로세스를 만들어 실제 작업을 수행합니다.

2.5.1 자주 쓰는 리눅스 명령어

자주 쓰는 리눅스 명령어는 다음과 같습니다.

용도	명령	설명
위치 확인	**pwd**	현재 위치
이동	**cd <경로>**	폴더 이동
목록	**ls, ls -la**	파일/폴더 목록 (숨김 포함)
폴더 생성	**mkdir <이름>**	폴더 만들기
파일 생성	**touch <이름>**	빈 파일
복사	**cp <원본> <사본>**	파일 복사
이동/이름변경	**mv <원본> <대상>**	이동 또는 rename
삭제	**rm <파일>, rm -r <폴더>**	삭제
패키지 설치	**apt update && apt install -y <이름>**	패키지 설치
프로세스	**ps -ef, kill <PID>**	실행 중 프로세스 / 종료
검색	**find <경로> -name <이름>**	파일 검색

용도	명령	설명
로그	**tail -n <수> <파일>**	마지막 N줄

경로를 쓸 때는 **절대 경로**와 **상대 경로**를 구분해야 합니다.

> **절대 경로 / 상대 경로**
> 둘의 차이는 **어디서부터 시작해서 읽느냐**에 있습니다.
>
> - **절대 경로**는 **루트 경로(/)** 부터 시작해서 목적지까지의 전체 주소를 명시합니다. 현재 위치가 어디든 같은 곳을 가리킵니다. 리눅스에서는 결과적으로 `/`로 시작하는 형태가 됩니다. 예: `/bin`, `/home/ubuntu/docs`
> - **상대 경로**는 **현재 디렉터리**를 기준으로 목적지까지의 경로를 나타냅니다. 현재 위치에 따라 같은 표기가 다른 곳을 가리킬 수 있습니다. 예: `bin`(현재 디렉터리 아래 bin), `../etc`(상위 디렉터리의 etc)
> - 경로에서 `.`은 **현재 디렉터리**, `..`은 **상위 디렉터리**를 뜻합니다.

2.5.2 실습: 컨테이너 안에 nginx 깔아보기

앞서 이미지로 실행했던 nginx를, 이번엔 컨테이너 안에서 직접 설치해 보겠습니다. 컨테이너는 최소 구성이라 필요한 패키지는 직접 설치해야 합니다.

터미널 **컨테이너 내부 - 패키지 목록 갱신**

```
apt update           # 최신 패키지 목록으로 업데이트
apt install -y nginx # nginx 설치
nginx                # nginx 실행
```

실행결과

```
root@c0b6dd5261d8:/# apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
7 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@c0b6dd5261d8:/# apt install -y nginx
Need to get 521 kB of archives.
After this operation, 1352 kB of additional disk space will be used.
Fetched 521 kB in 2s (217 kB/s)
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.24.0-2ubuntu7.6_amd64.deb ...
Unpacking nginx (1.24.0-2ubuntu7.6) ...
Setting up nginx (1.24.0-2ubuntu7.6) ...
root@c0b6dd5261d8:/# nginx
```

그림 2-20. nginx 설치와 실행 결과

설치 로그가 마무리되며 Nginx 설치가 끝났습니다.

네트워크 확인 도구인 `net-tools`를 추가로 설치하고 Nginx가 어느 포트에서 대기 중인지 확인합니다.

터미널 **컨테이너 내부 - net-tools 설치**

```
apt install -y net-tools   # net-tools: 네트워크 확인 도구 모음
netstat -nlpt              # 열려 있는 TCP 포트 확인
```

실행결과

```
root@c0b6dd5261d8:/# netstat -nlpt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address Foreign Address State PID/Program name
tcp 0 0 0.0.0.0:80 0.0.0.0:* LISTEN 348/nginx: master p
tcp6 0 0 :::80 :::* LISTEN 348/nginx: master p
```

그림 2-21. 포트 상태 확인

80 포트가 열려 있는 것을 확인합니다. 다음은 브라우저로 접속할 차례입니다. 주소창에 `localhost:80`을 칩니다.



그림 2-22. nginx 환영 페이지 응답

nginx 환영 페이지가 떴습니다. 컨테이너를 실행할 때 포트포워딩으로 포트를 열어 둔 덕에, 컨테이너 안의 nginx와 브라우저가 연결됐기 때문입니다.

2.5.3 vim으로 파일 편집

서버 설치를 마쳤더라도 운영하다 보면 설정 파일을 수정해야 할 일이 생기기 마련입니다. 리눅스에서 파일을 편집하려면 텍스트 편집기가 필요한데, 여기서는 널리 쓰이는 **Vim**을 설치합니다.

터미널 **컨테이너 내부 - vim 설치**

```
apt install -y vim   # 터미널 편집기 vim 설치
```

> **설치 중 시간대 선택 화면**
> apt install vim 도중 Geographic area와 Time zone을 선택하는 화면이 나옵니다. 이때 **Asia**, **Seoul**을 각각 선택합니다.

vim의 사용 명령어는 다음과 같습니다.

단계	동작	키
1	파일 열기/생성	**vim <파일명>**

단계	동작	키
2	입력 모드	**i**
3	일반 모드로	**ESC**
4	저장 후 종료	**:wq**

vim으로 간단한 파일을 만들어 봅니다.

터미널 **컨테이너 내부 - vim으로 파일 생성**

```
vim test1.txt   # test1.txt 파일 생성
```

`i`를 눌러 입력 모드로 전환하고 내용을 작성한 뒤 `ESC` → `:wq`로 저장합니다.

실행결과

```
hello docker
~
~
~
~
:wq
```

그림 2-23. vim 편집 화면

저장이 잘 됐는지 `cat`으로 내용을 확인합니다.

터미널 **컨테이너 내부 - 파일 내용 확인**

```
cat test1.txt   # 파일 내용 출력
```

실행결과

```
root@c0b6dd5261d8:/# cat test1.txt
hello docker
```

그림 2-24. 파일 내용 출력

실습을 마친 후 컨테이너 안에서 빠져나오기 위해 `exit`를 입력합니다. 그렇게 터미널로 돌아온 뒤 `docker ps`로 실행 중인 컨테이너를 확인하면, 방금까지 들어가 있던 ubuntu가 목록에 없습니다.

'컨테이너를 빠져나왔는데 같이 종료가 되네.'

방금처럼 컨테이너가 언제 종료되고 언제 계속 실행되는지 이해하려면, 그 중심에서 도는 **메인 프로세스**를 알아야 합니다.

2.6 컨테이너의 메인 프로세스

2.6.1 메인 프로세스란?

메인 프로세스는 도커 컨테이너가 돌리는 중심 프로그램으로, 컨테이너 안에서 가장 먼저 실행되어 **프로세스 ID(PID) 1번**을 받습니다. 자동차가 엔진이 도는 동안에만 달릴 수 있듯, 컨테이너 역시 **이 메인 프로세스가 멈추면 그 즉시 함께 종료됩니다**.



그림 2-25. 컨테이너 안에서 도는 중심 프로그램이 메인 프로세스입니다

2.6.2 메인 프로세스에서 빠져나오기

`exit`를 입력했을 때 컨테이너가 종료된 상황을 한 번 더 재연해 보겠습니다. 컨테이너에 `dead`라는 이름을 붙여 실행한 뒤, 안에서 `exit`로 빠져나옵니다.

터미널 **exit로 빠져나오는 시나리오**

```
docker run -it --name dead ubuntu   # 실행 후 exit로 나옴
exit
```

```
docker ps                                   # 종료 여부 확인
```

실행결과

```
root@f9a2b3c1d4e5:/# exit
exit
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

그림 2-26. exit로 빠져나오면 컨테이너가 종료

목록에 dead는 없습니다. `exit`**가 메인 프로세스를 종료하면서 컨테이너까지 함께 멈췄기 때문입니다.** 그런데 컨테이너를 빠져나오는 방법이 `exit`만 있는 것은 아닙니다.

`CTRL+P → CTRL+Q`를 누르면 메인 프로세스는 그대로 둔 채 터미널 연결만 끊을 수 있습니다. `alive`라는 이름으로 컨테이너를 실행한 뒤, 이 키로 빠져나옵니다.

터미널 **CTRL+P → CTRL+Q로 빠져나오는 시나리오**

```
docker run -it --name alive ubuntu   # CTRL+P → CTRL+Q로 나옴
docker ps                            # 종료 여부 확인
```

실행결과

```
root@17943f60b5cd:/#
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
17943f60b5cd ubuntu "/bin/bash" Less than a second ago Up Less than a second alive
```

그림 2-27. CTRL+P → CTRL+Q로는 컨테이너가 유지

이번에는 alive가 목록에 남아 있습니다. `CTRL+P → CTRL+Q`**는 메인 프로세스를 건드리지 않고 터미널 연결만 끊기 때문에 컨테이너는 그대로 살아 있습니다.**

2.6.3 메인 프로세스가 유지되는 조건

컨테이너가 내부 프로세스를 실행하는 방식은 이미지마다 다릅니다. 앞서 nginx를 `-d`로 백그라운드에서 실행했습니다. 같은 방식을 ubuntu에도 적용해 두 이미지가 어떻게 다른지 비교해 보겠습니다.

터미널 **ubuntu를 -d만으로 실행**

```
docker run -d ubuntu   # ubuntu를 백그라운드로 실행
docker ps              # 실행 중인 컨테이너 목록 확인
```

실행결과

```
$ docker run -d ubuntu
0b74e66f5b0f3f4cb30b353503f2ef0c162e63f19be512e938f28e07d01dc1e7
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

그림 2-28. ubuntu는 -d만으로는 바로 종료

'nginx는 백그라운드에서 실행이 됐는데, ubuntu는 안 되는 건가?'

ubuntu는 백그라운드로 실행되자마자 종료되어 실행 목록에 보이지 않습니다. 같은 -d 옵션인데 nginx와 결과가 다른 이유는 이미지마다 메인 프로세스가 다르기 때문입니다.



그림 2-29. 같은 PID 1 자리에 이미지마다 다른 프로그램이 메인 프로세스로 들어갑니다

ubuntu가 바로 종료된 이유: ubuntu 이미지의 메인 프로세스는 사용자의 명령을 입력받는 **셸(bash)** 입니다. `-it` 옵션을 주어 터미널을 연결하면 입력을 기다리기 위해 컨테이너가 계속 유지되지만, `-d` 옵션만 주면 입력을 받을 터미널이 없어져 버립니다. 즉, 셸이 더 이상 할 일이 없다고 판단하여 컨테이너가 그 즉시 종료됩니다.

nginx와의 차이: 반면 웹 서버인 nginx의 목적은 사용자의 입력을 받는 것이 아니라 외부에서 들어오는 **웹 요청**을 기다리는 것입니다. 터미널이 없어도 **요청 대기**라는 명확한 할 일이 멈추지 않으므로 종료되지 않고 계속 떠 있습니다.

ubuntu처럼 `-d` 만으로는 바로 종료되는 이미지는, `-dit` 옵션으로 터미널을 함께 열어야 백그라운드에서 계속 실행됩니다.

결국 컨테이너가 백그라운드에서 계속 살아 있으려면, **메인 프로세스가 멈추지 않고 실행되고 있어야 합니다.** 그러려면 두 가지 방법이 있습니다. **메인 프로세스가 입력을 기다리도록 터미널을 함께 열어 두거나, 메인 프로세스를 계속 실행되는 다른 프로그램으로 바꾸면 됩니다.**

2.6.4 메인 프로세스 바꾸기 - CMD

이번에는 `docker run <이미지> <CMD 옵션>` 형태로 메인 프로세스를 직접 바꿔 보겠습니다. ubuntu의 기본 메인 프로세스인 bash 대신, 1000초 동안 멈춰 있는 **sleep 1000** 을 지정합니다.

터미널 **sleep을 메인 프로세스로 실행**

```
docker run -d ubuntu sleep 1000   # bash 대신 sleep을 메인으로
docker ps                         # 1000초 동안 살아 있음
```

실행결과

```
$ docker run -d ubuntu sleep 1000
7aca4e791643d3a39917aecc97868a62f5c1f6fa3df1ac63248e703650835cd3
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
7aca4e791643 ubuntu "sleep 1000" Less than a second ago Up Less than a second goofy_bose
```

그림 2-30. CMD를 sleep으로 바꾸면 유지

이제 ubuntu가 목록에 남아 있습니다. 메인 프로세스가 bash에서, 터미널이 없어도 1000초 동안 멈춰 있는 sleep으로 바뀌었기 때문입니다. 이렇게 **기본 메인 프로세스를 바꾸는 설정을 CMD**라고 합니다.

2.6.5 실행 중인 컨테이너에 들어가기 - attach와 exec

이제 백그라운드로 실행 중인 컨테이너 안으로 들어가 보겠습니다. 들어가는 명령은 **attach**와 **exec**, 두 가지입니다.

먼저 attach를 알아보겠습니다. attach는 **메인 프로세스에 직접 연결**하는 방식입니다.

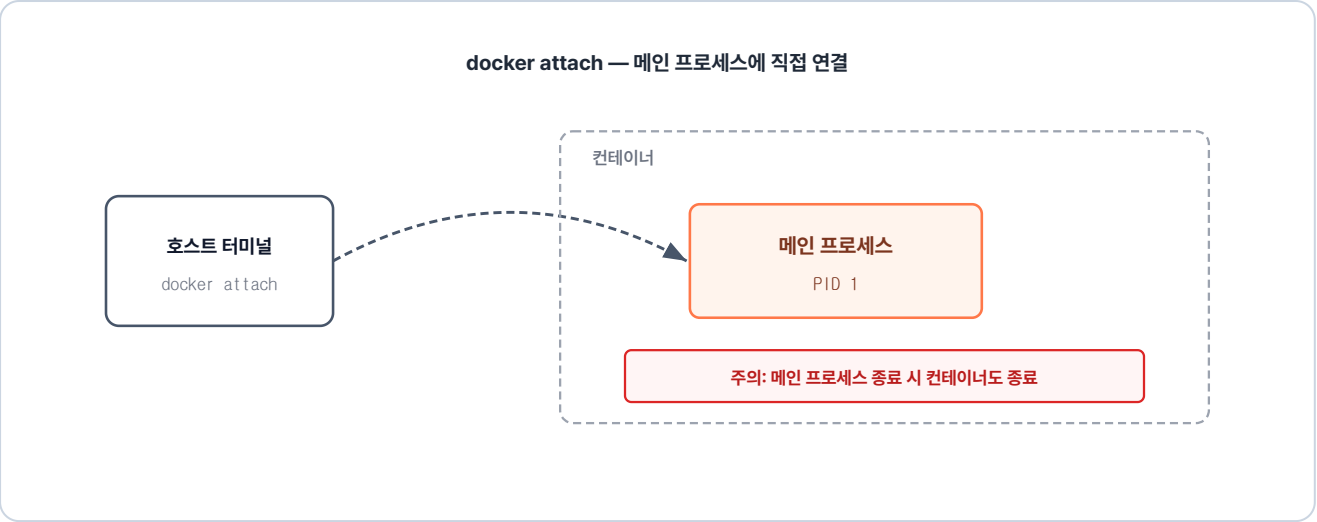


그림 2-31. attach는 이미 떠 있는 PID 1 프로세스에 터미널을 직접 연결하는 방식

확인하기 위해 ubuntu를 `-dit`로 백그라운드에서 실행합니다. 실행 중인 컨테이너의 ID를 확인한 뒤 `docker attach <컨테이너ID>`로 접속합니다.

터미널 **-dit 조합으로 ubuntu 살려 두기**

```
docker run -dit ubuntu  # -d(백그라운드)와 -it(터미널)를 합쳐 bash 유지
docker ps               # 실행중인 컨테이너 확인
docker attach d2b1      # attach로 접근
```

실행결과

```
$ docker run -dit ubuntu
d2b18acc2cf05a40644f0ca8536cb50a0ff560a1be0c9ce2fbf5ddbbba0e729c
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
d2b18acc2cf0 ubuntu "/bin/bash" Less than a second ago Up Less than a second angry_engelbart
$ docker attach d2b1
root@d2b18acc2cf0:/#
```

그림 2-32. attach로 접근

메인 프로세스인 bash의 터미널이 그대로 나타났습니다. 메인 프로세스에 연결됐으므로, 여기서 `exit`로 나오면 그 bash가 끝나고 컨테이너도 멈춥니다. 그래서 컨테이너를 살려 둔 채 빠져나오려면 `CTRL+P → CTRL+Q`를 써야 합니다.

이번에는 exec를 알아보겠습니다. exec는 **컨테이너 안에 새 프로세스를 실행**하는 방식입니다.

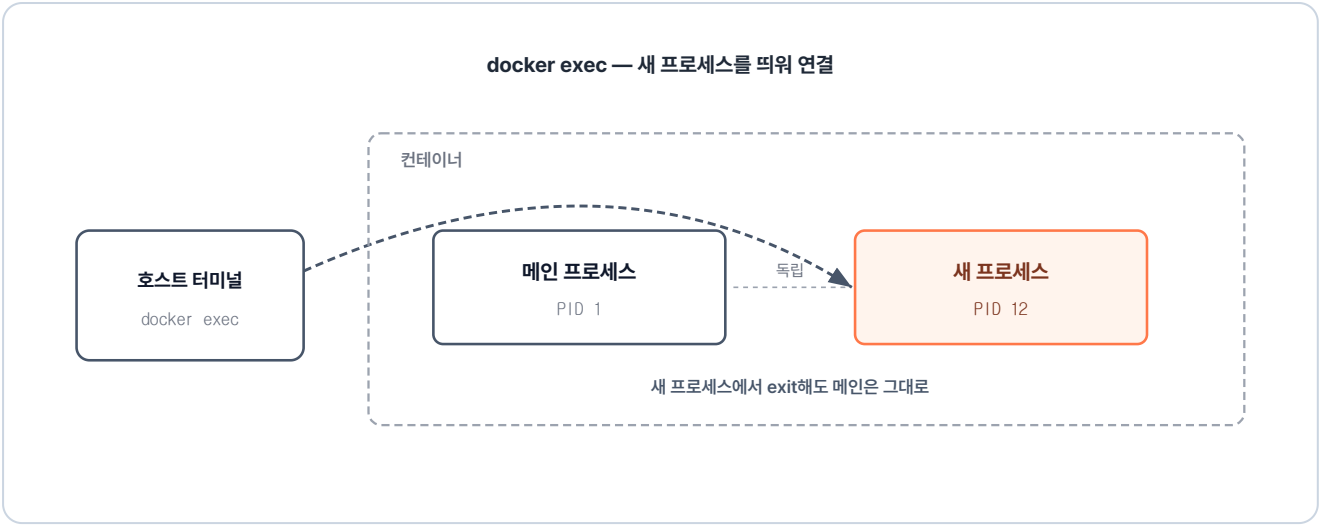


그림 2-33. exec는 컨테이너 안에 새 bash 프로세스를 띄우고 내 터미널에 연결하는 방식

새 프로세스에 연결되므로 exit해도 메인 프로세스는 그대로입니다. **exec는 새 프로세스가 실행되기 때문에 프로세스를 지정해야 합니다.** `docker exec -it <컨테이너ID> <프로세스>` 형태로 접근합니다.

터미널 **실행 중 컨테이너에 새 bash 접속**

```
docker exec -it d2b1 bash   # 실행 중인 컨테이너에 새 bash 접속
```

실행결과

```
$ docker exec -it d2b1 bash
root@d2b18acc2cf0:/#
```

그림 2-34. exec로 접근

attach와 똑같이 bash 터미널이 나타나서, 화면만 보면 둘이 같아 보입니다. exec가 정말 새 프로세스를 실행했는지 눈으로 확인해 보겠습니다. **새로운 터미널 창을 연 후,** `ps aux` 명령으로 컨테이너 안에서 실행 중인 프로세스 목록을 조회합니다.

터미널 **컨테이너 내부 프로세스 목록 확인**

```
docker exec d2b1 ps aux   # 컨테이너 내부 프로세스 목록 확인
```

프로세스 목록 확인

```
$ docker exec d2b1 ps aux
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 1 0.0 0.0 4588 3840 pts/0 S+ 03:29 0:00 /bin/bash
root 12 0.1 0.0 4588 3840 pts/1 S+ 03:34 0:00 bash
root 21 63.6 0.0 7888 4096 ? Rs 03:34 0:00 ps aux
```

그림 2-35. 프로세스 목록 확인

목록에 bash가 둘 있습니다. **PID 1**은 앞서 본 메인 프로세스이고, **PID 12**가 방금 exec로 새로 실행한 프로세스입니다. 두 프로세스는 같은 컨테이너의 파일과 네트워크를 공유하지만, 각자 독립적으로 동작합니다.

'결국 컨테이너를 제어한다는 건, 그 안에서 실행되는 메인 프로세스를 다루는 일이었구나.'

2.7 commit - 실행 중인 컨테이너를 이미지로

앞에서 컨테이너 안에 패키지를 설치하고 파일도 직접 만들어 봤습니다. 그런데 컨테이너를 내리면 이렇게 작업한 내용이 모두 사라집니다.

'내가 수정한 이 상태를 기록해두려면 어떻게 해야 하지?'

컨테이너의 현재 상태를 그대로 이미지로 저장하는 명령이 `docker commit`입니다. 이번엔 직접 만든 이미지를 Docker Hub에 올려보겠습니다.

2.7.1 Tomcat으로 commit 실습하기

나만의 이미지를 만들어보겠습니다. 우선 Tomcat 이미지를 받아 컨테이너를 실행합니다.

터미널 **Tomcat 컨테이너 실행**

```
docker run -d -p 8080:8080 tomcat   # 8080 포트로 Tomcat 백그라운드 실행
```

실행 결과

```
$ docker run -d -p 8080:8080 tomcat
5fcd8e3a1c9b07d4f2e6a5083b1d9c4f7e206a8d3b5f1e9c0a7d4b2f86e1c3a0
```

그림 2-36. Tomcat 백그라운드 실행 결과

실행 후 브라우저에서 **localhost:8080** 으로 접속하면 **404**가 뜹니다.



그림 2-37. Tomcat 404 에러 화면

'연결은 된 거 같은데 왜 404 에러가 뜨지?'

Tomcat은 루트 주소로 접속하면 `webapps/ROOT/` 폴더의 index.html을 기본 페이지로 응답합니다. 그런데 Docker Hub에서 받은 이미지는 `webapps/`가 비어 있어, 이 index.html이 없으니 404가 뜹니다.

그래서 `webapps/ROOT/`에 index.html을 직접 만들어 브라우저에서 응답되도록 해 보겠습니다.

터미널 **Tomcat 컨테이너 접속과 설정 수정**

```
docker exec -it 5fcd bash              # 컨테이너 안에 새 bash 터미널(입력창)을 열어 접속
cd /usr/local/tomcat/webapps           # Tomcat의 웹앱 기본 경로로 이동
mkdir ROOT && cd ROOT                  # ROOT 폴더 생성 후 그 안으로 이동
apt update && apt install -y vim       # 패키지 목록 갱신 후 vim 편집기 설치
vim index.html                         # 파일 만들고 HTML 작성 후 저장
```

vim으로 index.html을 만들어 저장한 뒤 **localhost:8080** 으로 다시 접속하면 방금 만든 페이지가 응답됩니다.

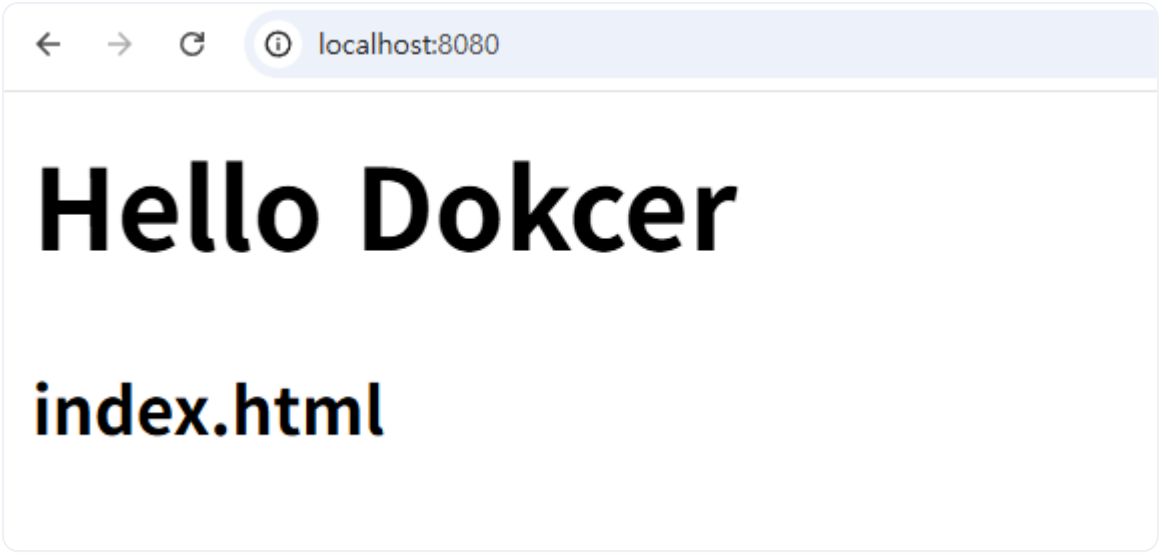


그림 2-38. index.html 응답 확인

확인이 끝나면 `exit`를 입력해 컨테이너를 빠져나옵니다. `exec`로 컨테이너에 접속했기 때문에 `exit`를 입력해도 메인 프로세스는 유지됩니다.

이제 `docker commit <컨테이너ID> <본인-dockerhub-id>/<이미지이름>` 형식으로 명령어를 입력해 현재 상태를 기록합니다.

터미널 컨테이너 상태를 새 이미지로 저장

```
docker commit 5fcd coderyu5523/tomcat   # 현재 컨테이너 상태를 새 이미지로 저장
```

실행결과

```
$ docker commit 5fcd coderyu5523/tomcat
sha256:0f8181585ca9e7f2b68c623f0ce6d64f5fddfeb19076c7c0e82e55ca4629cfbb
```

그림 2-39. 이미지 커밋 완료 화면

저장된 이미지가 목록에 잘 들어갔는지 확인합니다.

터미널 로컬 이미지 목록 확인

```
docker images   # 로컬에 저장된 이미지 목록 확인
```

실행 결과

```
$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
coderyu5523/tomcat latest 0f8181585ca9 12 seconds ago 541MB
tomcat latest f1e6c9d4a7b8 2 weeks ago 489MB
```

그림 2-40. 로컬 이미지 목록에 새 이미지 추가 확인

2.7.2 Docker Hub에 올리기

지금까지 저장한 이미지는 로컬에만 있습니다. 이 이미지를 Docker Hub에 올려두면 다른 PC나 서버에서 내려받을 수 있습니다.

Docker Hub에 로그인한 뒤 `docker push <본인-dockerhub-id>/<이미지이름>` 형식으로 이미지를 올립니다.

터미널 **Docker Hub 로그인과 푸시**

```
docker login                      # Docker Hub 로그인
docker push coderyu5523/tomcat    # 로컬 이미지를 Hub로 업로드
```

실행 결과

```
$ docker push coderyu5523/tomcat
The push refers to repository [docker.io/coderyu5523/tomcat]
247bb26cb831: Preparing
817807f3c64e: Preparing
6e713be20fa2: Preparing
cc1e0a391268: Preparing
de9be28b9519: Preparing
c318c44e952a: Preparing
4f4fb700ef54: Preparing
5dd5756e7123: Preparing
cc1e0a391268: Waiting
```

그림 2-41. docker push 실행 결과

업로드가 완료된 뒤 Docker Hub의 Repositories 탭을 열면 방금 올린 이미지를 확인할 수 있습니다.

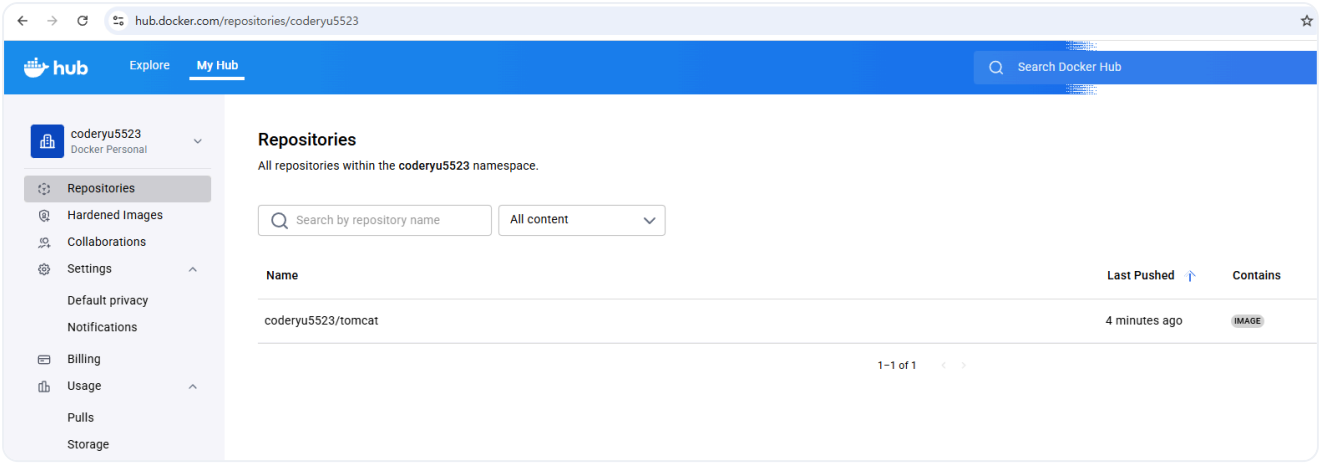


그림 2-42. Docker Hub 저장소 확인

2.8 마운트로 데이터 보존

방금 한 commit은 그 시점의 컨테이너를 이미지로 만들 뿐, 데이터를 계속 보관하는 저장소는 아닙니다. 컨테이너가 실행되며 저장되는 데이터는 이미지에 남지 않아, 컨테이너를 삭제하면 데이터도 통째로 날아갑니다.

'컨테이너를 내려도 데이터는 남아 있어야 하는데.'

데이터를 잃지 않으려면 컨테이너 외부에 보관해야 합니다. 컨테이너와 외부 저장소를 연결해 주는 기능이 **마운트**입니다. 마운트 방식은 두 가지입니다.

2.8.1 바인드 마운트: 호스트 폴더와 직접 연결

첫 번째 방식은 **호스트 PC의 특정 폴더**를 **컨테이너 내부의 경로**와 연결하는 **바인드 마운트(Bind Mount)**입니다. USB를 컴퓨터에 꽂으면 그 안의 파일을 그대로 볼 수 있듯, 컨테이너도 호스트 PC의 폴더를 꽂아서 쓰는 방식입니다.

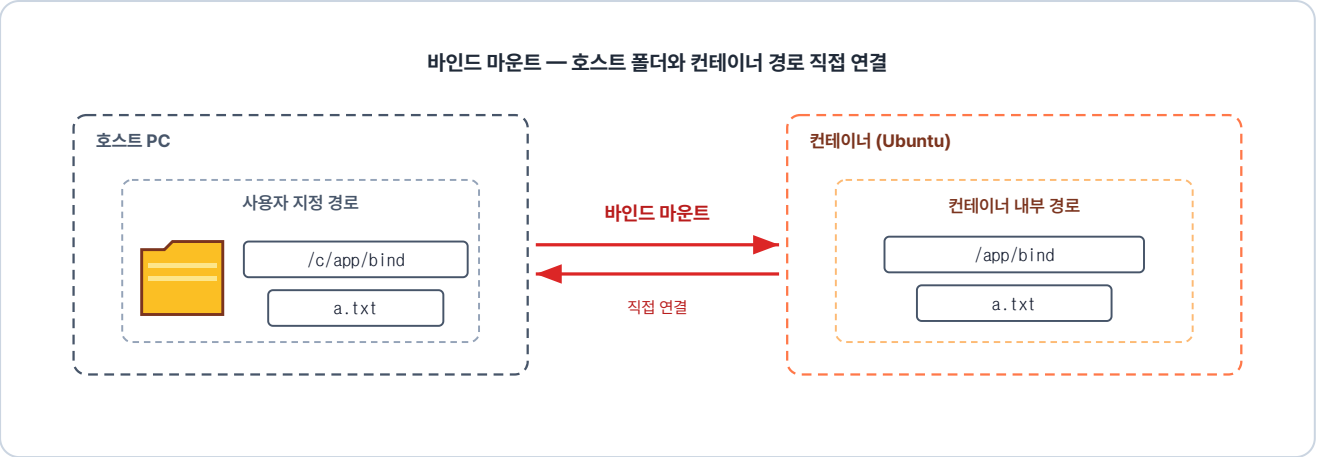


그림 2-43. 호스트 폴더와 컨테이너 경로가 같은 데이터를 바라보도록 묶인 상태

이제 터미널을 열고 직접 해보겠습니다. 호스트의 `/c/app/bind` 경로에 폴더를 하나 만들고, 컨테이너 안의 경로와 연결합니다.

터미널 **bind mount 폴더 준비와 마운트 실행**

```
# 호스트 PC에 공유할 폴더 생성
mkdir -p /c/app/bind
# 문법: docker run -it --mount type=bind,src=<호스트 경로>,dst=<컨테이너 경로> <이미지>
# 호스트 폴더(src)와 컨테이너 경로(dst)를 연결
docker run -it --mount type=bind,src=/c/app/bind,dst=/app/bind ubuntu
```

실행결과

```
$ docker run -it --mount type=bind,src=/c/app/bind,dst=/app/bind ubuntu
root@a3f1e2b4c5d6:/#
```

그림 2-44. 바인드 마운트로 ubuntu 실행

컨테이너 안 `/app/bind`에 `a.txt`를 만들고, 호스트 쪽 `/c/app/bind`에서 같은 파일이 보이는지 확인합니다.

터미널 **bind mount 양방향 동기화 확인**

```
touch /app/bind/a.txt   # 컨테이너 안에서 파일 생성
```

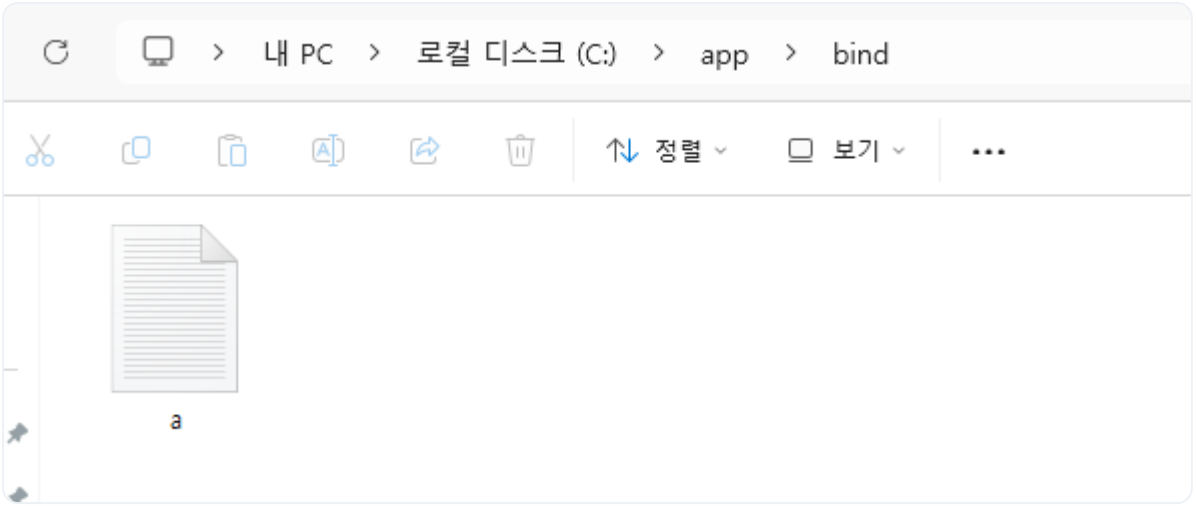


그림 2-45. 호스트 PC에서 같은 파일이 보이는 것을 확인

2.8.2 볼륨 마운트: Docker가 관리하는 저장소

바인드 마운트는 **사용자가 폴더를 직접 관리**해야 합니다. 반대로 **볼륨 마운트(Volume Mount)** 는 **Docker가 자동으로 관리**합니다. **사용자는 볼륨에 이름만 정하면, 호스트의 실제 저장 위치는 Docker가 알아서** 처리합니다.

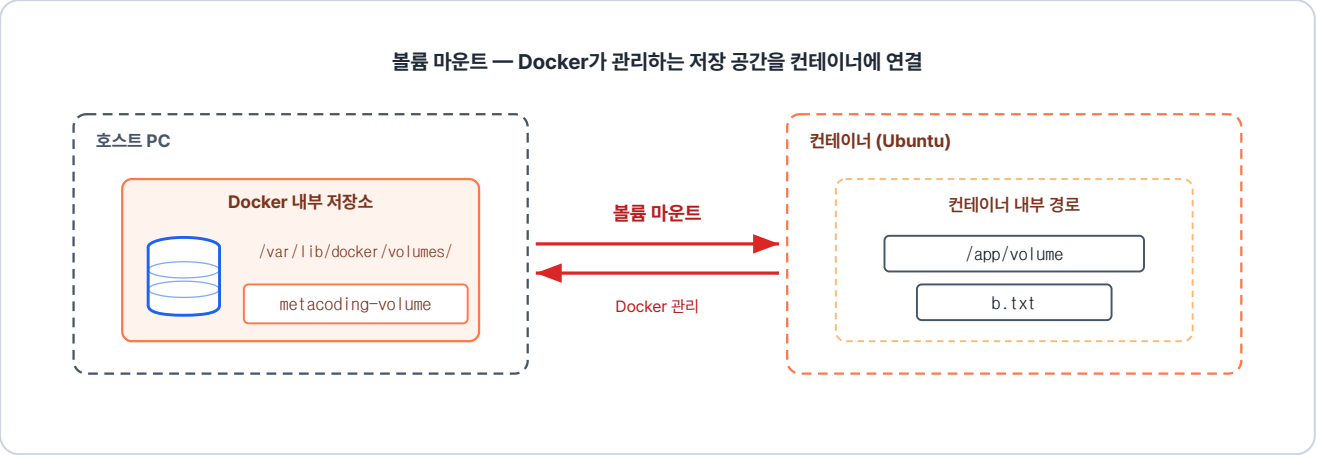


그림 2-46. Docker 엔진이 관리하는 내부 저장 공간에 데이터가 저장되는 구조

이제 직접 해보겠습니다. 다음 명령은 `metacoding-volume` 이라는 볼륨을 만들어 ubuntu 컨테이너를 실행합니다.

터미널 **named volume 마운트로 ubuntu 실행**

```
# 문법: docker run -it --mount type=volume,src=<볼륨 이름>,dst=<컨테이너 경로> <이미지>
# 볼륨 마운트로 ubuntu 실행
docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu
```

실행결과

```
$ docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu
root@d6f096d57b3e:/#
```

그림 2-47. 볼륨 마운트로 ubuntu 실행

데이터는 Docker가 관리하는 저장소에 보관되며, 컨테이너는 `/app/volume` 경로를 통해 이 데이터를 읽고 씁니다. 따라서 해당 경로에 저장한 데이터만 유지되고, 그 외의 경로에 저장된 파일은 컨테이너 삭제 시 함께 지워집니다.

이 폴더에 `b.txt`를 만들고, 컨테이너를 빠져나와 볼륨이 그대로 남아 있는지 확인합니다.

터미널 **볼륨에 파일 생성**

```
touch /app/volume/b.txt    # 볼륨 안에 빈 파일 생성
exit                       # 컨테이너 종료
docker volume ls           # 볼륨 목록 재확인
```

실행결과

```
root@d6f096d57b3e:/# exit
exit
$ docker volume ls
DRIVER VOLUME NAME
local metacoding-volume
```

그림 2-48. 컨테이너가 사라져도 볼륨은 유지

컨테이너는 사라졌지만 볼륨은 목록에 그대로 남아 있습니다. 이어서 같은 볼륨을 연결해 새 컨테이너를 다시 실행합니다.

터미널 **같은 볼륨으로 새 컨테이너에서 데이터 확인**

```
# 같은 볼륨으로 새 컨테이너 실행
docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu
ls /app/volume
```

실행 결과

```
$ docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu
root@e8a3f1c47b29:/#
root@e8a3f1c47b29:/# ls /app/volume
b.txt
```

그림 2-49. 볼륨 데이터 재사용

방금 만든 `b.txt`가 새 컨테이너 안에서도 그대로 보입니다. **컨테이너를 내려도 데이터는 사라지지 않습니다.**

다음 챕터로 넘어가기 전, 실행 중인 컨테이너를 모두 종료합니다.

터미널 **실습한 컨테이너 전체 종료**

```
docker stop $(docker ps -q)   # 실행 중인 모든 컨테이너 종료
```

오픈이는 하루 사이에 익힌 키워드를 정리했습니다. 격리, 이미지, 포트포워딩, CMD, 볼륨이 머릿속에 정리됐습니다. 어제 헤맸던 그 에러도 이제는 어디에서 막혔던 건지 짚을 수 있을 것 같았습니다.

다만 지금까지 익힌 건 도커와 컨테이너의 기본입니다. 실제 현장에서 그대로 쓰기에는 한계가 있습니다.

'기본은 익혔다. 그런데 실제 현장은 이것만으로는 부족하겠지.'

다음 챕터에서는 컨테이너를 활용해 실제 서비스를 구성하는 방법을 익혀 보겠습니다.

이것만은 기억하자

- → **컨테이너는 격리된 프로세스입니다.** 파일시스템·네트워크·프로세스 공간만 따로 나뉘고, 커널은 호스트와 공유합니다.
- → **이미지는 설계도, 컨테이너는 찍혀 나온 결과입니다.** 붕어빵 틀 하나로 여러 붕어빵을 찍듯, 이미지 하나로 여러 컨테이너를 찍어냅니다.
- → **컨테이너 통신은 docker0가 중심입니다.** 컨테이너끼리는 docker0로 연결되고, 외부 요청은 포트포워딩으로 들어옵니다.
- → **메인 프로세스가 살아 있어야 컨테이너도 살아 있습니다.** 그래서 이미지의 CMD에 실행할 프로그램을 미리 지정해 둡니다.
- → **마운트는 컨테이너와 외부 저장소를 잇습니다.** 바인드는 호스트의 특정 폴더를, 볼륨은 Docker 엔진이 관리하는 저장소를 컨테이너에 연결합니다.

CHAPTER

3

Docker 다루기

여러 컨테이너를 한 번에 실행하다

이번 챕터가 끝나면

- **Dockerfile**로 환경 구성을 자동화합니다.
- **NGINX**로 트래픽을 받아 넘기고 분산하는 구조를 설정해 봅니다.
- **Redis**로 여러 서버가 세션을 공유하는 구조를 만듭니다.
- **Docker Compose**로 여러 컨테이너를 한 번에 띄웁니다.

챕터 3. Docker 다루기

며칠 후, 화요일 오전 열 시. 회의실에 사람이 모였습니다. 팀장이 화이트보드 앞에 서서 이번에 만들 프로젝트를 설명했습니다. 사내에서 쓸 작은 서비스 하나를 새로 만든다는 이야기였습니다. 화면을 그릴 프론트엔드, 요청을 처리할 백엔드, 회원 정보를 보관할 DB가 함께 구성된 형태였습니다.

팀장이 펜을 내려놓고 오픈이 쪽을 봤습니다.

팀장 : "환경은 Docker로 구성해봐요. 요즘 공부했잖아요."

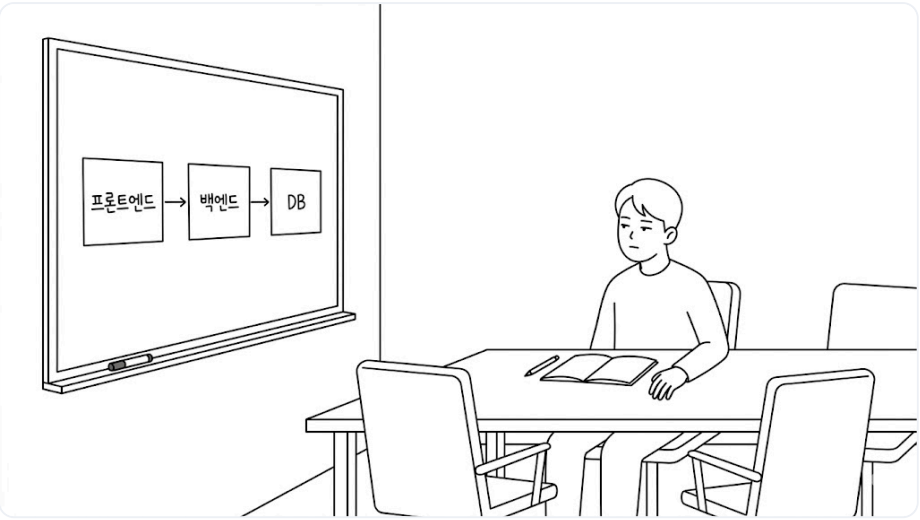


그림 3-1. 화이트보드에 그려진 프론트엔드·백엔드·DB 구성

회의실을 나서는 발걸음이 가볍지 않았습니다. 컨테이너 하나를 실행하는 건 익혔지만, 여러 개를 한꺼번에 관리해 본 적은 없었습니다.

'한 대씩은 실행해봤는데, 여러 대를 함께 구성하는 건 또 다른 이야기인데.'

자리로 돌아온 오픈이가 옆자리 선배에게 물었습니다.

오픈이 : "프론트엔드·백엔드·DB까지 구성해야 하는데, 어디서부터 손대야 할지 모르겠어요."

선배 : "한꺼번에 다 하려니 막막한 거예요. 하나씩 하면 돼요. 먼저 매번 직접 세팅할 수는 없으니, 서비스마다 **필요한 걸 다 갖춘 환경을 미리 준비**해 둬요. 그리고 외부에서 요청을 받아 **적절한 컨테이너로 전달**하는 거죠. 마지막에 전체를 하나의 구성으로 합쳐 한 번에 실행하면 돼요."

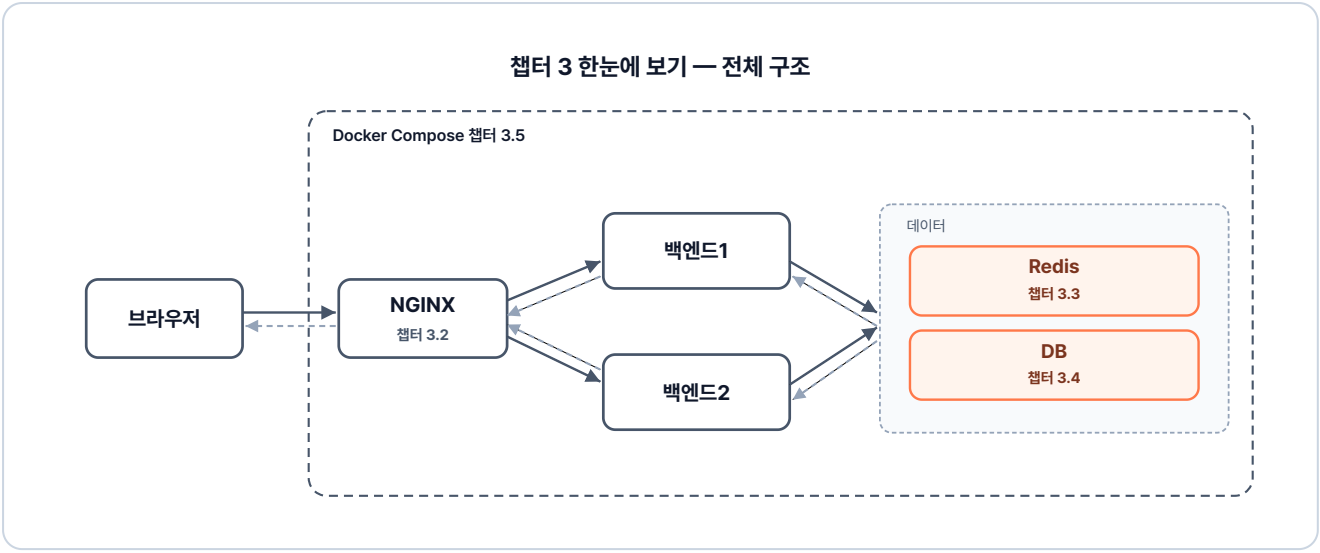


그림 3-2. 챕터 3 한눈에 보기 - 전체 구조

실습 준비. 예제 코드

이 책의 실습 코드는 GitHub 레포 하나에 챕터별 폴더로 담겨 있습니다. 아래 명령으로 레포를 한 번 git clone 합니다. 이후 챕터마다 해당 폴더로 이동해 실습합니다.

터미널 **예제 코드 클론**

```
git clone https://github.com/metacoding-10-linux-docker/start
```

이번 챕터는 `ex00` ~ `ex07` 폴더를 사용합니다.

완성된 코드는 아래 주소에서 확인하세요.

터미널 **완성 코드**

```
https://github.com/metacoding-10-linux-docker/final
```

3.1 Dockerfile - 환경을 자동으로 만들기

3.1.1 프로비저닝

앞에서 우리는 컨테이너 안으로 들어가 필요한 패키지와 환경을 직접 설치했습니다. 이 방법은 관리해야 할 컨테이너가 늘어날수록 일일이 설치하기 번거로워집니다. 밀키트를 한번 떠올려 보세요. 밀키트는 이미 재료와 레시피가 준비되어 있습니다. 그대로 조리하면 누가 만들어도 같은 요리가 나옵니다.

이런 방식이 도커에 이미 마련되어 있습니다. 필요한 환경을 파일 하나에 미리 정의해 두면, 컨테이너를 새로 만들 때 같은 작업을 반복하지 않아도 됩니다. 이렇게 환경을 미리 구성해 두는 작업을 **프로비저닝(Provisioning)** 이라고 부릅니다.

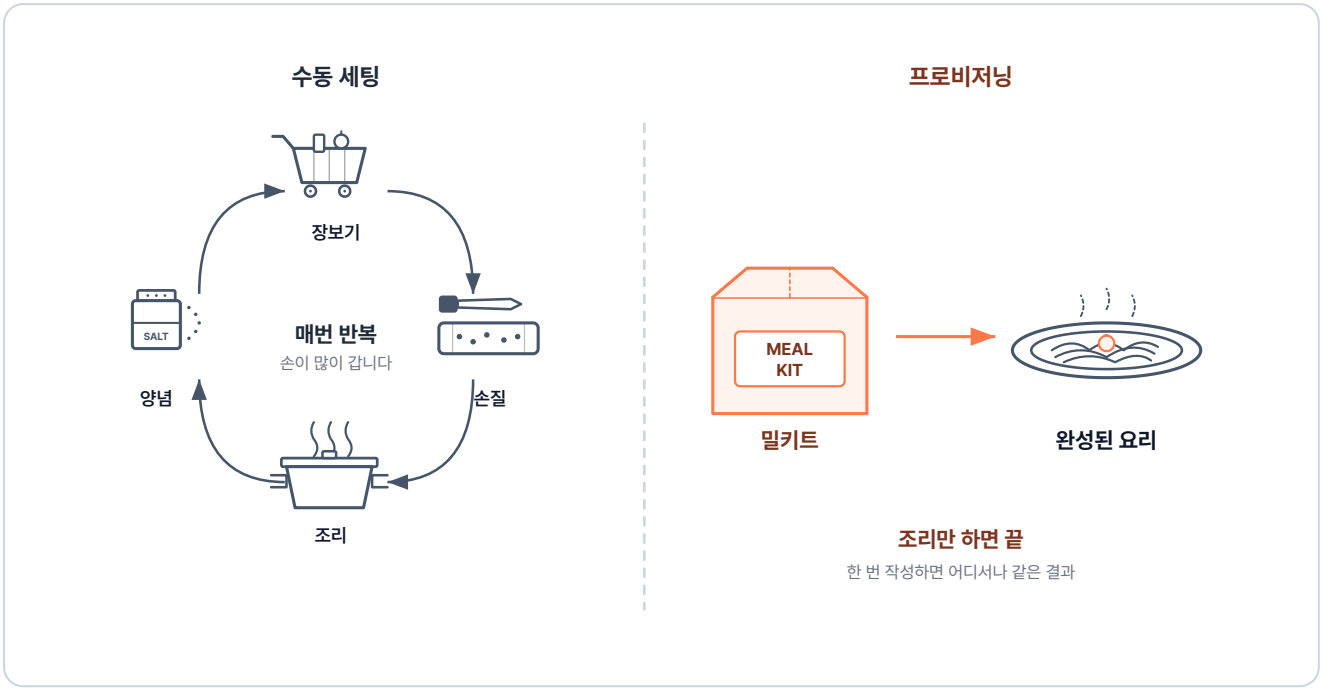


그림 3-3. 수동 세팅과 프로비저닝 비교

도커에서 이 프로비저닝을 담당하는 정의 파일이 바로 **Dockerfile**입니다. 밀키트처럼 필요한 설정을 한 번 적어 두면, 도커가 그대로 따라 같은 이미지를 자동으로 만들어 줍니다.

3.1.2 Dockerfile에서 컨테이너까지의 세 단계

Dockerfile은 이미지를 만드는 데 필요한 환경 구성을 담은 스크립트입니다. 베이스 이미지, 설치할 패키지, 복사할 파일, 실행할 명령을 순서대로 적어 두면, 그 내용대로 컨테이너가 만들어집니다.

Dockerfile에서 컨테이너로 실행되기까지 크게 세 단계를 거칩니다.

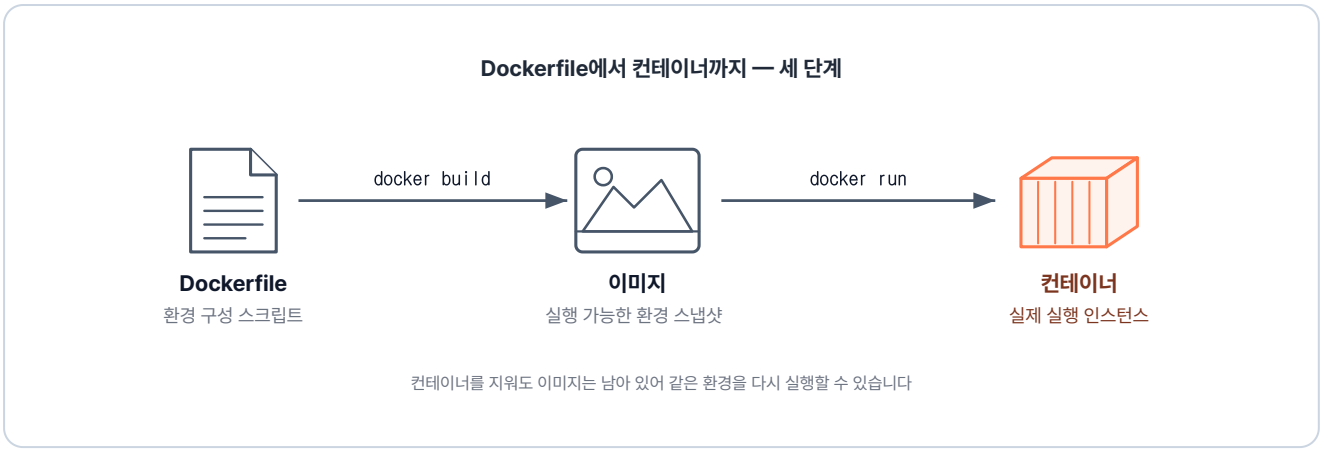


그림 3-4. Dockerfile → 이미지 → 컨테이너의 세 단계

1. **Dockerfile 작성**: 구축하고 싶은 환경을 텍스트 파일에 차례대로 적습니다.
2. **docker build**: 도커 엔진이 Dockerfile에 적힌 명령을 순서대로 처리합니다. 이 과정이 끝나면 결과물이 이미지로 저장됩니다.
3. **docker run**: 생성된 이미지를 기반으로 실제 컨테이너를 실행합니다.

'그러면 이 Dockerfile에 뭘 어떻게 적느냐가 핵심이겠네.'

3.1.3 Dockerfile 기본 문법

Dockerfile에서 가장 자주 쓰이는 지시어를 표로 정리하면 다음과 같습니다.

지시어	역할
`FROM`	베이스 이미지를 지정합니다. (어떤 환경에서 시작할지)
`WORKDIR`	컨테이너 내부에서 명령이 실행될 기본 디렉토리를 지정합니다.
`COPY`	호스트 컴퓨터의 파일을 컨테이너 안으로 복사합니다.
`RUN`	이미지를 빌드하는 동안 실행할 명령어입니다. (패키지 설치 등)
`ENV`	컨테이너 안에서 사용할 환경 변수를 설정합니다.
`CMD`	컨테이너 시작 시 실행할 기본 명령어입니다.
`ENTRYPOINT`	컨테이너 시작 시 반드시 실행되는 메인 프로세스입니다.

첫 실습으로 Ubuntu 환경에 vim이 미리 설치된 이미지를 만들어 보겠습니다. 실습 코드는 클론한 레포의 `ex00` 폴더에 있습니다.

`Dockerfile`을 열어 다음 내용을 작성합니다.

실습 1 **ex00/Dockerfile. ubuntu-vim 이미지 정의**

```
# Dockerfile
FROM ubuntu:24.04                       # 베이스 이미지
RUN apt update && apt install -y vim    # vim 패키지 설치
CMD ["/bin/bash"]                       # 컨테이너 시작 시 bash 실행
```

파일을 저장하고 빌드 명령을 입력합니다.

터미널 **ubuntu-vim 이미지 빌드와 컨테이너 실행**

```
docker build -t ubuntu-vim ex00   # ex00 폴더의 Dockerfile로 이미지 빌드
docker run -it ubuntu-vim                   # 빌드한 이미지로 컨테이너 실행
```

빌드가 끝나면 이미지가 완성됩니다. 이 이미지로 컨테이너를 실행하고 vim을 입력하면, 별도의 설치 과정 없이 편집기가 그대로 뜹니다.

실행 결과

```
$ docker run -it ubuntu-vim
root@a1b2c3d4e5f6:/# vim
~
~
~
:q!
```

그림 3-5. 컨테이너에 들어가 vim을 실행한 결과

3.1.4 WORKDIR와 COPY

다음으로 로컬에 있는 파일을 컨테이너 내부로 옮겨보겠습니다. 여기에 쓰는 지시어가 **WORKDIR**와 **COPY**입니다.

Dockerfile과 같은 폴더에 `index.html`이 들어 있습니다. 작업 디렉토리를 `/app`으로 정하는 `WORKDIR`와, 그 안으로 index.html을 복사하는 `COPY` 두 줄을 추가합니다.

실습 2 **ex00/Dockerfile. WORKDIR과 COPY 추가**

```
FROM ubuntu:24.04                       # 베이스 이미지
WORKDIR /app                            # 작업 디렉토리 설정
COPY ./index.html ./index.html          # 로컬의 index.html을 컨테이너의 /app으로 복사
RUN apt update && apt install -y vim    # vim 패키지 설치
CMD ["/bin/bash"]                       # 컨테이너 시작 시 bash 실행
```

수정한 Dockerfile로 이미지를 다시 만들어 실행합니다.

터미널 **ubuntu-html 이미지 빌드와 실행**

```
docker build -t ubuntu-html ex00   # ex00 폴더의 Dockerfile로 이미지 빌드
docker run -it ubuntu-html       # 컨테이너 실행
ls
```

실행 결과

```
$ docker run -it ubuntu-html
root@5497d6efff3c:/app# ls
index.html
```

그림 3-6. 실행 결과 확인

터미널 프롬프트가 처음부터 `/app`을 가리키고, 그 안에 index.html이 들어 있습니다.

3.1.5 CMD와 ENTRYPOINT

CMD와 ENTRYPOINT는 둘 다 컨테이너가 시작될 때 무엇을 실행할지 정하는 지시어입니다. CMD는 **실행할 때 다른 명령으로 바꿀 수 있는 기본 프로세스**이고, ENTRYPOINT는 **반드시 실행되는 메인 프로세스**입니다.

커피 머신을 떠올리면 이해하기 쉽습니다. ENTRYPOINT는 **커피를 내린다는 동작 자체**라, 어떤 메뉴를 선택하든 반드시 실행됩니다. 반면 CMD는 따로 옵션을 선택하지 않으면 **기본 메뉴인 아메리카노**가 나오고, 라떼를 선택하면 라떼가 나옵니다. 즉 기본값이 쓰이되, 실행할 때 다른 값을 주면 그 값으로

바뀝니다.

실습을 위해 Dockerfile에 ENTRYPOINT를 추가합니다.

실습 3 **ex00/Dockerfile. ENTRYPOINT로 자동 실행**

```
FROM ubuntu:24.04                       # 베이스 이미지
WORKDIR /app                            # 작업 디렉토리 설정
COPY ./index.html ./index.html          # 로컬의 index.html을 컨테이너의 /app으로 복사
RUN apt update && apt install -y vim    # vim 패키지 설치
CMD ["/bin/bash"]                       # ubuntu의 기본 프로세스
ENTRYPOINT ["echo", "컨테이너 실행"]     # 컨테이너 시작 시 echo 실행
```

이미지를 다시 빌드하고 컨테이너를 실행합니다.

터미널 **ubuntu-entry 이미지 빌드와 실행**

```
docker build -t ubuntu-entry ex00   # ex00 폴더의 Dockerfile로 이미지 빌드
docker run -it ubuntu-entry                    # 컨테이너 실행
```

실행결과

```
$ docker build -t ubuntu-entry ex00
#7 naming to docker.io/library/ubuntu-entry:latest done
#7 DONE 0.3s
$ docker run -it ubuntu-entry
컨테이너 실행 /bin/bash
```

그림 3-7. ENTRYPOINT 실행 결과

ENTRYPOINT의 `echo "컨테이너 실행"` 뒤에 CMD의 `/bin/bash`가 인자로 붙어 함께 출력됩니다. CMD만 있을 때는 **CMD가 메인 프로세스**가 되지만, ENTRYPOINT와 함께 쓰이면 ENTRYPOINT가 메인 프로세스로 실행되고 CMD의 값은 실행되지 않은 채 **그 인자(Argument)로 전달**됩니다.

'이렇게 환경을 미리 설정해 두면, 매번 명령을 실행할 필요 없이 관리할 수 있구나.'

3.2 NGINX - 요청을 앞에서 받아 나눠주기

앞에서 Dockerfile로 컨테이너 하나를 만들어 실행해 봤습니다. 그런데 실제 서비스는 컨테이너 하나로 끝나지 않습니다. 여러 컨테이너가 함께 돌아가고, 들어오는 요청마다 처리해야 할 컨테이너가 다릅니다.

'들어온 요청을 알맞은 컨테이너로 보내려면 어떻게 해야 할까.'

3.2.1 NGINX의 역할

이 역할을 맡는 도구가 **NGINX**입니다. NGINX는 들어온 요청을 받아, 그 요청을 처리할 알맞은 컨테이너로 전달합니다.

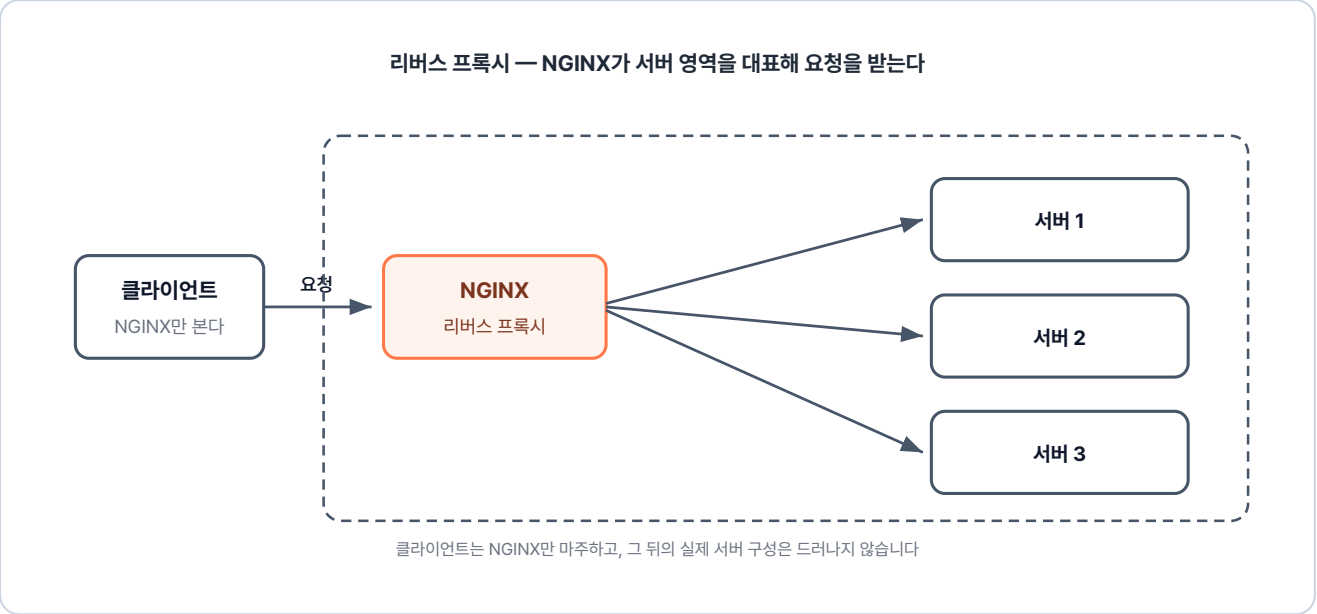


그림 3-8. 클라이언트에게는 NGINX 하나로 보이는 서버 영역

NGINX는 가볍고 빠른 오픈소스 웹 서버이자 **리버스 프록시(Reverse Proxy)** 입니다. 리버스 프록시는 클라이언트의 요청을 대신 받아 알맞은 뒤쪽 서버로 넘겨주는 중계 역할을 합니다. 이 구조 덕분에 클라이언트에게는 NGINX만 보여 내부 서버의 보안이 유지됩니다. 요청 전달 외에도 여러 서버로 트래픽을 분산시키는 로드 밸런싱이나 캐싱 등 트래픽 관리에도 사용됩니다.

3.2.2 NGINX 설정과 요청 흐름

NGINX는 전국의 택배가 모이는 대형 분류 센터처럼 동작합니다. 전국에서 들어온 택배가 한곳에 모였다가, 주소지에 따라 알맞은 지역 센터로 나뉘어 나갑니다. NGINX의 설정도 같은 방식으로 요청을 나눠 보냅니다.

- **upstream (서버 그룹 정의)** : 특정 지역을 담당할 **배송 센터(서버 그룹)** 로 모아 이름을 붙이는 작업입니다. 물건을 넘겨줄 목적지를 미리 등록해 둡니다.
- **location (요청 경로 매칭)** : 택배의 **주소지(URL)를 확인**하고 분류하는 게이트입니다. 주소를 확인해 최종 행선지를 결정합니다.
- **proxy_pass (요청 전달)** : 분류된 택배를 **지정된 물류 센터로 이동**시키는 지시어입니다. **"이 물품은 서울 센터로 보내"** 라는 최종 명령입니다.

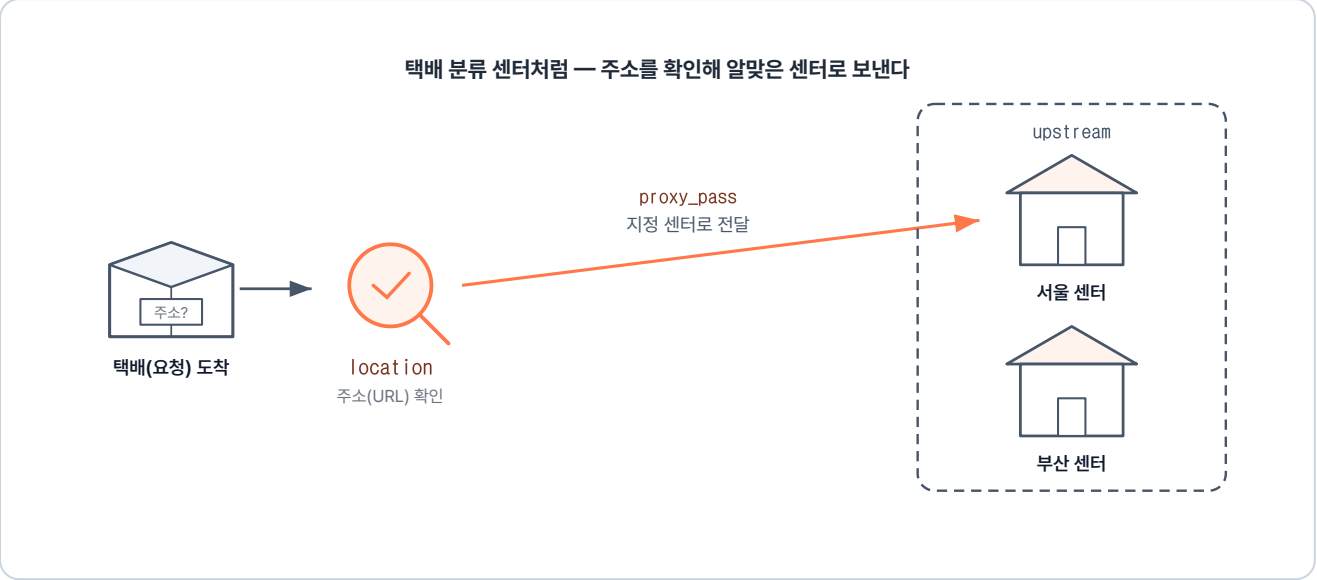


그림 3-9. 주소를 확인해(location) 지정 센터로 보내는(proxy_pass) NGINX 설정의 흐름

이 옵션을 포함한 다양한 설정을 `nginx.conf` 파일에 작성합니다. 옵션 조합에 따라 라우팅이나 로드밸런싱 같은 다양한 역할을 수행할 수 있습니다.

3.2.3 경로 기반 라우팅

먼저 해볼 실습은 요청 경로에 따른 라우팅입니다. `/app1`로 들어오면 1번 서버, `/app2`로 들어오면 2번 서버로 보냅니다.

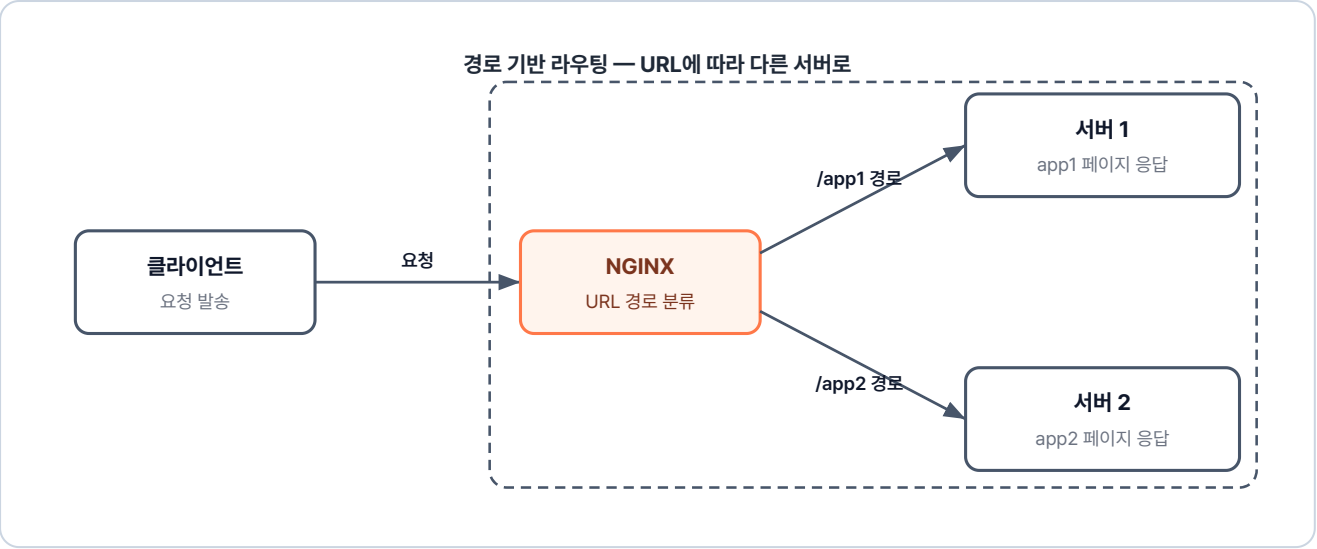


그림 3-10. 경로 기반 라우팅 구조

실습 코드는 `ex01` 폴더에 있습니다. 컨테이너로 실행할 단위는 화면을 담당하는 `app1`·`app2`와 그 앞에서 경로를 분기하는 `lb`입니다. 각 폴더에는 Dockerfile이 있습니다.

```
ex01/
├── app1/
│   ├── Dockerfile       # nginx 이미지 + index.html 복사
│   └── index.html       # app1 페이지
├── app2/
│   ├── Dockerfile       # nginx 이미지 + index.html 복사
│   └── index.html       # app2 페이지
├── lb/
│   ├── Dockerfile       # nginx 이미지 + nginx.conf 복사
│   └── nginx.conf       # 경로 라우팅 설정
└── README.md            # 실습 안내
```

핵심은 `lb` 폴더의 `nginx.conf` 파일입니다. 경로별로 어느 서버 그룹으로 보낼지 적는 파일입니다.

참고 **ex01/lb/nginx.conf. 경로 라우팅 설정**

```
upstream app1 {                          # app1 그룹 지정
    server host.docker.internal:8000;    # 호스트의 8000번 포트 = app1 컨테이너
}

upstream app2 {                          # app2 그룹 지정
    server host.docker.internal:9000;    # 호스트의 9000번 포트 = app2 컨테이너
}

server {
    listen 80;
```

```
    location /app1 {                    # /app1 주소로 오는 요청 분류
        proxy_pass http://app1/;        # upstream의 app1 그룹으로 전달
    }

    location /app2 {                    # /app2 주소로 오는 요청 분류
        proxy_pass http://app2/;        # upstream의 app2 그룹으로 전달
    }
}
```

이제 이 설정 파일을 포함한 이미지가 필요합니다. `lb` 폴더의 Dockerfile로 만듭니다.

실습 4 **ex01/lb/Dockerfile. NGINX 라우터 이미지**

```
FROM nginx                                      # NGINX 공식 이미지 사용
COPY nginx.conf /etc/nginx/conf.d/default.conf  # 작성한 설정 파일을 기본 경로에 복사
ENTRYPOINT ["nginx", "-g", "daemon off;"]       # NGINX를 포그라운드로 실행
```

세 폴더가 준비되면 차례로 빌드하고 실행합니다.

터미널 **app1·app2·lb 빌드와 실행**

```
# app1: 이미지 빌드 + 호스트 8000 → 컨테이너 80
docker build -t app1 ex01/app1
docker run -dit -p 8000:80 app1

# app2: 이미지 빌드 + 호스트 9000 → 컨테이너 80
docker build -t app2 ex01/app2
docker run -dit -p 9000:80 app2

# lb: 이미지 빌드 + 호스트 80 → 컨테이너 80
docker build -t lb ex01/lb
docker run -dit -p 80:80 lb
```

브라우저 주소창에 `localhost:80/app1`을 입력하면 1번 서버의 화면이 뜹니다. 주소를 `/app2`로 바꾸면 이번에는 2번 서버 화면이 나타납니다.



그림 3-11. /app1 경로로 접속한 결과

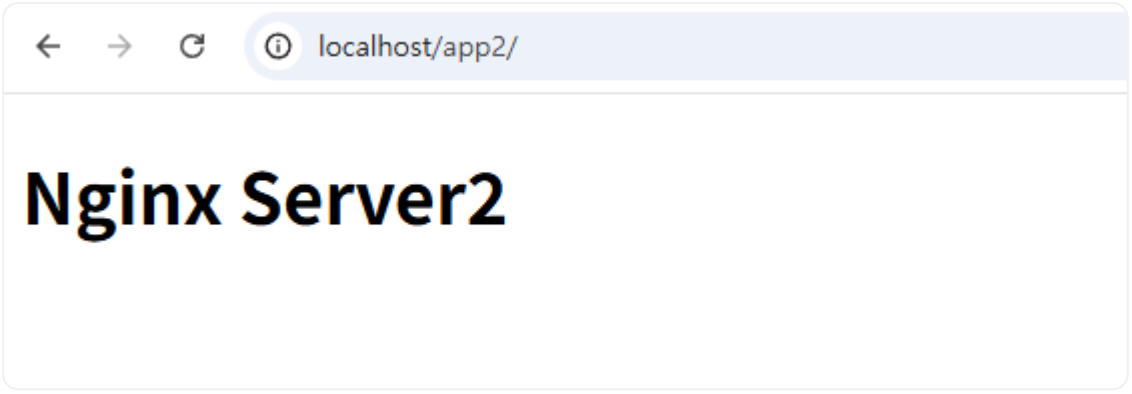


그림 3-12. /app2 경로로 접속한 결과

포트 충돌이 발생할 수 있으니, 각 예제 실습 후에 다음 명령어를 실행해 실행 중인 컨테이너를 종료합니다.

터미널 **실행 중인 컨테이너 전체 종료**

```
docker stop $(docker ps -q)   # 실행 중인 모든 컨테이너 종료
```

3.2.4 host.docker.internal과 사용자 정의 네트워크

host.docker.internal이 왜 필요한가

`nginx.conf`에는 서버 주소가 `host.docker.internal:8000`으로 설정되어 있습니다.

> **host.docker.internal**: 컨테이너 내부에서 호스트 PC를 가리킬 때 사용하는 특수한 주소입니다. 컨테이너 안에서 localhost를 사용하면 호스트 PC가 아닌 컨테이너 자신을 가리키게 되므로, 호스트 PC의 포트에 접근할 때는 반드시 이 별칭을 사용해야 합니다.

'같은 도커 위에서 도는 컨테이너끼리인데, 왜 호스트를 한 번 거쳐서 가야 하지.'

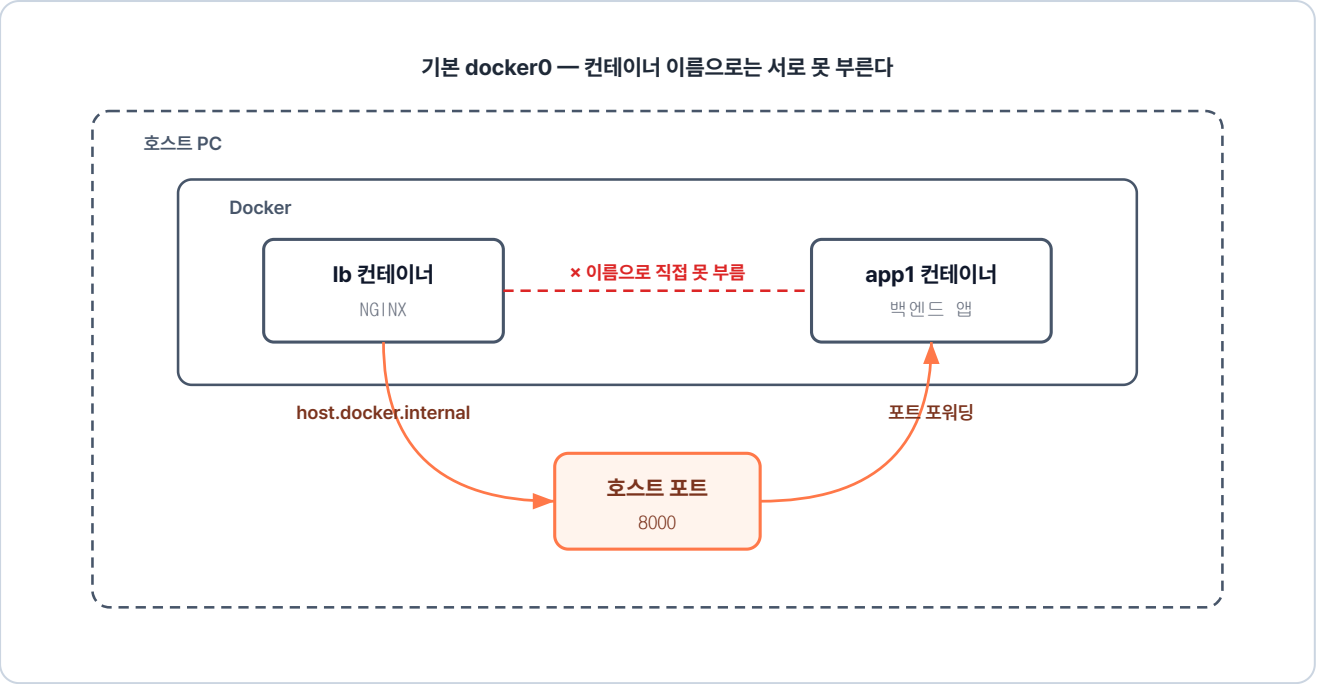


그림 3-13. lb 컨테이너 → 호스트 PC → app1 컨테이너로 가는 우회 경로

`docker run`으로 컨테이너를 하나씩 실행하면 도커는 그 컨테이너를 **기본 네트워크**에 자동으로 할당합니다. 이 기본 네트워크에는 두 가지 한계가 있습니다.

> **기본 네트워크의 두 가지 한계**
>
> - **변동되는 IP**: 컨테이너는 재시작할 때마다 IP가 새로 부여됩니다. 고정되지 않는 내부 IP는 `nginx.conf`에 하드코딩할 수 없습니다.
> - **이름으로 통신 불가**: 기본 네트워크에서는 컨테이너 이름(예: `app1`)을 IP로 변환해 주는 내장 DNS가 없어, 이름으로 컨테이너끼리 직접 통신할 수 없습니다.

이처럼 `nginx.conf`에 컨테이너의 주소를 명시할 수 없기 때문에, 다른 컨테이너에 접근하려면 호스트 PC를 거치는 `host.docker.internal`을 설정에 써야 합니다.

오픈이는 이런 궁금증이 생겼습니다.

'호스트를 거치지 않고 컨테이너끼리 바로 연결하려면 어떻게 해야 할까.'

사용자 정의 네트워크 — 이름으로 부르기

챕터 2에서 본 **사용자 정의 네트워크(User-defined Network)**가 이 문제를 해결합니다. 이 네트워크에서는 내장 DNS가 컨테이너 이름을 IP로 변환합니다. 그래서 `host.docker.internal` 같은 우회 없이 컨테이너 이름을 그대로 사용할 수 있습니다.

실습에 필요한 명령어는 다음과 같습니다.

명령	역할
`docker network create <네트워크이름>`	새 네트워크 생성
`docker run --name <컨테이너이름> --network <네트워크이름> <이미지>`	컨테이너를 해당 네트워크에 참여시키며 실행
`docker network ls`	현재 생성된 네트워크 목록 확인

'네트워크를 따로 만들어서 컨테이너들을 모아 두면, 이름만으로 통신할 수 있겠네.'

3.2.5 로드밸런싱

다음 실습은 **로드밸런싱(Load Balancing)** 입니다. 요청이 한 대의 서버로만 몰리면 처리 속도가 느려지거나 서버가 멈출 수 있습니다. 로드밸런싱은 이처럼 한 곳에 부하가 집중되지 않도록 트래픽을 여러 서버로 고르게 분산시키는 기술입니다.

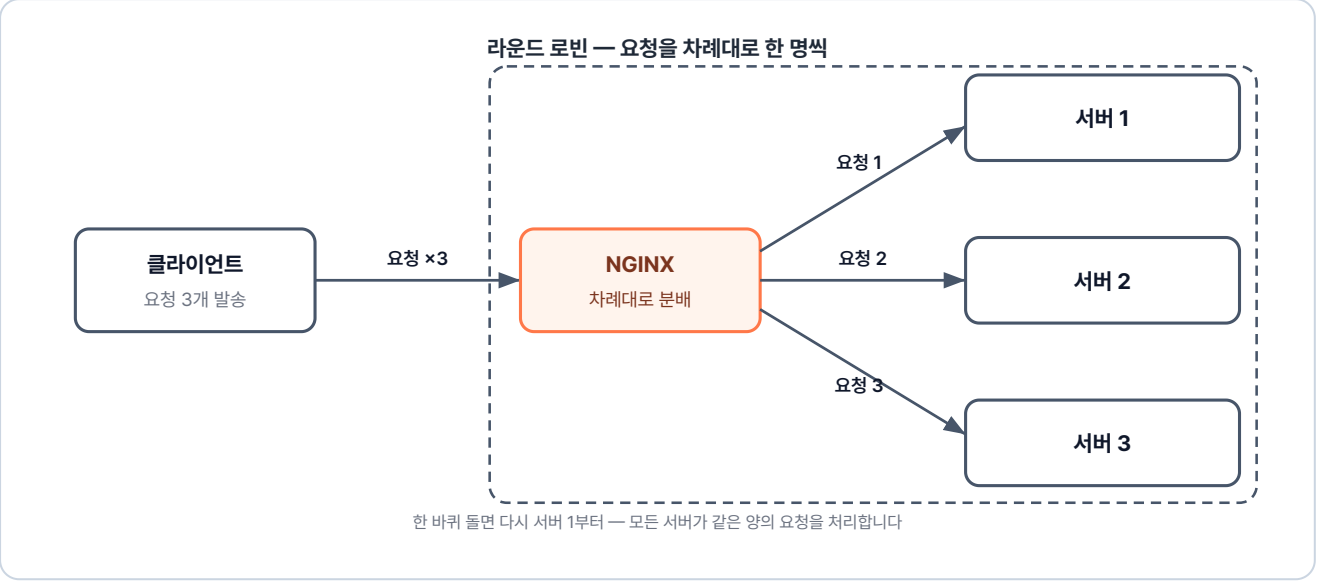


그림 3-14. 라운드 로빈 로드밸런싱 구조

NGINX는 `upstream` 블록 안에 서버 주소를 여러 개 적어 두면, 들어오는 요청을 등록된 순서대로 한 곳씩 돌아가며 보내 줍니다. 이 방식을 **라운드 로빈(Round Robin)** 이라고 부릅니다.

이번 실습은 `ex02` 폴더에 있습니다. 앞 실습과의 차이는 `nginx.conf`의 `upstream` 안에 **서버 주소 두 개가 컨테이너 이름으로 적혀 있다는 점**입니다.

```
ex02/
├── app1/                # 같은 이미지를 두 번 실행해 부하 분산
│   ├── Dockerfile       # nginx 이미지 + index.html 복사
│   └── index.html       # app1 페이지
├── lb/
│   ├── Dockerfile       # nginx 이미지 + nginx.conf 복사
│   └── nginx.conf       # upstream에 서버 둘을 묶은 로드밸런싱 설정
└── README.md            # 실습 안내
```

참고 **ex02/lb/nginx.conf. 로드밸런싱 설정**

```
upstream app1 {
    server app1-1:80;     # 첫 번째 서버. 컨테이너 이름으로 호출
    server app1-2:80;     # 같은 그룹에 두 번째 서버 추가
}

server {
    listen 80;
    location /app1 {
        proxy_pass http://app1/;          # 두 서버에 번갈아 분배. 라운드 로빈 방식
    }
}
```

그리고 같은 이미지로 컨테이너를 두 번 실행합니다. 이번 예제부터 사용자 정의 네트워크를 사용합니다.

터미널 **사용자 정의 네트워크에 LB와 두 app 연결하기**

```
# 1. 사용자 정의 네트워크 생성 (lb·app1-1·app1-2가 모두 이 네트워크에 연결됩니다)
docker network create ex02-network

# 2. app1 이미지 빌드
docker build -t app1 ex02/app1

# 3. 같은 이미지로 컨테이너 두 개 실행 (--name으로 다른 이름을 부여해 도커 DNS에 등록)
# app1-1을 ex02-network에 연결
docker run -dit --name app1-1 --network ex02-network app1
# app1-2를 ex02-network에 연결
docker run -dit --name app1-2 --network ex02-network app1

# 4. lb(NGINX) 빌드 + 실행 (-p로 외부에 80 포트만 노출, 내부 통신은 네트워크 이름으로)
docker build -t lb ex02/lb
docker run -dit --name lb --network ex02-network -p 80:80 lb
```

컨테이너 실행 후 `localhost:80/app1`에 여러 번 접속합니다.



그림 3-15. /app1 경로로 접속한 결과 (라운드 로빈)

두 컨테이너가 같은 이미지라, 화면만으로는 어느 서버가 응답한 건지 알 수 없습니다. 각 서버로 요청이 제대로 오는지 확인하기 위해 `docker logs` 명령을 실행합니다.

실 행 결 과

```
$ docker logs app1-1
2025/11/25 15:09:44 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2025/11/25 15:09:44 [notice] 1#1: start worker processes
2025/11/25 15:09:44 [notice] 1#1: start worker process 16
2025/11/25 15:09:44 [notice] 1#1: start worker process 17
2025/11/25 15:09:44 [notice] 1#1: start worker process 18
...
172.18.0.4 - - [25/Nov/2025:15:11:23 +0000] "GET / HTTP/1.0" 200 273 "-" "Mozilla/5.0
(Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36"
172.18.0.4 - - [25/Nov/2025:15:11:43 +0000] "GET / HTTP/1.0" 200 273 "-" "Mozilla/5.0
(Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36"
```

그림 3-16. app1-1 컨테이너 로그에 찍힌 요청

실행결과

```
$ docker logs app1-2
2025/11/25 15:09:54 [notice] 1#1: start worker process 20
2025/11/25 15:09:54 [notice] 1#1: start worker process 21
2025/11/25 15:09:54 [notice] 1#1: start worker process 22
...
172.18.0.4 - - [25/Nov/2025:15:11:33 +0000] "GET / HTTP/1.0" 200 273 "-" "Mozilla/5.0
(Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36"
172.18.0.4 - - [25/Nov/2025:15:11:53 +0000] "GET / HTTP/1.0" 200 273 "-" "Mozilla/5.0
(Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36"
```

그림 3-17. app1-2 컨테이너 로그에 찍힌 요청

두 로그를 비교해 보면, 두 서버 모두에 요청이 전달된 것을 확인할 수 있습니다. NGINX는 upstream에 서버가 둘 이상이면 별도 설정 없이 라운드 로빈으로 분배합니다.

3.2.6 캐싱

그림 3-16과 3-17을 보면 클라이언트의 모든 요청이 app 서버까지 도달합니다. 동일한 요청에 대해서도 매번 서버의 응답을 받아와야 하기 때문에, 사용자가 증가할수록 서버 부하가 심해집니다.

이 비효율을 해결해 주는 방법이 **캐싱(Caching)** 입니다. 이미지처럼 **잘 바뀌지 않는 응답을 NGINX 앞단에 저장**해 두면, 같은 요청이 다시 들어와도 서버까지 가지 않고 저장해 둔 응답을 바로 돌려줍니다.

캐시의 처리 결과에는 여러 상태가 있습니다. 이번 실습에서 다룰 상태는 두 가지입니다.

상태	의미
MISS	캐시에 저장된 응답이 없어 백엔드 서버까지 다녀온 상태
HIT	캐시에 보관된 응답을 백엔드 거치지 않고 바로 돌려준 상태



그림 3-18. 첫 번째 요청 (MISS) — 캐시가 비어있어 서버까지 요청 후 응답을 캐시에 저장



그림 3-19. 두 번째 요청 (HIT) — 캐시에 저장된 응답을 바로 반환, 서버 접근 없음

캐싱 실습은 ex03 폴더에 들어 있습니다. 파이썬 API 서버는 `/image.png`로 요청이 오면 이미지를 응답하고, 그 앞단의 NGINX가 이 응답을 캐싱합니다.

```
ex03/
├── api/                  # 백엔드. Flask 기반
│   ├── app.py            # /image.png 라우트에서 image.png 반환
│   ├── Dockerfile        # Python 이미지 + app.py·image.png 복사
│   └── image.png         # 응답으로 내려주는 이미지 파일
├── nginx/                # NGINX. 캐싱 + 프록시
│   ├── Dockerfile        # nginx 이미지 + nginx.conf 복사
│   └── nginx.conf        # proxy_cache 설정 + 프록시 라우팅
└── README.md             # 실습 안내
```

설정 파일에 새로 등장한 지시어는 두 가지입니다. `proxy_cache_path`는 캐시를 저장할 경로와 메모리 공간을 선언하고, `proxy_cache`는 그 공간을 어떤 요청 경로(location)에 적용할지 지정합니다.

참고 **ex03/nginx/nginx.conf. 캐싱 설정**

```
# 캐시를 저장할 경로와 메모리 공간 이름을 선언합니다.
proxy_cache_path /var/cache/nginx keys_zone=my_cache:10m;

server {
    listen 80;

    location / {
        proxy_pass http://api:5000;
        proxy_cache off;                                # 일반 경로는 캐시를 끕니다.
    }

    location = /image.png {                             # 이미지 파일 요청에 대해서만
        proxy_pass http://api:5000;
        proxy_cache my_cache;                           # 선언한 캐시 공간을 사용합니다.
        proxy_cache_valid 200 1m;                       # 정상 응답을 1분 동안 보관
        add_header X-Cache-Status $upstream_cache_status always;  # HIT/MISS 표시
        proxy_ignore_headers Cache-Control Expires;  # 백엔드 캐시 헤더 무시
    }
}
```

설정을 마쳤으니 터미널에서 컨테이너를 띄웁니다.

터미널 **api·nginx 컨테이너 함께 띄우기**

```
# 1. 사용자 정의 네트워크 생성
docker network create ex03-network

# 2. api 이미지 빌드 + 실행
docker build -t api ex03/api
docker run -dit --name api --network ex03-network api

# 3. nginx 캐싱 빌드 + 실행 (-p로 외부에 80 포트만 노출)
docker build -t nginx-cache ex03/nginx
docker run -dit --name nginx-cache --network ex03-network -p 80:80 nginx-cache
```

브라우저에서 `localhost:80/image.png`로 요청을 보내면 화면에 이미지가 뜹니다.



그림 3-20. 캐싱 실습 — 이미지 응답

이제 캐싱이 됐는지 확인해 보겠습니다. **개발자 도구(F12, 브라우저 디버그 창)** 의 Network 탭을 열고 **Disable cache**를 체크해 브라우저의 캐시를 막습니다. 그 다음 **응답 헤더**를 확인합니다. 첫 요청의 `X-Cache-Status`는 예상대로 **MISS**로 찍혀 있습니다.

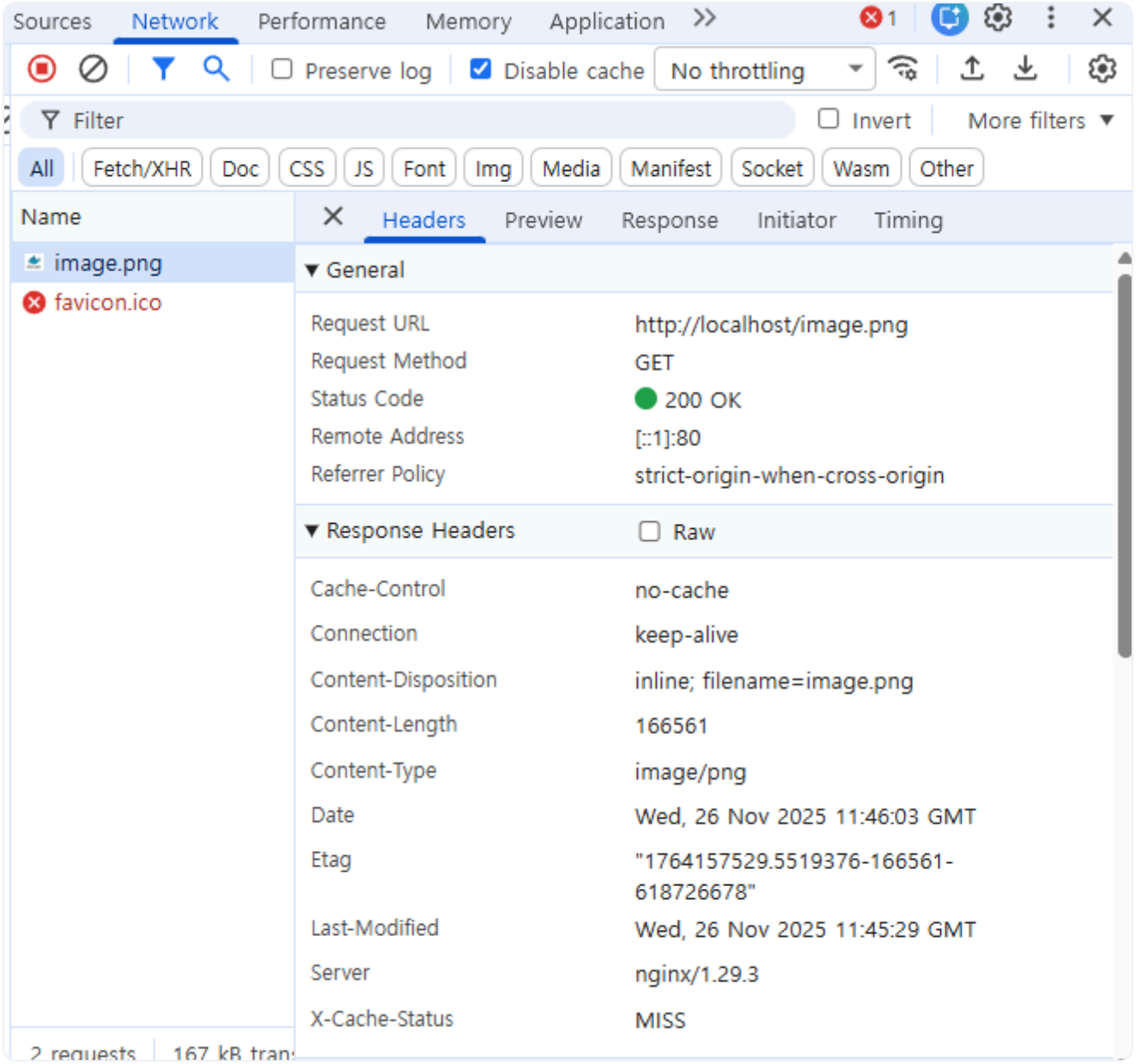


그림 3-21. X-Cache-Status: MISS 확인

새로고침을 한 번 더 하면 **HIT**로 바뀝니다. 두 번째 요청은 서버까지 가지 않고, NGINX가 캐시에 저장해 둔 응답을 돌려준 결과입니다.

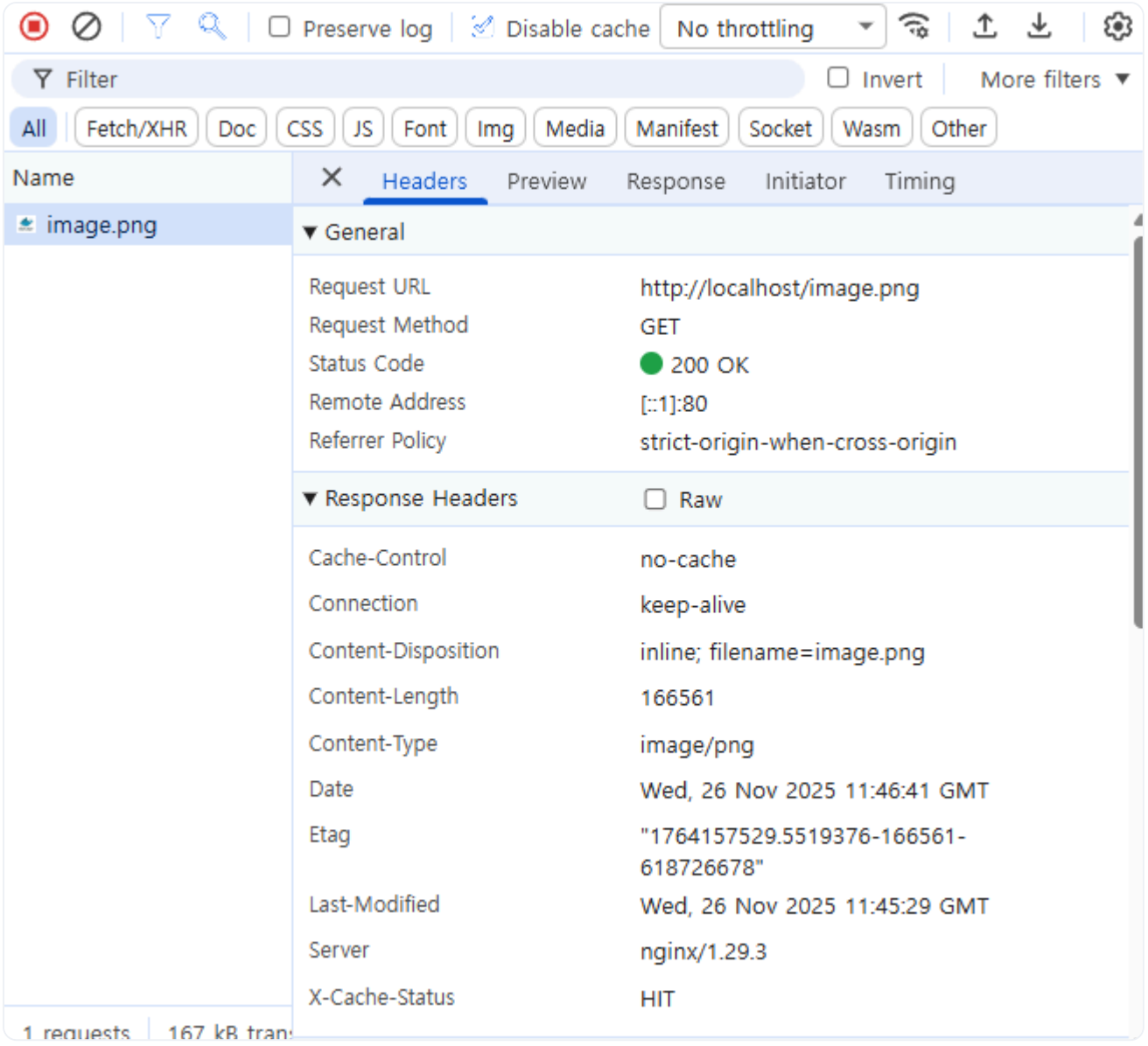


그림 3-22. X-Cache-Status: HIT 확인

'이제 컨테이너가 여러 개라도, 들어온 요청을 알맞은 곳으로 보낼 수 있겠다.'

3.3 Redis - 서버 여러 대가 함께 쓰는 공용 저장소

다음 날 점심을 먹고 자리에 돌아왔을 때 옆자리 동료가 의자를 돌려 말을 걸어 왔습니다.

동료 : "오픈 씨, 어제 만든 거 테스트해 봤는데요. 로그인은 되는데, 새로고침할 때마다 인증이 됐다 안 됐다 해요."

오픈이는 의자를 끌어당기며 화면을 같이 들여다봤습니다. 동료가 페이지를 반복해서 새로고침했습니다. 그러자 정상적인 페이지와 인증 실패 알림이 번갈아 나타났습니다.

'분명 로그인했는데, 왜 자꾸 풀리지.'

3.3.1 서버 여러 대면 생기는 세션 문제

원인은 로그인 기록이 처음 요청을 처리한 서버에만 남기 때문입니다. 사용자가 로그인하면 해당 **인증 정보(세션)** 는 요청을 받은 특정 서버의 메모리에 저장됩니다. 서버가 한 대일 때는 문제가 없지만, 서버가 두 대 이상이 되면 요청이 번갈아 처리됩니다. 이 과정에서 세션이 없는 서버로 요청이 전달될 때마다 **로그인이 풀립니다.**

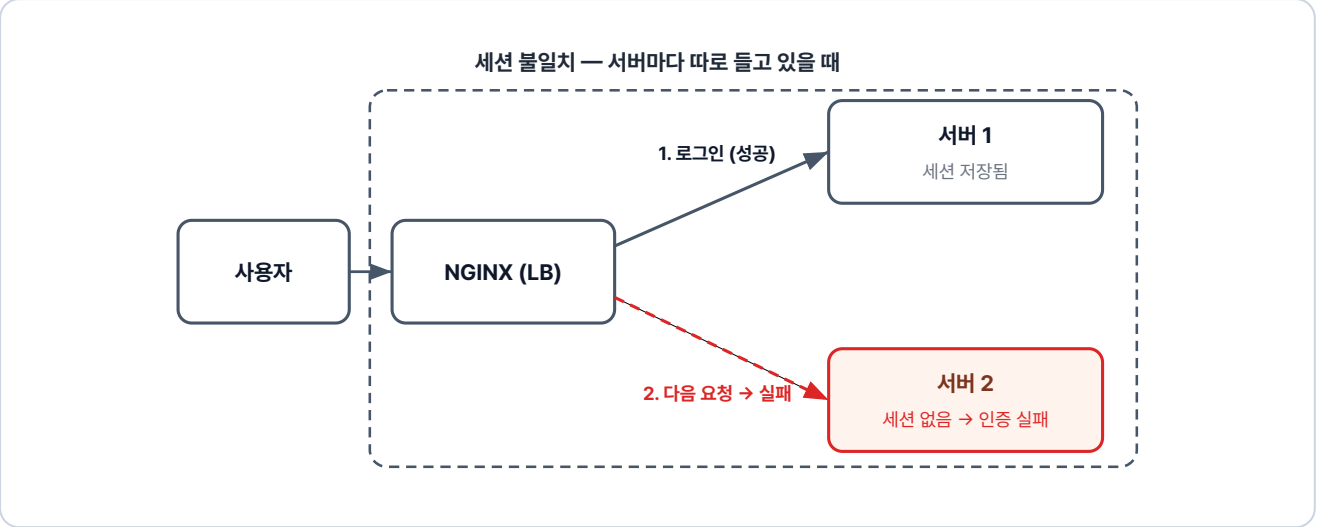


그림 3-23. 세션 불일치 — 1번 서버에 저장된 세션이 2번 서버엔 없어 인증 실패

3.3.2 Redis - 공용 세션 저장소

해결법은 세션을 각 서버 메모리에 따로 두지 않고 모든 서버가 공유하는 공간에 함께 두는 방식입니다. 그 공간을 제공하는 도구가 **Redis** 입니다.

> **Redis**: 메모리 기반의 키-값(Key-Value) 데이터베이스입니다. 데이터를 디스크가 아닌 메모리에 저장하기 때문에 처리 속도가 매우 빠릅니다. 그래서 세션 저장소나 캐싱처럼 짧은 시간 안에 빈번한 읽기/쓰기가 필요한 곳에 주로 사용됩니다.

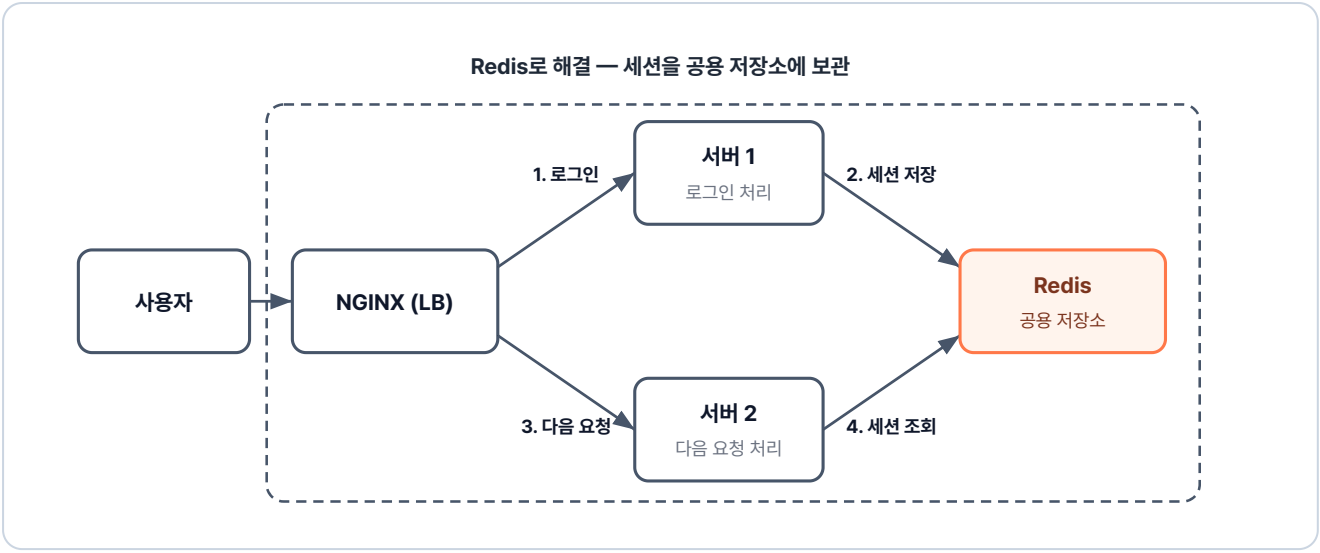


그림 3-24. Redis로 해결 — 세션을 공용 저장소에 보관하여 어느 서버에서든 조회 가능

로그인을 처리한 1번 서버는 세션을 Redis에 저장하고, 다음 요청을 받은 2번 서버는 내부 메모리 대신 **Redis에서 세션을 조회합니다.** 두 서버가 같은 세션을 공유하므로, 사용자는 한 번만 로그인하면 인증이 끊기지 않습니다.

3.3.3 실습: Redis로 세션 공유

ex04 폴더를 엽니다. Redis는 공식 이미지를 그대로 쓰고, API 서버 이미지는 직접 만듭니다.

```
ex04/
├── api/
│   ├── Dockerfile       # Python 이미지 + app.py 복사
│   └── app.py           # name 값을 Redis에 저장/조회하는 API
└── README.md            # 실습 안내
```

`app.py`는 두 개의 경로를 들고 있는 서버입니다. `/save`로 들어오면 name 값을 Redis에 저장하고, `/read`로 들어오면 저장된 값을 조회해 응답합니다.

이 서버를 실행할 컨테이너의 Dockerfile은 다음과 같습니다.

실습 5 **ex04/api/Dockerfile. Python API 이미지**

```dockerfile
FROM python:3.10-alpine                  # 파이썬 이미지 사용
WORKDIR /app                             # 작업 디렉토리 설정
COPY app.py .                            # 위의 app.py 복사
RUN pip install flask redis              # Flask + Redis 클라이언트 설치
CMD ["python", "app.py"]                 # 컨테이너 시작 시 app.py 실행
```

사용자 정의 네트워크를 만든 뒤 세 컨테이너를 같은 네트워크에 연결해 차례로 실행합니다.

터미널 **세 컨테이너를 같은 네트워크에 연결해 실행하기**

```
# 1. 사용자 정의 네트워크 생성
docker network create ex04-network

# 2. Redis 컨테이너 실행 (-p는 호스트에서 확인용으로 노출, 같은 네트워크 내 통신에는 불필요)
docker run -d --name redis --network ex04-network -p 6379:6379 redis

# 3. API 서버 두 대 실행 (같은 이미지, 다른 포트)
docker build -t api ex04/api
docker run -d --name api1 --network ex04-network -p 5001:5000 api
docker run -d --name api2 --network ex04-network -p 5002:5000 api
```

두 서버가 같은 Redis를 공유하는지 확인합니다. `localhost:5001/save`로 이름을 저장한 뒤, `localhost:5002/read`로 같은 이름이 나오는지 조회합니다.

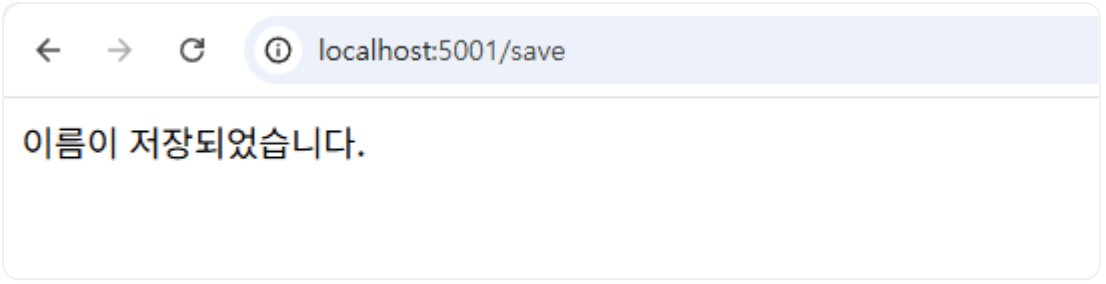


그림 3-25. api1에서 데이터 저장

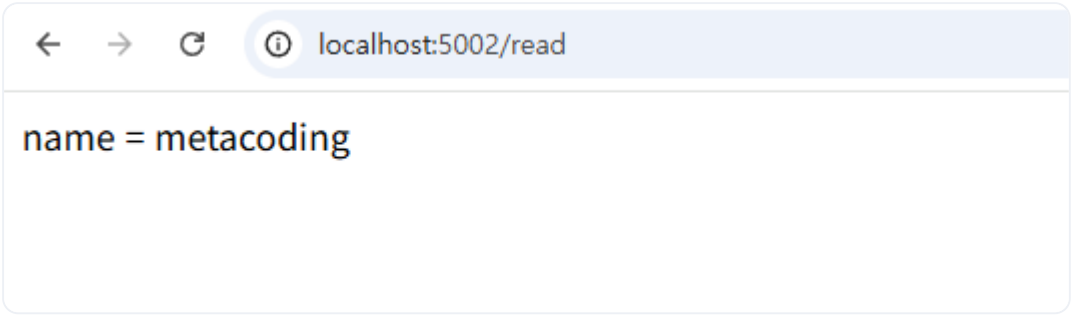


그림 3-26. api2에서 같은 데이터 조회

한 서버에 저장한 값이 다른 서버에서 그대로 조회됩니다. 두 서버가 하나의 Redis를 공유한다는 뜻입니다.

3.4 MySQL - 영구 데이터 저장하기

Redis는 메모리 위에서 동작하기 때문에, 컨테이너를 내렸다가 다시 실행하면 그 안의 값이 모두 사라집니다. 보관해야 할 데이터는 메모리가 아니라 데이터베이스에 둬야 합니다. 회원 정보를 저장할 데이터베이스를 컨테이너로 실행해 보겠습니다.

3.4.1 MySQL 컨테이너 실행하기

데이터베이스로는 MySQL을 사용합니다. ex05 폴더 안에는 DB 이미지를 만드는 `db` 폴더가 들어 있습니다.

```
ex05/
├── db/
│   ├── Dockerfile       # MySQL 이미지 + init.sql 복사
│   └── init.sql         # 테이블과 초기 데이터를 만드는 SQL
└── README.md            # 실습 안내
```

실습 6 **ex05/db/Dockerfile. MySQL + 초기 스크립트 이미지**

```
FROM mysql                                          # MySQL 공식 이미지 사용
COPY init.sql /docker-entrypoint-initdb.d           # 첫 기동 시 자동 실행될 SQL 복사
ENV MYSQL_USER=metacoding                           # 사용자 계정 설정
ENV MYSQL_PASSWORD=metacoding1234                   # 사용자 비밀번호
ENV MYSQL_ROOT_PASSWORD=root1234                    # root 비밀번호
ENV MYSQL_DATABASE=metadb                           # 기본 생성할 데이터베이스 이름
CMD ["--character-set-server=utf8mb4", "--collation-server=utf8mb4_unicode_ci"]
```

Dockerfile의 **환경 변수(ENV)** 에는 MySQL 계정과 비밀번호, 기본 데이터베이스 이름을 작성합니다.

> **Dockerfile에 비밀번호를 직접 적는 방식**
> 이번 실습에서는 과정을 단순하게 보여드리기 위해 비밀번호를 Dockerfile에 직접 적었습니다. 하지만 실무에서 이런 방식은 매우 위험합니다. 이미지를 빌드하는 순간 이미지를 가진 사람 누구나 비밀번호를 볼 수 있기 때문입니다. 그래서 실제 서비스를 운영할 때는 비밀번호 같은 민감한 정보를 이미지 내부에 기록하지 않고 외부에 따로 보관해 두었다가, 컨테이너가 실행되는 시점에 주입하는 방식을 사용합니다. 이 방식은 쿠버네티스에서 사용해보겠습니다.

ex05 폴더의 파일로 이미지를 빌드하고 컨테이너를 실행합니다.

터미널 **db 이미지 빌드와 실행**

```
docker build -t db ex05/db          # ex05/db 폴더의 Dockerfile로 이미지 빌드
docker run -dit -p 3306:3306 db     # db 컨테이너 실행. 호스트 3306 → 컨테이너 3306
```

실행결과

```
$ docker build -t db ex05/db
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV
"MYSQL_ROOT_PASSWORD") (line 7)
View build details: docker-desktop://dashboard/build/desktop-linux/s40kn0konmqqxbyvoabl3c6up
$ docker run -dit -p 3306:3306 db
ad3d9b0d81fefe86d6185bb1b865545d914f2be742c7046d5c6b1d984c62ff05
```

그림 3-27. MySQL 컨테이너 실행 로그

이제 DB 내부로 접속해보겠습니다. `docker ps`로 컨테이너 ID를 확인한 뒤, `docker exec`로 접속합니다.

터미널 **DB 컨테이너 상태 확인**

```
docker ps                    # 실행 중인 컨테이너 확인
docker exec -it ad3d bash    # MySQL 컨테이너 내부 접속
```

실행결과

```
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ad3d9b0d81fe db "docker-entrypoint.s…" 10 seconds ago Up 10 seconds 0.0.0.0:3306->3306/tcp,
[::]:3306->3306/tcp admiring_burnell
$ docker exec -it ad3d bash
bash-5.1#
```

그림 3-28. MySQL 컨테이너 내부 진입

컨테이너 안에서 MySQL에 접속합니다. 접속 정보는 Dockerfile에 적어 둔 값 그대로 입력합니다.

터미널 **DB 접속 확인**

```
mysql -u metacoding -p      # MySQL 접속. 비밀번호로 metacoding1234 입력
```

실행결과

```
bash-5.1# mysql -u metacoding -p
Enter password:
Welcome to the MySQL monitor. Commands end with ; or \g.
mysql>
```

그림 3-29. MySQL 접속 성공

접속 후 metadb를 선택하고 user_tb의 초기 데이터를 조회합니다.

터미널 **DB 초기화 결과 확인 쿼리**

```
use metadb;                 -- metadb를 사용할 DB로 선택
select * from user_tb;      -- user_tb 테이블의 전체 행 조회
```

실행결과

```
mysql> use metadb;
Database changed
mysql> select * from user_tb;
+----+------+
| id | name |
+----+------+
| 1 | ssar |
| 2 | cos |
+----+------+
2 rows in set (0.00 sec)
```

그림 3-30. user_tb 데이터 조회 결과

`init.sql`에 적어 둔 초기 데이터가 테이블 안에 그대로 들어와 있습니다.

3.5 Docker Compose - 여러 컨테이너를 한 번에

3.5.1 컨테이너를 하나씩 실행하는 한계

지금까지 프로비저닝, 외부 요청 처리 등 프로젝트에 필요한 요소들을 다뤘습니다. 하지만 이 요소들을 연동해 하나의 서비스로 구동하려면, 매번 컨테이너마다 빌드와 실행 명령을 반복해야 합니다. 따라서 여러 컨테이너를 통합해서 관리할 방법이 필요합니다.

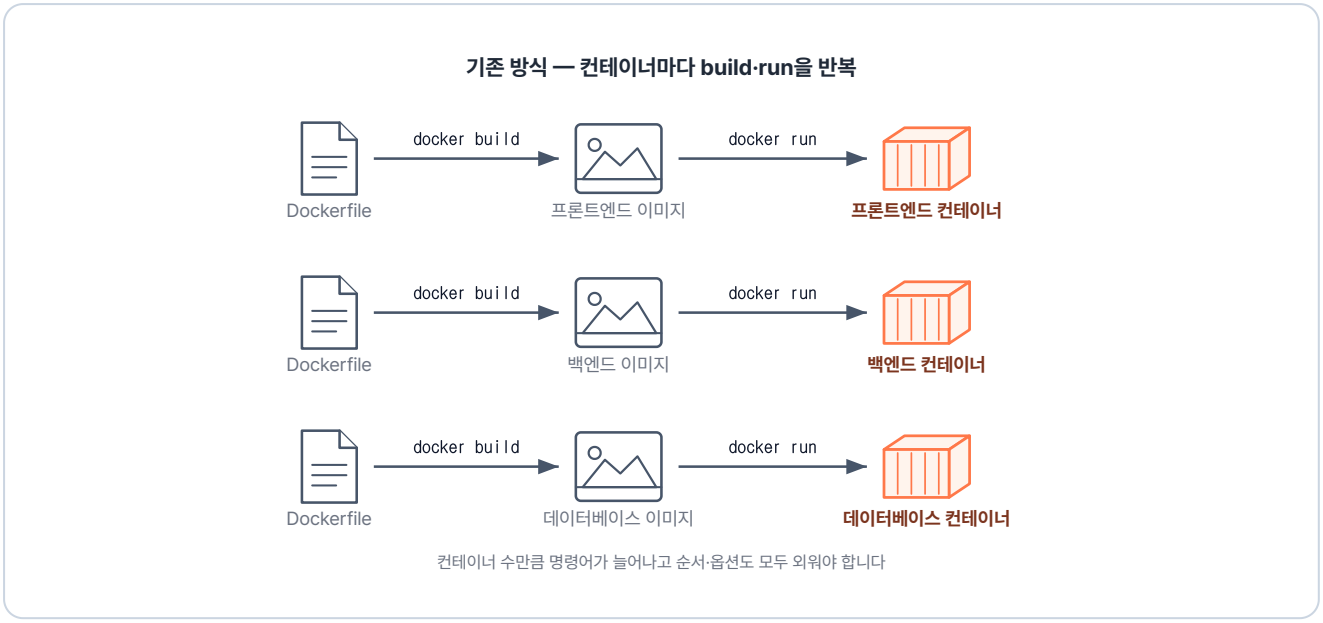


그림 3-31. 기존 방식 — 개별 빌드 및 실행 반복

이러한 번거로움을 명령어 하나로 해결해 주는 도구가 바로 **Docker Compose**입니다.

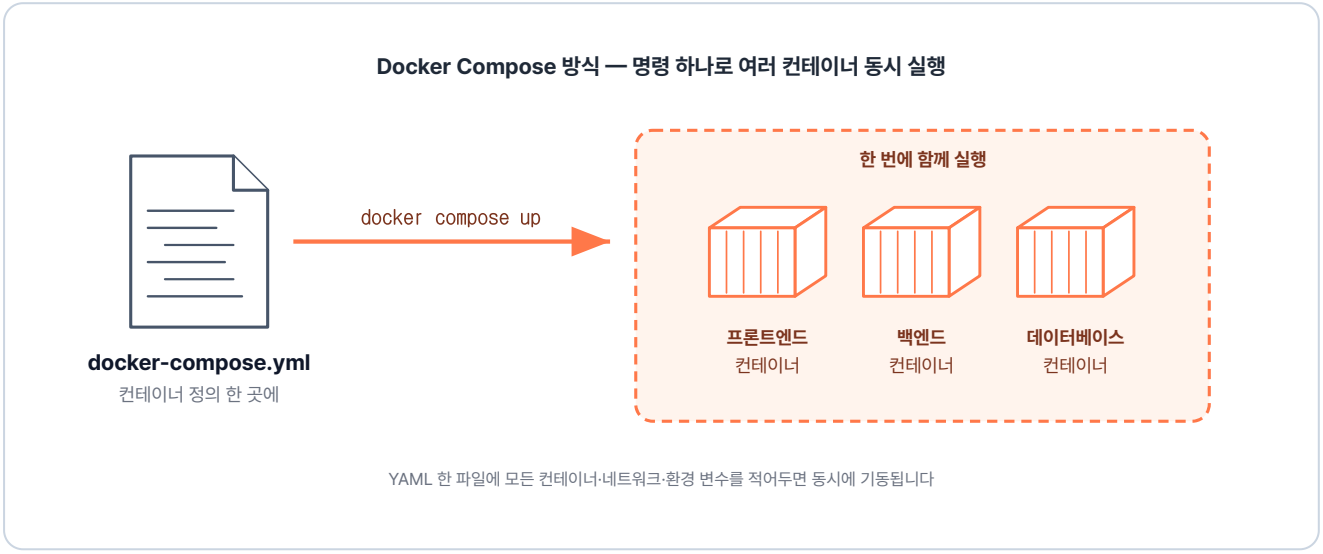


그림 3-32. Docker Compose 방식 — 한 번에 생성 및 연결

Docker Compose는 여러 컨테이너의 설정을 **하나의 YAML 파일(.yml)** 에 모아 관리합니다. 이 파일에 컨테이너 간의 네트워크 연결이나 실행 옵션을 미리 정의해 두면, **명령어 한 번으로 전체 컨테이너를 동시에 실행하거나 중지**할 수 있습니다.

3.5.2 docker-compose.yml 기본 구조

docker-compose.yml의 구조는 단순합니다. 자주 쓰이는 옵션을 정리해 보겠습니다.

참고 **docker-compose.yml 기본 골격**

```
services:
  <서비스명>:                    # 예: app, db, proxy 등
    container_name: <컨테이너명> # 실제로 생성될 컨테이너 이름
    image: <이미지명>            # Docker Hub에서 이미지를 가져와야 한다면 여기 이미지명 작성
    build: <경로>                # Dockerfile로 직접 이미지를 빌드한다면 여기 경로 입력
    ports:
      - "호스트포트:컨테이너포트"
    depends_on:
      - <먼저 떠야 할 서비스>
    environment:
      - KEY=VALUE                # 환경 변수 설정
    volumes:
      - <호스트경로:컨테이너경로> # 데이터 보관을 위한 볼륨 연결
    networks:
      - <네트워크명>             # 아래 networks에서 만든 망에 이 컨테이너를 참여시킴

networks:
  <네트워크명>:                  # 이 프로젝트에서 사용할 네트워크 이름을 생성
```

3.5.3 실습: Compose로 ex01 다시 만들기

실습을 위해 ex01을 Docker Compose로 다시 만들어 보겠습니다. 폴더 구조는 ex01에 컨테이너 실행 설정을 담은 `docker-compose.yml`이 더해진 형태입니다.

```
ex06/
├── app1/
│   ├── Dockerfile       # nginx 이미지 + index.html 복사
│   └── index.html       # app1 페이지
├── app2/
│   ├── Dockerfile       # nginx 이미지 + index.html 복사
│   └── index.html       # app2 페이지
├── lb/
│   ├── Dockerfile       # nginx 이미지 + nginx.conf 복사
```

```
|   └── nginx.conf        # 경로 라우팅 설정
├── docker-compose.yml    # app1·app2·lb를 함께 실행하는 Compose 설정
└── README.md             # 실습 안내
```

Compose는 네트워크를 자동으로 만들어 줍니다. 덕분에 `nginx.conf`에서 백엔드 주소로 서비스 이름을 그대로 사용할 수 있습니다.

실습 7 **ex06/docker-compose.yml. ex01을 Compose로 다시 짜기**

```yaml
services:              # 컨테이너 단위로 실행할 서비스 목록
  app1:                # 첫 번째 서비스. 다른 서비스가 호스트명으로 호출
    build:
      context: ./app1 # ./app1 폴더의 Dockerfile로 이미지 빌드
    networks:
      - ex06-network  # ex06-network에 연결
  app2:                # 두 번째 서비스
    build:
      context: ./app2
    networks:
      - ex06-network
  lb:                  # 로드밸런서 서비스
    build:
      context: ./lb
    ports:
      - 80:80          # 호스트 80 → 컨테이너 80. 외부 진입점
    networks:
      - ex06-network

networks:
  ex06-network:        # 세 서비스가 공유하는 사용자 정의 네트워크
```

ex06 폴더로 들어가서 Compose를 실행해 보겠습니다.

터미널 **docker compose up으로 일괄 실행**

```
cd ex06
docker compose up   # 현재 폴더의 docker-compose.yml 기준으로 일괄 실행
```

실행 결과

```
$ docker compose up
[+] up 7/7
✓ Image ex06-app1 Built
✓ Image ex06-app2 Built
✓ Image ex06-lb Built
✓ Network ex06_ex06-network Created
✓ Container ex06-app1-1 Created
✓ Container ex06-lb-1 Created
✓ Container ex06-app2-1 Created
Attaching to app1-1, app2-1, lb-1
```

그림 3-33. docker compose up 실행 결과

실행 후 `localhost:80/app1`과 `localhost:80/app2`로 접속하면 ex01과 같은 결과를 확인할 수 있습니다.

'매번 따로 만들던 컨테이너들을 이제는 하나로 모아 관리할 수 있겠구나.'

3.6 종합 실습 - 프론트엔드·백엔드·DB 연결하기

지금까지 학습한 내용을 바탕으로 프론트엔드·백엔드·DB를 연결해 보겠습니다.

3.6.1 전체 아키텍처

브라우저의 요청은 프론트엔드의 NGINX가 받아 백엔드로 전달합니다. 백엔드는 MySQL에서 회원 데이터를 조회해 브라우저로 응답합니다.

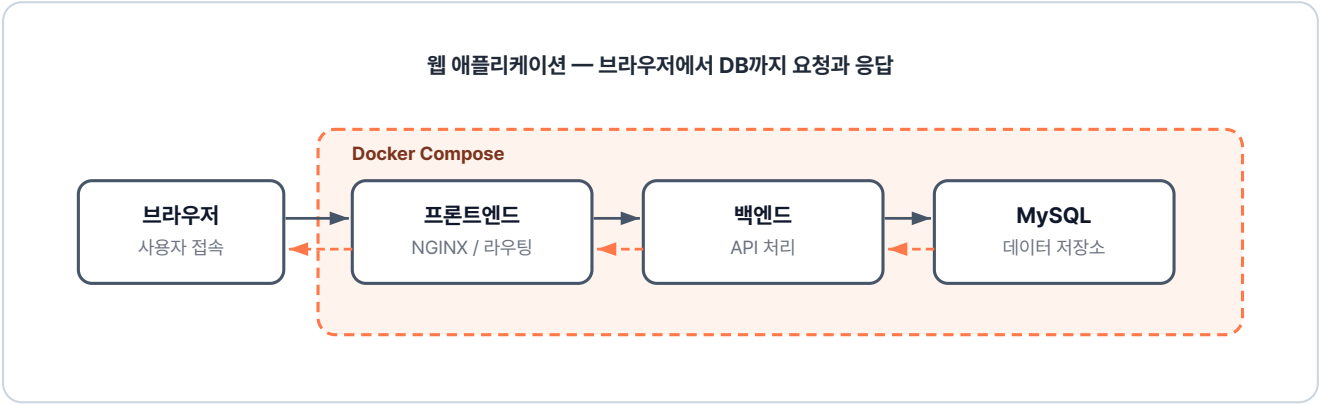


그림 3-34. 세 컨테이너로 구성되는 웹 애플리케이션 아키텍처

ex07 폴더에는 `backend` · `db` · `frontend` 폴더와 `docker-compose.yml`이 들어 있습니다.

```
ex07/
├── backend/              # Spring Boot 백엔드
│   ├── Dockerfile        # JDK 이미지 + entrypoint.sh 복사
│   └── entrypoint.sh     # Git clone + Gradle 빌드 + JAR 실행
├── db/                   # MySQL. ex05와 동일한 구성
│   ├── Dockerfile        # MySQL 이미지 + init.sql 복사
│   └── init.sql          # 테이블·초기 데이터 생성 스크립트
├── frontend/             # NGINX + HTML
│   ├── Dockerfile        # nginx 이미지 + index.html·nginx.conf 복사
│   ├── index.html        # 로그인/게시판 UI
│   └── nginx.conf        # /api 경로를 backend로 프록시
├── docker-compose.yml    # 세 서비스 + 네트워크 정의
└── README.md             # 실습 안내
```

3.6.2 Backend - Spring Boot 컨테이너

backend 폴더에는 Dockerfile과 `entrypoint.sh`가 있습니다. Dockerfile이 컨테이너의 실행 환경을 만드는 **설계도**라면, `entrypoint.sh`는 완성된 컨테이너가 시작될 때 수행할 구체적인 **행동 지침서**입니다.

이 예제의 `entrypoint.sh`는 컨테이너가 켜지는 시점에 **깃허브에서 소스 코드를 내려받아 빌드한 뒤, 서버를 실행**하는 동작을 담당합니다.

참고 **ex07/backend/entrypoint.sh. Git clone + 빌드 스크립트**

```
#!/bin/bash
# 백엔드 소스 내려받기
git clone https://github.com/metacoding-10-linux-docker/backend-server
cd backend-server                          # 클론한 폴더로 이동
chmod +x gradlew                           # 실행 권한 부여
./gradlew build                            # Gradle로 JAR 빌드
# prod 프로필로 실행
java -Dspring.profiles.active=prod -jar build/libs/*.jar
```

이 스크립트를 Dockerfile의 `ENTRYPOINT`로 지정하면, 컨테이너가 시작될 때 실행됩니다.

실습 8 **ex07/backend/Dockerfile. Spring Boot 이미지**

```
FROM eclipse-temurin:21-jdk                    # JDK 21 공식 이미지 사용
WORKDIR /var/current/app                       # 작업 디렉토리 설정
```

```
COPY entrypoint.sh /entrypoint.sh                # 위의 스크립트 복사
RUN apt-get update && apt-get install -y git     # git clone에 필요한 git 설치
ENTRYPOINT ["/entrypoint.sh"]                    # 컨테이너 시작 시 스크립트 실행
```

백엔드는 `/api/users` 요청이 오면 DB에서 회원 목록을 꺼내 JSON으로 돌려줍니다.

3.6.3 DB - MySQL 컨테이너

DB는 챕터 3.4에서 만든 MySQL 구성과 동일합니다. Dockerfile과 `init.sql` 모두 그대로 사용합니다.

파일	역할
Dockerfile	컨테이너로 띄우면 backend가 접속해 회원 정보를 읽고 쓸 수 있는 MySQL 저장소가 되는 이미지
init.sql	컨테이너 첫 기동 시 `user_tb` 테이블을 만들고 초기 회원 데이터를 넣어, backend가 회원 목록을 조회할 수 있게 하는 초기화 스크립트

3.6.4 Frontend - 정적 페이지 + API 프록시

frontend 폴더에는 `index.html`과 `nginx.conf`가 들어 있습니다.

파일	역할
index.html	사용자가 접속하면 backend의 `/api/users`를 호출해 받은 회원 목록을 화면에 그리는 정적 페이지
nginx.conf	정적 페이지(`/`)는 `index.html`로 응답하고, API 요청(`/api/`)은 backend로 넘기는, 브라우저 요청이 처음 도착하는 입구

3.6.5 docker-compose.yml

마지막으로 frontend·backend·db를 함께 실행하는 `docker-compose.yml`을 작성합니다.

실습 9 **ex07/docker-compose.yml. ex07 전체 구성**

```
services:
```

```
  backend:                  # Spring Boot 백엔드 서비스
    build:
      context: ./backend    # ./backend 폴더의 Dockerfile로 빌드
    environment:            # 컨테이너에 주입할 환경 변수
      # DB 접속 URL (호스트명 db = 아래 db 서비스명)
      SPRING_DATASOURCE_URL: "jdbc:mysql://db:3306/metadb?useSSL=false\
        &serverTimezone=UTC&allowPublicKeyRetrieval=true"
      SPRING_DATASOURCE_USERNAME: metacoding     # DB 계정
      SPRING_DATASOURCE_PASSWORD: metacoding1234 # DB 비밀번호
    networks:
      - ex07-network        # 공용 네트워크 연결

  db:                       # MySQL DB 서비스
    build:
      context: ./db
    networks:
      - ex07-network

  frontend:                 # nginx 프론트엔드 서비스
    build:
      context: ./frontend
    ports:
      - "80:80"             # 호스트 80 → 컨테이너 80. 외부 진입점
    networks:
      - ex07-network

networks:
  ex07-network:             # backend·db·frontend가 공유하는 네트워크
```

ex07 폴더로 들어가서 Compose를 실행해 보겠습니다.

터미널 **ex07 빌드와 실행**

```
cd ex07
docker compose up   # 현재 폴더의 docker-compose.yml 기준으로 일괄 실행
```

실행 결과

```
$ docker compose up
[+] up 7/7
✓ Image ex07-frontend Built
✓ Image ex07-backend Built
✓ Image ex07-db Built
✓ Network ex07_ex07-network Created
✓ Container ex07-db-1 Created
✓ Container ex07-frontend-1 Created
✓ Container ex07-backend-1 Created
Attaching to backend-1, db-1, frontend-1
```

그림 3-35. docker compose up 명령으로 프론트엔드·백엔드·DB가 함께 실행된 결과

백엔드는 첫 실행에서 깃허브 소스를 받고 Gradle 빌드를 거치므로 시간이 더 걸립니다. 빌드가 끝나면 브라우저에서 `localhost:80`에 접속합니다.

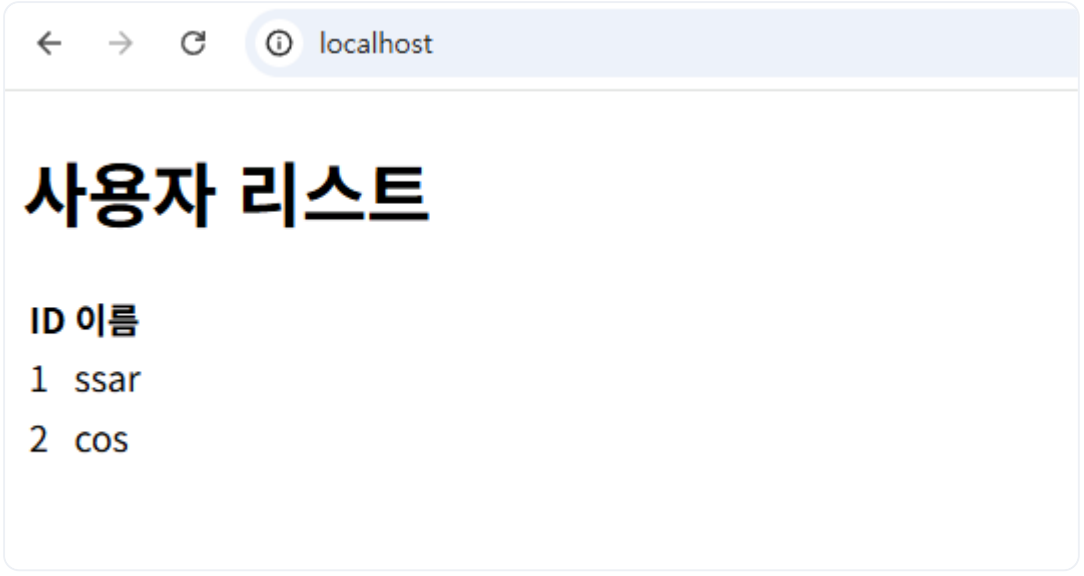


그림 3-36. 사용자 목록 조회 성공

처음 빈 리스트가 뜬 후, DB에서 조회가 완료되면 회원 목록이 화면에 표시됩니다.

금요일 오전 열 시. 완성한 프로젝트를 시연하기 위해 회의실에 모였습니다.

오픈이 : "준비한 시연 보여드리겠습니다."

오픈이가 터미널에 명령어를 입력했습니다.

팀장 : "환경 구성은 깔끔하게 됐네요. 그런데 운영 서버에 올린 다음 새벽에 컨테이너 중 하나라도 죽으면 누가 살리죠?"

오픈이는 끝내 그 질문에 답하지 못했습니다. 도커로 컨테이너를 **실행**할 수는 있어도, 그것들을 지속적으로 **관리**할 수는 없기 때문입니다. 다음 장에서는 실제 서비스 운영을 책임지는 쿠버네티스(Kubernetes)를 다뤄보겠습니다.

이것만은 기억하자

- **Dockerfile은 필요한 설정을 담아 둔 밀키트입니다.** 베이스 이미지·설치할 패키지·실행 명령을 적어 두면 어디서든 동일한 환경을 자동으로 재현합니다.
- **NGINX는 서버 앞단의 리버스 프록시입니다.** 경로 라우팅·로드밸런싱·캐싱이 핵심이며, 클라이언트에게는 NGINX 하나만 보입니다.
- **Redis는 여러 서버가 공유하는 메모리 저장소입니다.** 서버가 여러 대로 늘어도 세션을 이곳에 두면 로그인 상태가 흩어지지 않습니다.
- **사용자 정의 네트워크는 컨테이너끼리 이름으로 통신할 수 있게 합니다.** `host.docker.internal`처럼 호스트를 우회할 필요가 없어집니다.
- **Docker Compose는 여러 컨테이너를 한 파일로 관리하는 설계도입니다.** 명령 한 번으로 빌드·네트워크·실행 순서가 함께 처리됩니다.

CHAPTER

4

Kubernetes 시작하기

컨테이너를 자동으로 관리하다

이번 챕터가 끝나면

- 명령형과 선언형 방식의 차이를 이해합니다.
- 로컬에 클러스터를 띄우고 그 구조를 살펴봅니다.
- **Pod**를 직접 만들 때 생기는 문제를 자동 복구·스케일링으로 해결합니다.
- 무중단 배포와 되돌리기를 손에 익힙니다.

챕터 4. Kubernetes 시작하기

회의실을 나선 오픈이의 머릿속은 복잡했습니다. 시연은 문제없이 끝났지만, 팀장이 마지막에 던진 질문이 마음에 걸렸습니다. 운영 중에 컨테이너가 죽으면 어떻게 되살릴 것인가. 도커만으로는 그 답이 보이지 않았습니다.

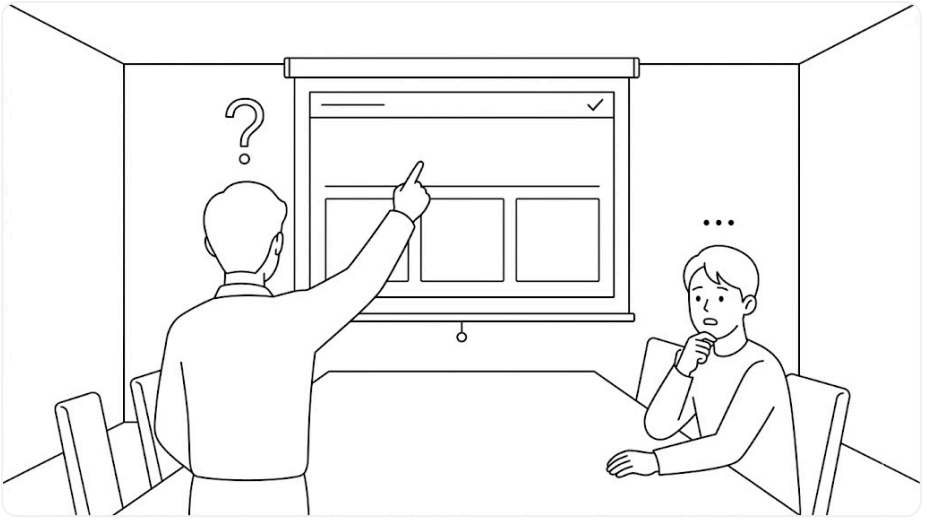


그림 4-1. 팀장의 운영 질문에 답이 막힌 순간

점심을 먹고 돌아오는 길에 선배에게 고민을 털어놨습니다.

오픈이 : "도커를 공부하면 끝일 줄 알았는데, 운영에서 컨테이너가 죽거나 트래픽이 몰릴 때 어떻게 해야 할지 모르겠어요."

선배 : "도커만으로는 운영까지 감당하기 어려워요. 그래서 쿠버네티스를 같이 알아야 해요. 컨테이너에 대한 관리는 쿠버네티스 담당이거든요."

선배의 말을 들은 오픈이는 퇴근 후 집에서 노트북을 열었습니다. 그리고 쿠버네티스가 무엇이고, 어떤 역할을 하는지 살펴보기로 했습니다.

이번 챕터에서 다룰 쿠버네티스 리소스의 전체 구조는 다음과 같습니다.



그림 4-2. 챕터 4 한눈에 보기 - 전체 구조

준비하기

Minikube 설치

쿠버네티스 실습에 쓰는 로컬 클러스터 도구 Minikube를 설치합니다. OS에 맞는 명령을 터미널에서 실행합니다.

터미널 **Minikube 설치**

```
# macOS
brew install minikube

# Windows
winget install Kubernetes.minikube
```

예제 코드

이 책의 실습 코드는 GitHub 레포 하나에 챕터별 폴더로 담겨 있습니다. 아래 명령으로 레포를 한 번 git clone 합니다. 이후 챕터마다 해당 폴더로 이동해 실습합니다.

터미널 **예제 코드 클론**

```
git clone https://github.com/metacoding-10-linux-docker/start
```

이번 챕터는 `ex08` ~ `ex10` 폴더를 사용합니다.

완성된 코드는 아래 주소에서 확인하세요.

터미널 **완성 코드**

```
https://github.com/metacoding-10-linux-docker/final
```

4.1 Kubernetes - 원하는 상태를 유지하기

4.1.1 도커는 실행 명령, Kubernetes는 약속 유지

도커와 쿠버네티스는 컨테이너를 다루는 방식이 다릅니다.

- **Docker(명령형)**: "이 컨테이너를 실행해라."
- **Kubernetes(선언형)**: "이 서비스를 원하는 상태로 유지해라."

이 선언형 방식은 프랜차이즈 본사가 가맹점을 관리하는 방식과 비슷합니다.

본사는 가맹점을 직접 운영하지 않습니다. 대신 가맹점 수나 모습 같은 **규칙**을 정하고 그 규칙이 지켜지는지를 관리합니다. 만약 매장이 문을 닫으면 **새 매장을 다시 열어** 개수를 맞추고, 수요가 많아지면 **새로운 매장을 추가**합니다.

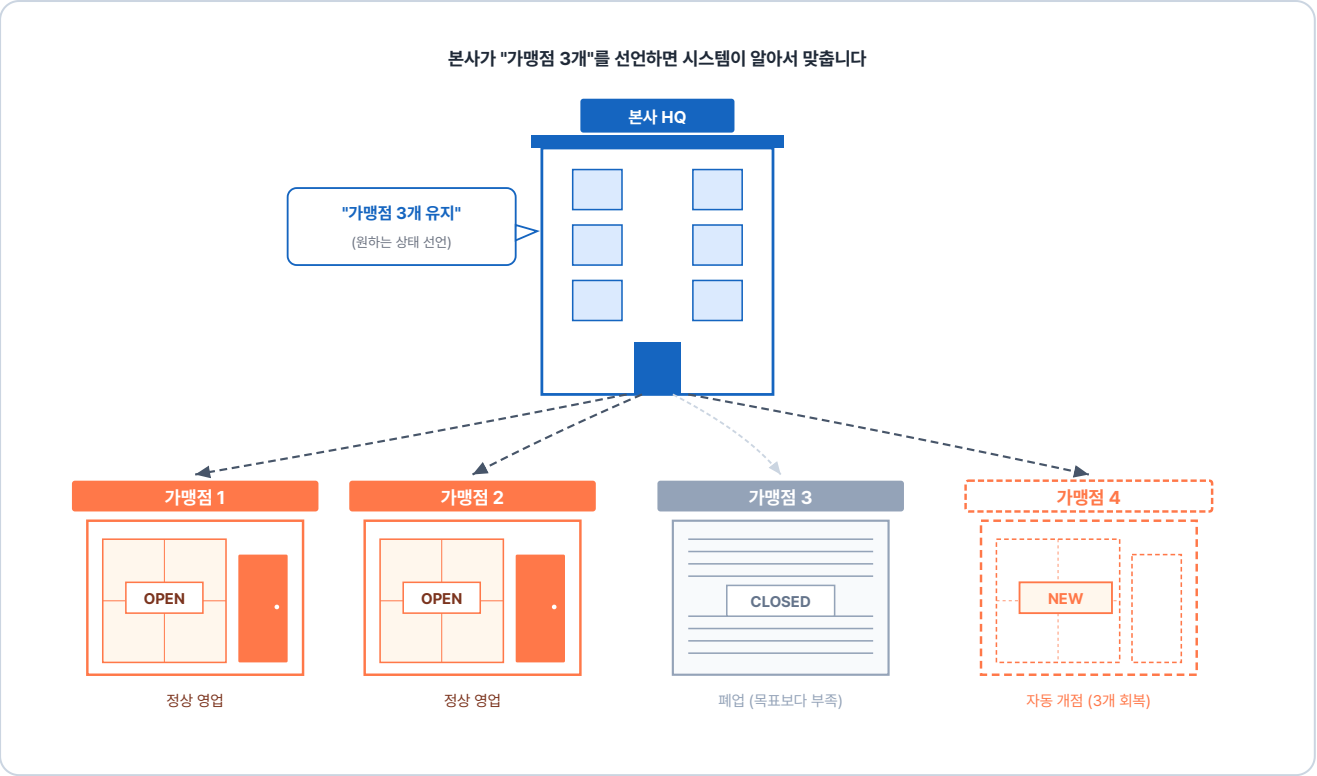


그림 4-3. 본사가 "가맹점 3개 유지"를 선언하면 시스템이 가맹점 개수를 자동으로 맞추는 구조

쿠버네티스도 마찬가지입니다. 컨테이너 수나 상태 규칙 등 **원하는 상태**를 선언해 두면, 시스템이 자동으로 그 상태를 **유지**합니다.

4.1.2 컨트롤 플레인과 워커 노드

쿠버네티스는 크게 **컨트롤 플레인(Control Plane)** 과 **워커 노드(Worker Node)** 로 이루어집니다.

> **노드(Node)**: 컨테이너가 실제로 돌아가는 컴퓨터 한 대를 말합니다. 실제 서비스 환경에서는 수십~수백 대의 노드(서버)가 모여 하나의 **클러스터(Cluster)** 를 이룹니다. 컨트롤 플레인과 워커 노드 모두 이런 노드 위에서 동작합니다.

컨트롤 플레인은 **프랜차이즈 본사 관리팀**처럼 컨테이너를 관리할 **규칙을 정하고 지시를 내립니다**. 지시를 받은 워커 노드가 실제로 컨테이너를 **실행하고 관리합니다**.

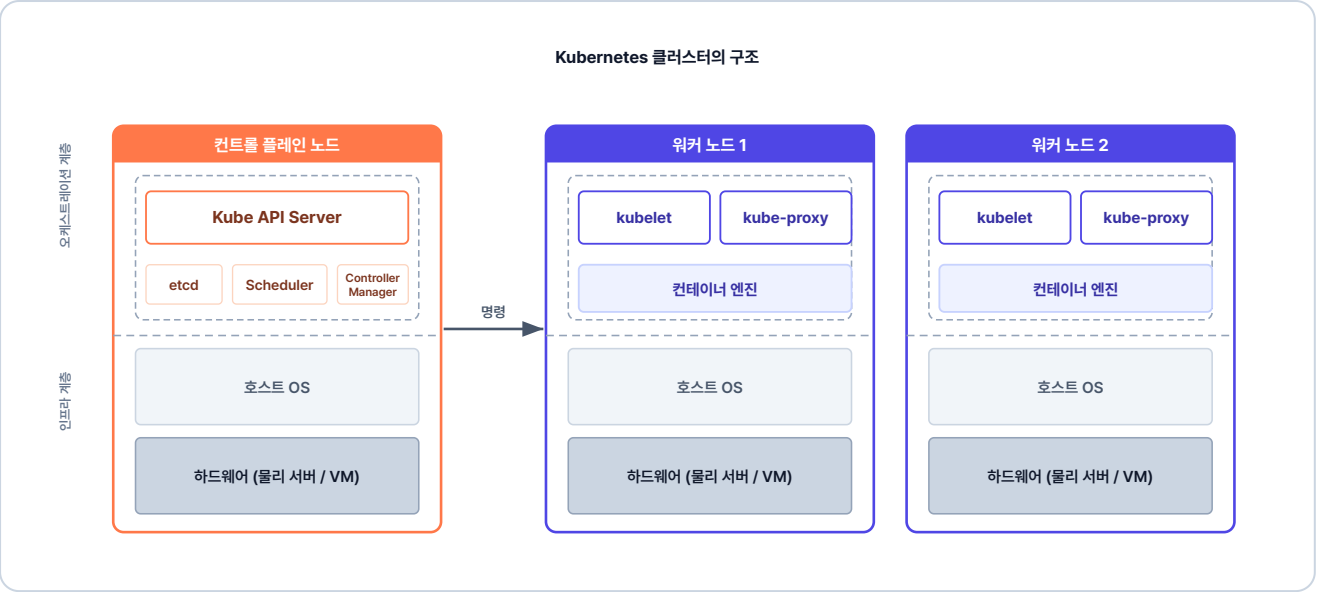


그림 4-4. Kubernetes 클러스터의 구조

쿠버네티스는 **컨테이너를 Pod라는 단위로 묶어 관리합니다**.

4.1.3 Kubernetes의 동작 원리

개발자가 명령을 내려 Pod 하나가 만들어지는 일은 **프랜차이즈 본사가 새 가맹점을 여는 흐름**과 비슷합니다. 크게 **명령 → 판단 → 지시** 세 단계로 일어납니다. 하나씩 들여다보겠습니다.

1단계 - 개발자가 클러스터에 명령을 보낸다

개발자가 클러스터에 명령을 보내는 일은 **프랜차이즈 본사에 가맹점을 신청하는 것**과 같습니다. 외부에서 들어오는 모든 요청은 **가장 먼저** 컨트롤 플레인 내부의 **Kube API Server**를 통과합니다.

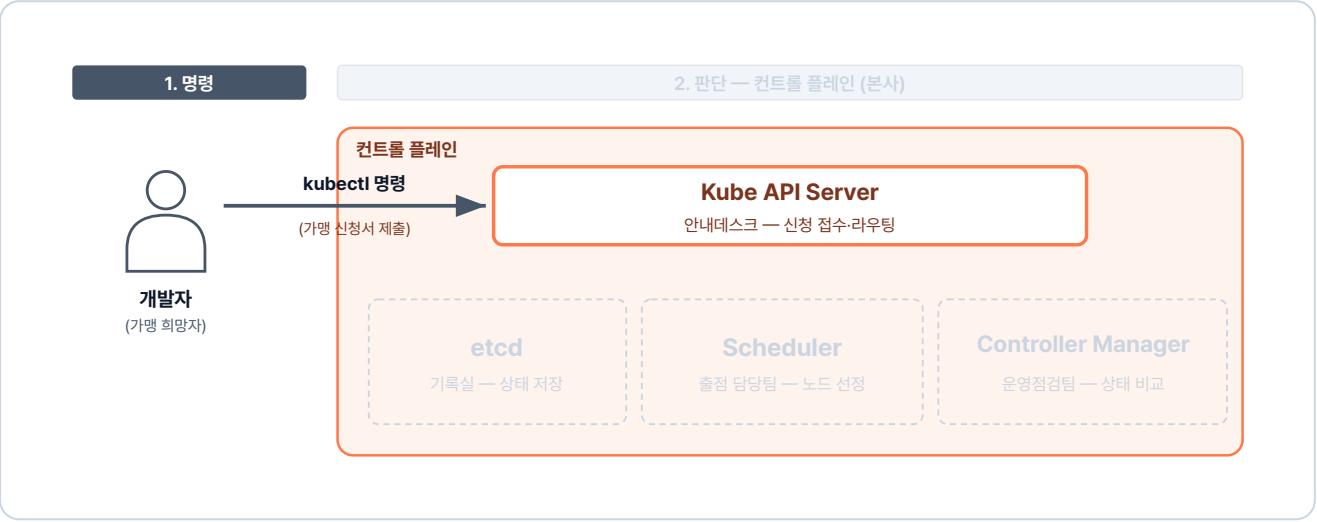


그림 4-5. 명령 단계 — 개발자의 명령이 Kube API Server에 도착하는 길

2단계 - 컨트롤 플레인이 명령을 어떻게 처리할지 판단한다

본사는 신청을 받으면 **새 매장의 위치**를 정하고 **기존 매장들의 상태**를 점검합니다. 이 과정에서 컨트롤 플레인의 구성 요소는 **Kube API Server**를 통해 서로 상태를 주고받으며 **이 명령을 어떻게 처리할지** 결정합니다.

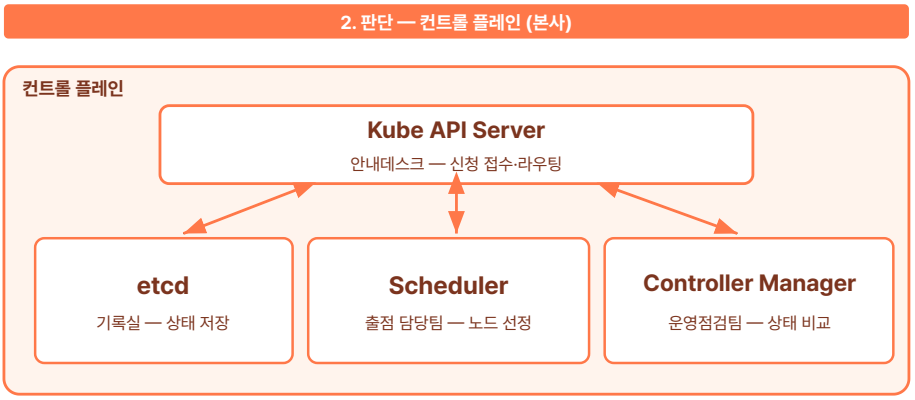


그림 4-6. 판단 단계 — Kube API Server를 중심으로 구성 요소들이 협의하는 모습

Kube API Server는 들어온 요청을 바탕으로 **클러스터의 상태**를 업데이트하고, 이 정보를 **etcd**에 보관합니다. **Scheduler**는 보관된 정보를 확인해 새 Pod를 **어느 노드에 배치할지** 정합니다. 동시에 **Controller Manager**는 클러스터 상태가 **원래 설정된 모습대로 유지되고 있는지** 감시합니다.

3단계 - 컨트롤 플레인이 워커 노드 kubelet에 지시한다

판단이 완료되면 **Kube API Server**가 워커 노드의 **kubelet**에게 지시를 내립니다. 지시를 받은 **kubelet**은 노드 안에서 컨테이너를 실행하고, 그 상태가 정의된 모습에서 벗어나지 않게 **계속 점검합니다**.

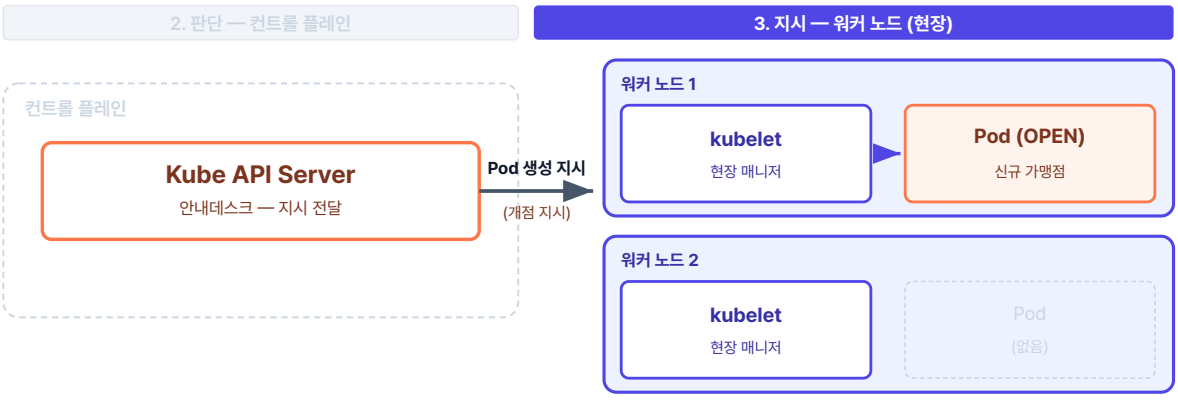


그림 4-7. 지시 단계 — kubelet이 컨트롤 플레인의 지시를 받아 Pod를 띄우는 모습

이런 명령 → 판단 → 지시를 거쳐 **Pod가 생성됩니다**.

4.1.4 Kubernetes 핵심 리소스 한눈에

쿠버네티스 세상에서는 작업 단위를 **리소스(Resource)** 라고 부릅니다. 사용자의 요청은 여러 리소스를 거쳐 실제 컨테이너에 도달합니다. 전체적인 흐름은 다음과 같습니다.

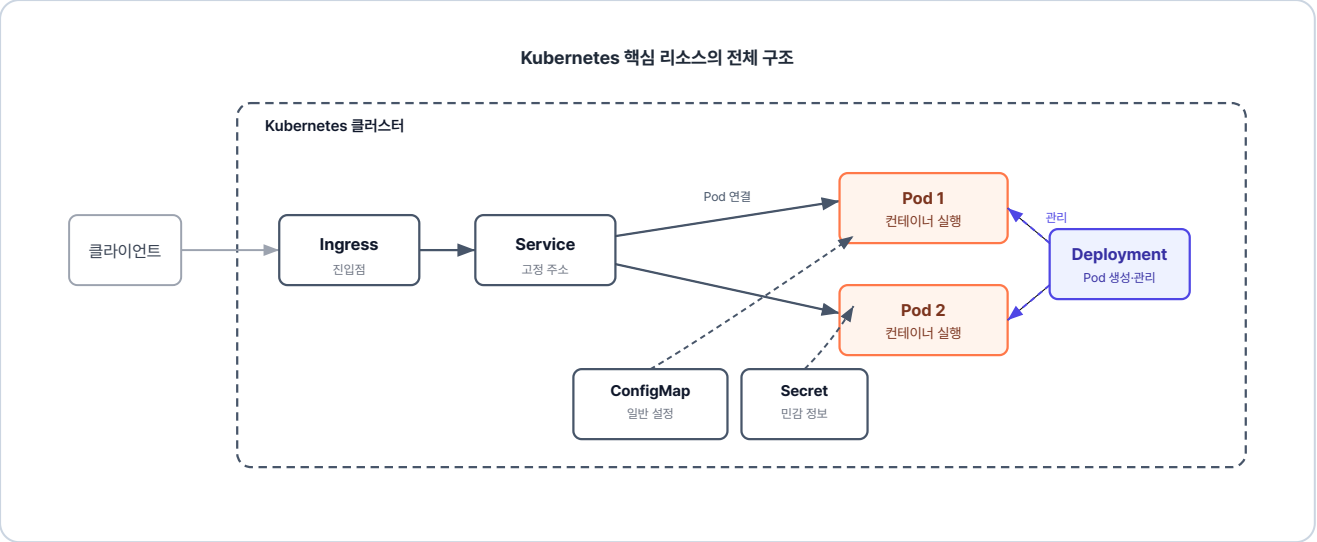


그림 4-8. Kubernetes 핵심 리소스의 전체 구조

이번 챕터에서는 가장 기본이 되는 Pod와 Deployment를 먼저 다뤄 보겠습니다.

4.2 Minikube - 로컬에 만드는 작은 클러스터

4.2.1 노트북 한 대로 클러스터 흉내 내기

컨트롤 플레인과 워커 노드를 모두 갖추려면 서버가 여러 대 필요해 보이지만, 연습 단계에서는 노트북 한 대로 충분합니다. Minikube를 쓰면 내 컴퓨터 안에 쿠버네티스 환경을 구축할 수 있습니다.

Minikube는 **노드 하나로 동작합니다**. 이 노드가 컨트롤 플레인 역할과 워커 노드 역할을 함께 수행합니다.

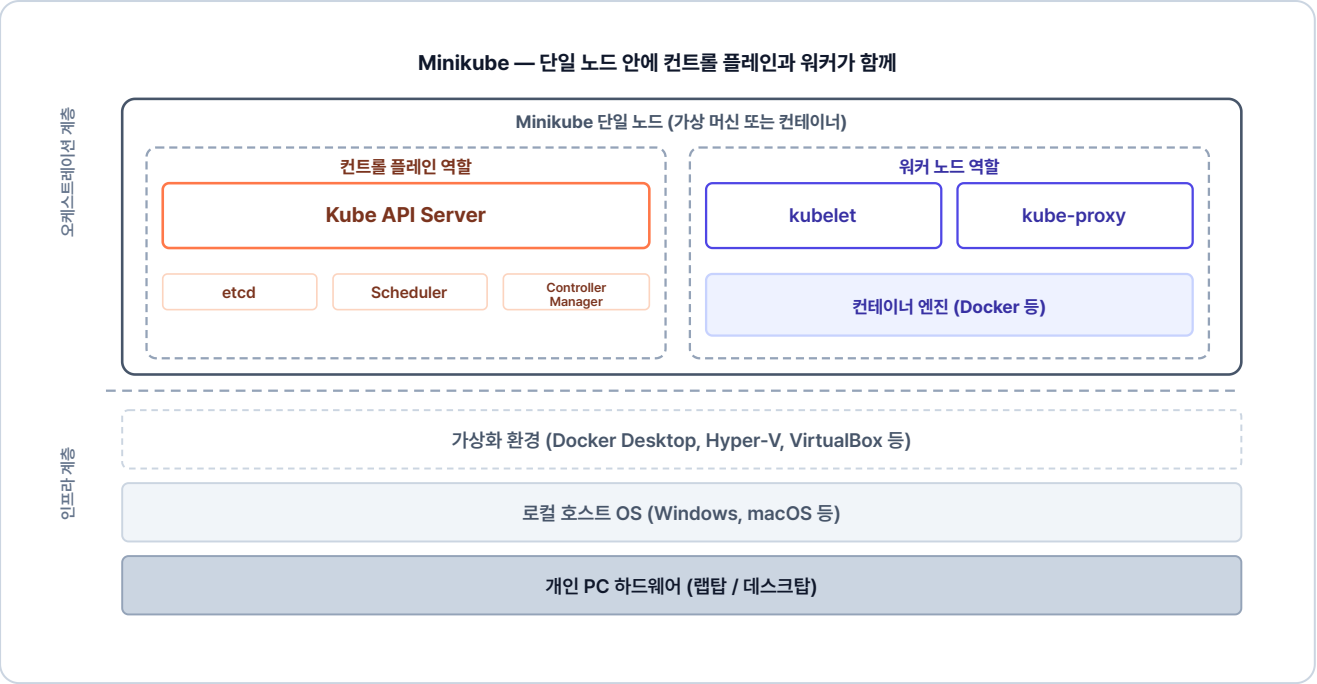


그림 4-9. Minikube는 단일 노드 안에 컨트롤 플레인과 워커 노드 기능이 함께 들어간 구조

이렇게 구조가 단순해 리소스도 적게 사용합니다. Minikube에서 작성한 설정 파일은 실제 운영 환경에서도 그대로 사용할 수 있습니다.

'이제 Minikube를 직접 실행해 봐야겠다.'

4.2.2 클러스터 시작

앞의 준비하기를 참고해 Minikube를 먼저 설치합니다. 그 다음 아래 명령으로 노트북 안에 클러스터를 시작합니다.

터미널 **클러스터 시작**

```
minikube start          # 클러스터 시작
```

실행 결과

```
$ minikube start
* Microsoft Windows 11 Pro 10.0.26200.6584 Build 26200.6584 의 minikube v1.37.0
* 자동적으로 docker 드라이버가 선택되었습니다. 다른 드라이버 목록: hyperv, ssh
* Docker Desktop 드라이버를 루트 권한으로 사용 중
* "minikube" 클러스터의 "minikube" primary control-plane 노드를 시작하는 중
* 기본 이미지 v0.0.48를 가져오는 중 ...
* docker container (CPUs=2, 메모리=3500MB) 를 생성하는 중 ...-
* 쿠버네티스 v1.34.0 을 Docker 28.4 런타임으로 설치하는 중###
* bridge CNI (Container Networking Interface) 를 구성하는 중 ...
* Kubernetes 구성 요소를 확인...
  - 이미지 gcr.io/k8s-minikube/storage-provisioner:v5 사용 중
* 애드온 활성화 : storage-provisioner, default-storageclass
* 끝났습니다! kubectl이 "minikube" 클러스터와 "default" 네임스페이스를 기본적으로 사용하도록 구성되었습니다
```

그림 4-10. minikube start 실행 결과

4.2.3 자주 쓰는 Minikube 명령어

실습에서 자주 쓰게 될 명령어는 다음과 같습니다.

명령어	설명
`minikube start`	클러스터를 시작합니다.
`minikube stop`	실행 중인 클러스터를 중지합니다.
`minikube status`	현재 클러스터의 구동 상태를 확인합니다.
`minikube service <서비스명> --url`	생성한 서비스에 접근할 수 있는 URL을 생성합니다.

4.3 Pod - Kubernetes의 최소 단위

Minikube 환경을 구성했으니 이제 실제 쿠버네티스의 리소스를 하나씩 생성해 볼 차례입니다. 도커만 쓸 때와 무엇이 다른지, 쿠버네티스라는 시스템이 컨테이너를 다루는 최소 단위부터 확인해 보겠습니다.

4.3.1 Pod란?

Pod(파드)는 하나 이상의 컨테이너로 구성된 단위입니다. 쿠버네티스는 컨테이너를 하나씩 직접 다루지 않고, 이 Pod를 단위로 관리합니다.

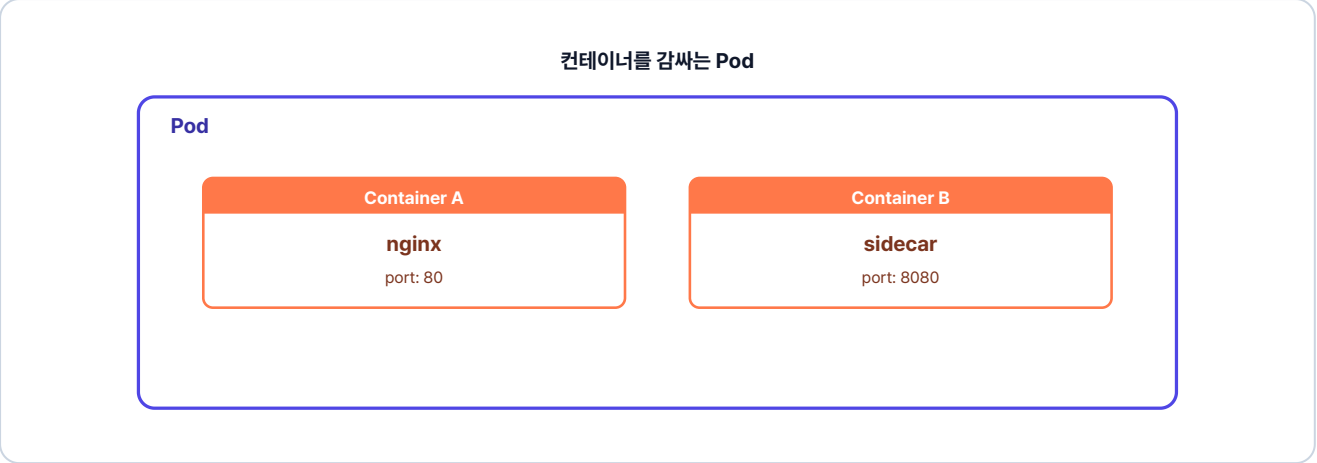


그림 4-11. Pod의 구조

왜 컨테이너 대신 Pod를 쓸까요

컨테이너는 서로 독립적으로 동작해 네트워크·스토리지·생명주기를 함께 관리하기 어렵습니다. Pod는 이를 다음과 같이 해결합니다.

- **하나처럼 묶어서 관리**: 여러 컨테이너를 하나로 묶어 같이 생성·종료되고, 함께 동작하도록 합니다.
- **네트워크 공유**: Pod 내부 컨테이너들은 하나의 IP를 공유해 같은 서버 안처럼 통신합니다.
- **관리 기준 단위**: 스케줄링, 확장, 장애 복구 등은 컨테이너가 아니라 Pod 기준으로 이루어집니다.

그래서 쿠버네티스는 컨테이너가 아닌 Pod를 '배포·운영의 최소 단위'로 관리합니다.

4.3.2 명령어로 Pod 만들기

클러스터에 명령을 내릴 때 사용하는 도구는 **kubectl**입니다. "이런 상태를 만들어 달라"고 쿠버네티스에 요청하는 명령어입니다.

가장 먼저 시도해 볼 방식은 도커와 비슷한 명령어 기반의 생성입니다.

터미널 **명령어로 첫 Pod 생성**

```
kubectl run hello-pod1 --image=nginx   # nginx 이미지로 Pod 생성
```

실행 결과

```
$ kubectl run hello-pod1 --image=nginx
pod/hello-pod1 created
```

그림 4-12. kubectl run으로 Pod 생성

Pod가 생성됐습니다. 이 방식은 `docker run`과 비슷한 방식입니다.

'이건 명령형 방식이네. 그럼 선언형으로는 어떻게 만들까?'

4.3.3 YAML로 Pod 만들기

선언형은 **원하는 상태**를 YAML 파일로 작성하여 클러스터에 반영하는 방식입니다. 명령어로 일일이 지시하는 대신, 쿠버네티스가 YAML을 읽어 원하는 상태로 리소스를 생성합니다.

실습 코드는 `ex08` 폴더에 있습니다.

```
ex08/
├── hello-pod2.yml   # nginx Pod 정의
└── README.md        # 실습 안내
```

다음은 nginx 이미지로 Pod 하나를 만드는 YAML입니다.

실습 1 **ex08/hello-pod2.yml. nginx Pod 정의**

```yaml
apiVersion: v1
kind: Pod                  # 리소스 종류
metadata:
  name: hello-pod2         # 리소스명
spec:
  containers:
    - name: hello-container
      image: nginx:1.20        # 이미지
```

이렇게 작성한 파일을 클러스터에 적용할 때는 **kubectl apply** 명령어를 사용합니다.

터미널 **YAML로 Pod 생성**

```
kubectl apply -f ex08/hello-pod2.yml   # YAML로 Pod 생성
```

실행결과

```
$ kubectl apply -f ex08/hello-pod2.yml
pod/hello-pod2 created
```

그림 4-13. kubectl apply로 Pod 생성

YAML 파일을 통해 Pod가 생성되었습니다.

4.3.4 Pod 조회

Pod 두 개를 만들었으니 이제 목록을 조회해 보겠습니다.

터미널 **Pod 목록 조회**

```
kubectl get pod   # Pod 목록 조회
```

실행결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
hello-pod1 1/1 Running 0 8m39s
hello-pod2 1/1 Running 0 23s
```

그림 4-14. Pod 목록 조회 결과

Pod의 상태나 컨테이너, IP 같은 상세 정보를 보고 싶다면 **kubectl describe** 명령어를 사용합니다.

터미널 **Pod 상세 정보 조회**

```
kubectl describe pod hello-pod2       # Pod 상세 정보 조회
```

실행결과

```
$ kubectl describe pod hello-pod2
Name: hello-pod2
Namespace: default
Priority: 0
Service Account: default
Node: minikube/192.168.49.2
Start Time: Sun, 15 Mar 2026 15:41:30 +0900
Labels: <none>
Annotations: <none>
Status: Running
IP: 10.244.0.43
IPs:
 IP: 10.244.0.43
Containers:
 hello-container:
  Container ID: docker://65158d189989f41de190d076d953eb0da58c0f28cbfed088da7d91018bdcb305
  Image: nginx:1.20
  Image ID: docker-
pullable://nginx@sha256:38f8c1d9613f3f42e7969c3b1dd5c3277e635d4576713e6453c6193e66270a6d
  Port: <none>
  Host Port: <none>
  State: Running
   Started: Sun, 15 Mar 2026 15:41:31 +0900
  Ready: True
  Restart Count: 0
  Environment: <none>
  Mounts:
```

그림 4-15. Pod 상세 조회

4.3.5 자주 쓰는 kubectl 명령어

실습에서 자주 쓰게 될 기본 명령어는 다음과 같습니다.

명령어	설명
`kubectl apply -f <파일>`	YAML로 리소스 생성/업데이트
`kubectl get <리소스>`	리소스 목록 조회
`kubectl describe <리소스> <이름>`	리소스 상세 정보

명령어	설명
`kubectl delete <리소스> <이름>`	리소스 삭제
`kubectl exec -it <Pod명> -- bash`	Pod 내부 접속
`kubectl logs <Pod명>`	Pod 로그 확인

Pod를 생성한 오픈이는 이런 의문이 들었습니다.

'YAML로 정의해 뒀으니, 이제 쿠버네티스가 알아서 관리해 주는 걸까?'

4.4 Deployment - 자동 복구·스케일링·무중단 배포

4.4.1 Pod 하나를 직접 만들면 생기는 문제

방금 만든 Pod로 실험을 해 보겠습니다. 운영 중에 누군가의 실수로 Pod가 삭제되거나, 프로그램 오류로 종료된다면 어떤 일이 벌어질까요.

YAML로 생성한 hello-pod2를 삭제해 보겠습니다.

터미널 **Pod 삭제 후 자동 복구 여부 확인**

```
kubectl delete pod hello-pod2   # hello-pod2 삭제
kubectl get pod                 # 남은 Pod 목록 확인
```

실행 결과

```
$ kubectl delete pod hello-pod2
pod "hello-pod2" deleted
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
hello-pod1 1/1 Running 0 19m
```

그림 4-16. Pod 삭제 후 목록을 조회하면 hello-pod2가 사라져 있습니다

hello-pod2는 목록에서 사라졌고, 시간이 지나도 다시 나타나지 않았습니다.

'아, 그냥 선언만 해서는 자동 복구가 안 되네.'

Pod 자체는 사라져도 자동으로 복구되지 않습니다. 그래서 Pod를 **원하는 상태로 계속 유지하는 리소스**가 필요합니다. 그 리소스가 쿠버네티스의 **Deployment(디플로이먼트)** 입니다.

> **Deployment**: 파드(Pod)에 대한 **선언적 업데이트**를 제공하는 쿠버네티스 리소스입니다. 원하는 상태를 선언해 두면 시스템이 그 상태를 자동으로 유지하며, 자동 복구(Self-healing)·롤링 업데이트·롤백·스케일링까지 함께 담당합니다.

4.4.2 Deployment - 원하는 상태를 유지하는 매뉴얼

Deployment는 "**Pod를 몇 개 유지할지, 문제가 생기면 어떻게 교체할지**"를 적어 두는 **매뉴얼**과 같습니다. 선언한 상태와 실제 상태를 비교해, 차이가 생기면 스스로 조정해 원하는 상태로 되돌립니다.

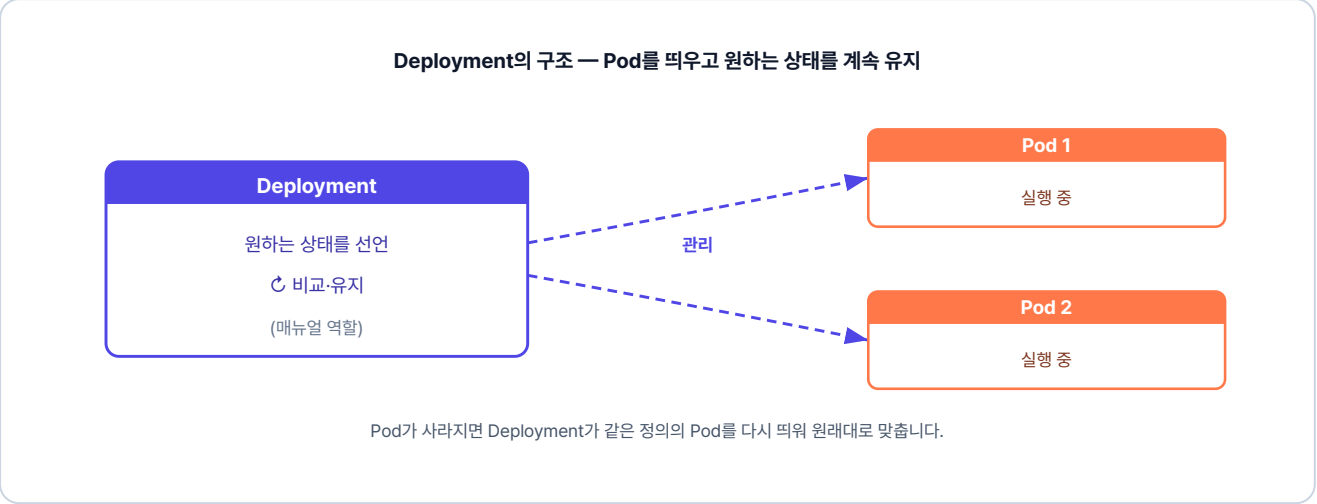


그림 4-17. Deployment의 구조 — Pod를 띄우고 원하는 상태를 계속 유지

실습 코드는 `ex09` 폴더에 있습니다.

```
ex09/
├── deploy-ex01.yml  # nginx Pod 1개를 유지하는 Deployment 정의
└── README.md        # 실습 안내
```

다음은 nginx Pod 한 개를 항상 유지하도록 정의한 Deployment YAML입니다.

실습 2 **ex09/deploy-ex01.yml. Deployment 정의 (replicas 1)**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:                    # pod에 대한 상태 지정
  replicas: 1            # 생성할 pod 수 지정
  selector:
    matchLabels:
      app: nginx         # 'app: nginx' 라벨이 붙은 Pod를 내 관리 대상으로 지정
  template:
    metadata:
      labels:
        app: nginx       # pod에 붙일 라벨
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.20
```

여기서 중요한 개념은 **selector** 와 **labels** 의 연결입니다.

Deployment가 관리할 Pod를 생성할 때, Pod의 **labels** 영역에 이름표를 붙입니다. 그리고 Deployment의 **selector** 에 똑같은 이름표를 지정해 두면, 쿠버네티스는 그 이름표를 가진 Pod들만 식별하여 관리합니다.

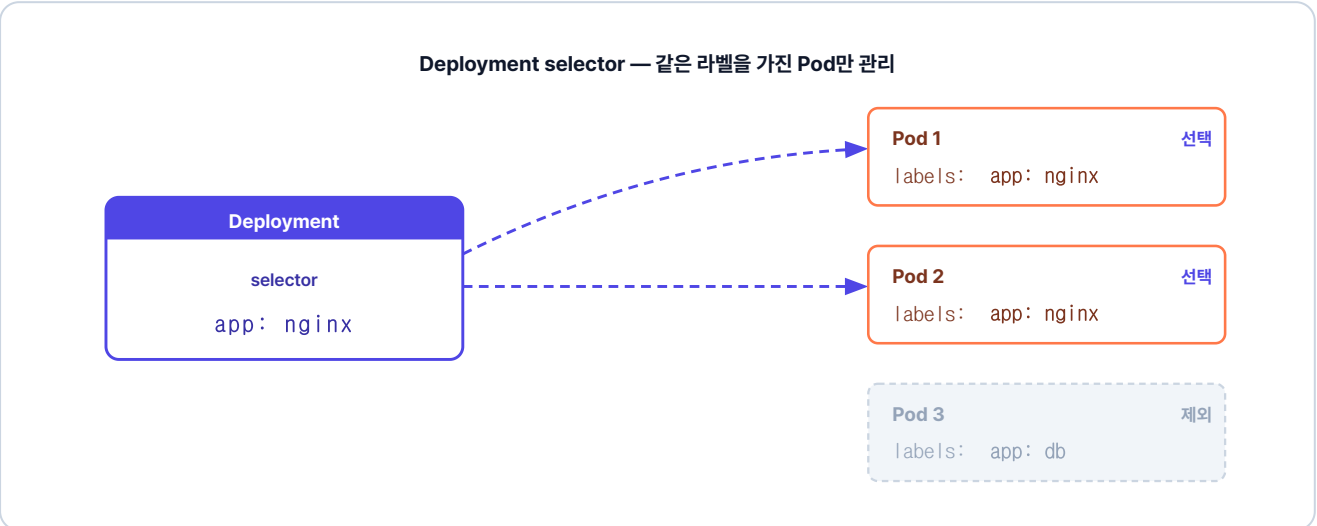


그림 4-18. selector가 지정한 라벨(app: nginx)을 가진 Pod만 골라 관리, 다른 라벨은 건드리지 않음

작성한 파일을 적용해 보겠습니다.

터미널 **Deployment 생성**

```
kubectl apply -f ex09/deploy-ex01.yml   # Deployment 생성
kubectl get pod                         # Pod 목록
```

실행결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
hello-pod1 1/1 Running 0 19m
nginx-deploy-756b46b54c-8q8p1 1/1 Running 0 5s
```

그림 4-19. Deployment가 만든 Pod가 뜬 모습

이제 Deployment가 정말로 원하는 상태를 유지하는지 확인하기 위해, 현재 실행 중인 Pod를 전부 지웁니다.

터미널 **Pod 전체 삭제 후 자동 복구 확인**

```
kubectl delete pod --all          # 모든 Pod 삭제
kubectl get pod                   # 목록 재확인
```

실행결과

```
$ kubectl delete pod --all
pod "hello-pod1" deleted
pod "nginx-deploy-756b46b54c-8q8p1" deleted
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-deploy-756b46b54c-2sv8x 1/1 Running 0 4s
```

그림 4-20. Pod를 다 지워도 Deployment가 자동으로 새 Pod를 띄웁니다

hello-pod1은 그대로 사라졌지만 Deployment가 만든 Pod는 삭제된 후 새 Pod로 다시 실행되었습니다.

'팀장님이 물어봤던 게 이거였구나. 원하는 상태만 정해 두면, 그다음은 Deployment가 자동으로 관리해 주네.'

4.4.3 ReplicaSet - Pod 개수 자동 관리

운영 환경에서는 부하를 분산하기 위해 Pod를 여러 개 실행합니다. 이때 Pod의 개수를 일정하게 유지하는 리소스가 바로 **ReplicaSet(레플리카셋)** 입니다. 사용자가 선언한 개수와 실제 작동 중인 Pod의 개수를 비교해, 모자라거나 남는 만큼 조절합니다.

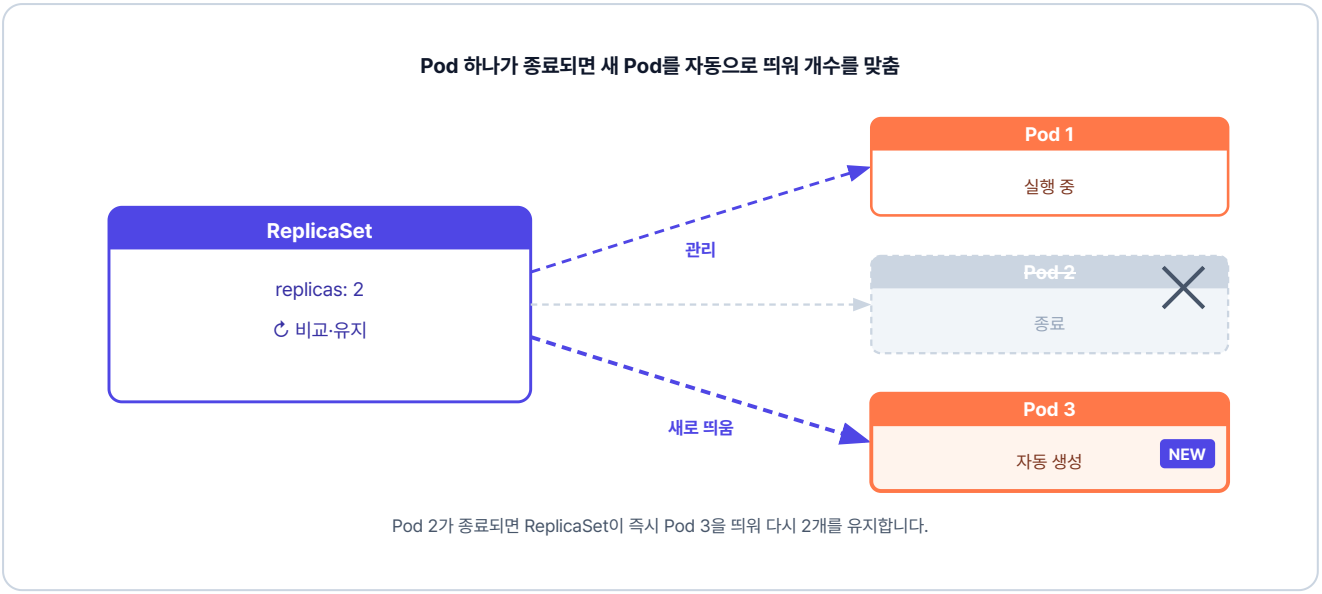


그림 4-21. Pod 하나가 종료되면 새 Pod를 자동으로 띄워 개수를 맞춤

> **ReplicaSet과 Deployment**
>
> ReplicaSet은 쿠버네티스에 별도로 존재하는 리소스이지만, 롤링 업데이트나 롤백 같은 배포 기능이 없습니다. 그래서 실무에서는 **ReplicaSet을 직접 만들지 않고 Deployment를 사용**합니다. Deployment의 **replicas 속성**을 통해 내부적으로 ReplicaSet을 자동으로 만들어 Pod 개수를 유지하고, 롤링 업데이트와 롤백까지 함께 제공합니다.

실습 코드는 `ex10` 폴더에 있습니다.

```
ex10/
├── deploy-ex02.yml  # nginx Pod 4개를 유지하는 Deployment 정의
└── README.md        # 실습 안내
```

다음은 `replicas`를 4로 늘려 같은 Pod 4대를 동시에 띄우고 유지하는 Deployment YAML입니다.

실습 3 **ex10/deploy-ex02.yml. replicas 4와 롤링 업데이트 전략**

```
apiVersion: apps/v1
```

```yaml
kind: Deployment
metadata:
  name: nginx-replica
spec:
  replicas: 4            # pod 수 지정

  strategy:
    type: RollingUpdate  #  롤링 업데이트 전략
    rollingUpdate:
      maxSurge: 4        # 업데이트 중 최대 4개까지 추가 생성
      maxUnavailable: 0  # 기존 Pod를 먼저 종료하지 않음 (무중단 배포)

  selector:              # 라벨이 app: nginx 인 pod를 관리
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx       # pod에 붙일 라벨
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.20
```

이 YAML 파일을 클러스터에 적용하고, 설정한 대로 4개의 Pod가 생성되는지 확인합니다.

터미널 **replicas 4 적용**

```
kubectl apply -f ex10/deploy-ex02.yml   # Deployment YAML 적용
kubectl get pod                          # 생성된 Pod 목록 확인
```

실행결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-replica-756b46b54c-l5l7f 1/1 Running 0 5s
nginx-replica-756b46b54c-t8fhl 1/1 Running 0 5s
nginx-replica-756b46b54c-vp5g5 1/1 Running 0 5s
nginx-replica-756b46b54c-vzv7n 1/1 Running 0 5s
```

그림 4-22. replicas 설정대로 Pod 4개가 생성

4.4.4 롤링 업데이트 - 끊김 없는 버전 교체

운영 중인 서비스를 새 버전으로 교체할 때, 여러 Pod를 수동으로 교체하면 번거로울 뿐만 아니라 그 과정에서 서비스가 중단되기 쉽습니다. 쿠버네티스의 **롤링 업데이트(Rolling Update)** 는 기존 버전을 한꺼번에 내리지 않고 새 버전으로 조금씩 교체하여, **서비스가 멈추지 않는 무중단 배포**를 제공합니다.

앞서 만든 `deploy-ex02.yml`에는 이 롤링 업데이트 동작을 정하는 `strategy` 블록이 들어 있습니다. 각 속성은 다음과 같습니다.

- **maxSurge**: 업데이트 중 replicas로 지정한 수보다 잠시 더 늘어나는 Pod의 수
- **maxUnavailable**: 업데이트 중 replicas로 지정한 수보다 잠시 줄어들어도 되는 Pod의 수

예를 들어 replicas: 4, maxSurge: 4, maxUnavailable: 0이라면, 기존 4개는 그대로 둔 상태에서 새 버전 4개를 추가합니다. 새 Pod가 준비되면 기존 Pod를 차례로 종료시킵니다.

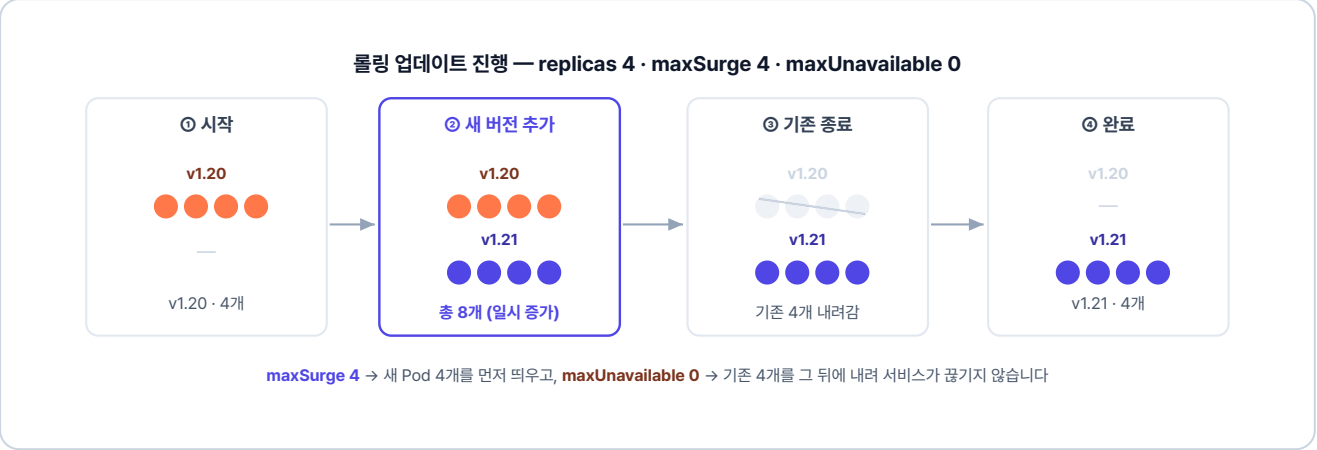


그림 4-23. 롤링 업데이트 진행 - 새 Pod 4개를 띄운 뒤 기존 Pod 4개를 차례로 내립니다

maxSurge를 줄이면 한 번에 띄울 새 Pod 수가 제한되어 천천히 교체되고, **maxUnavailable을 늘리면** 기존 Pod가 한 번에 더 많이 종료되는 것을 허용해 빠르게 교체할 수 있습니다. 이렇게 두 값을 조정해 원하는 배포 방식을 설정합니다.

다음 명령어로 이미지 버전을 교체해 보겠습니다.

터미널 **이미지 버전 교체 (롤링 업데이트 트리거)**

```
# 이미지를 nginx:1.21로 교체
kubectl set image deployment/nginx-replica nginx-container=nginx:1.21
```

실행 결과

```
$ kubectl set image deployment/nginx-replica nginx-container=nginx:1.21
deployment.apps/nginx-replica image updated
```

그림 4-24. 이미지 버전 업데이트 실행

`kubectl get` 명령어에 `-w` 옵션을 추가하면 Pod 상태 변화를 실시간으로 볼 수 있습니다.

터미널 **Pod 상태 실시간 감시**

```
kubectl get pod -w   # Pod 상태 실시간 감시
```

실행 결과

```
$ kubectl get pod -w
NAME READY STATUS RESTARTS AGE
nginx-replica-576bff5654-7gtrn 0/1 Pending 0 1s
nginx-replica-576bff5654-8bf7j 0/1 Pending 0 1s
nginx-replica-576bff5654-hkqwp 0/1 Pending 0 1s
nginx-replica-576bff5654-kcqh4 0/1 Pending 0 1s
nginx-replica-576bff5654-hkqwp 0/1 ContainerCreating 0 1s
nginx-replica-576bff5654-7gtrn 0/1 ContainerCreating 0 1s
nginx-replica-576bff5654-8bf7j 0/1 ContainerCreating 0 1s
nginx-replica-576bff5654-kcqh4 0/1 ContainerCreating 0 2s
nginx-replica-576bff5654-8bf7j 1/1 Running 0 6s
nginx-replica-576bff5654-kcqh4 1/1 Running 0 6s
nginx-replica-576bff5654-7gtrn 1/1 Running 0 7s
nginx-replica-576bff5654-hkqwp 1/1 Running 0 7s
nginx-replica-756b46b54c-l5l7f 1/1 Terminating 0 9m20s
nginx-replica-756b46b54c-t8fhl 1/1 Terminating 0 9m20s
nginx-replica-756b46b54c-vp5g5 1/1 Terminating 0 9m20s
nginx-replica-756b46b54c-vzv7n 1/1 Terminating 0 9m20s
nginx-replica-756b46b54c-vp5g5 0/1 Completed 0 9m23s
nginx-replica-756b46b54c-vzv7n 0/1 Completed 0 9m23s
nginx-replica-756b46b54c-t8fhl 0/1 Completed 0 9m23s
nginx-replica-756b46b54c-l5l7f 0/1 Completed 0 9m24s
```

그림 4-25. 롤링 업데이트 진행 화면

출력에서 새 Pod는 **Pending → ContainerCreating → Running** 순서로 실행되고, 기존 Pod는 **Running → Terminating → Completed** 순서로 종료됩니다.

업데이트가 끝난 뒤 Deployment의 상세 정보를 확인하면 이미지가 `nginx:1.21` 버전으로 변경된 것을 볼 수 있습니다.

터미널 **Deployment 상세 정보 조회**

```
kubectl describe deployment nginx-replica  # Deployment 상세 정보 조회
```

4.4.5 Rollback - 되돌리기

배포한 새 버전에서 뒤늦게 문제가 발견될 때가 있습니다. 그럴 때는 `kubectl rollout undo` 명령어로 이전 버전으로 되돌립니다.

터미널 **롤백**

```
kubectl rollout undo deployment/nginx-replica      # 이전 버전으로 롤백
```

실행결과

```
$ kubectl rollout undo deployment/nginx-replica
deployment.apps/nginx-replica rolled back
```

그림 4-26. Rollback 실행 결과

이렇게 쿠버네티스로 컨테이너를 운영하는 법을 익히고 나니, 문득 회의실에서의 그 질문이 떠올랐습니다.

'컨테이너가 죽거나 트래픽이 몰리면 어쩌나 막막했는데. 이제 쿠버네티스한테 맡기면 되겠다.'

다음 챕터에서는 Service와 Ingress로 외부와 연결되는 진입점을 만들어 보겠습니다.

이것만은 기억하자

- **Docker는 "지금 띄워라", Kubernetes는 "이 상태를 유지해라".** 선언형 관리의 핵심입니다. 원하는 상태를 YAML에 정의해 두면 쿠버네티스가 실제 상태를 계속 확인하며 그에 맞춥니다.
- **쿠버네티스는 컨트롤 플레인과 워커 노드로 이루어집니다.** 컨트롤 플레인이 워커 노드 위의 Pod를 관리합니다.
- **Pod는 쿠버네티스가 컨테이너를 다루는 최소 단위입니다.** 쿠버네티스는 컨테이너를 직접 다루지 않고 Pod 단위로 관리합니다.
- **운영에서는 Pod를 단독으로 만들지 않습니다.** 직접 만든 Pod는 장애가 나도 복구되지 않습니다. Deployment로 관리해야 자동 복구와 스케일링이 가능합니다.
- **롤링 업데이트로 무중단 배포를 합니다.** 새 Pod를 먼저 띄운 뒤 기존 Pod를 차례로 종료해, 서비스 중단 없이 버전을 올립니다.

CHAPTER

5

Kubernetes 네트워킹

외부에서 접속할 주소를 만들다

이번 챕터가 끝나면

- Pod IP가 바뀌어도 변하지 않는 **Service** 주소를 만들어 접속합니다.
- 서비스를 외부에 노출하는 여러 방식의 차이를 익힙니다.
- **Ingress**로 외부 도메인 한 곳에서 URL 경로별로 요청을 분기합니다.
- 브라우저에서 Pod까지 패킷이 이동하는 경로를 이해합니다.

챕터 5. Kubernetes 네트워킹

오픈이는 지금까지 학습한 내용을 바탕으로 테스트 서버를 Pod로 재구성했습니다. Pod가 종료되더라도 Deployment가 즉시 새 Pod를 실행해 주니 마음이 든든했습니다.

다음 날 아침, 동료가 찾아왔습니다.

동료 : "오픈 씨, 어제 테스트 서버에 올렸다는 서비스, 접속이 안 되는데 어떻게 들어가요?"

오픈이는 당황했습니다. 로그상으로는 정상 실행 중이었지만, 정작 브라우저로 접속해 본 적은 없었기 때문입니다. 서둘러 Pod 목록에서 IP를 확인해 브라우저에 입력해 보았지만, 페이지는 열리지 않았습니다.

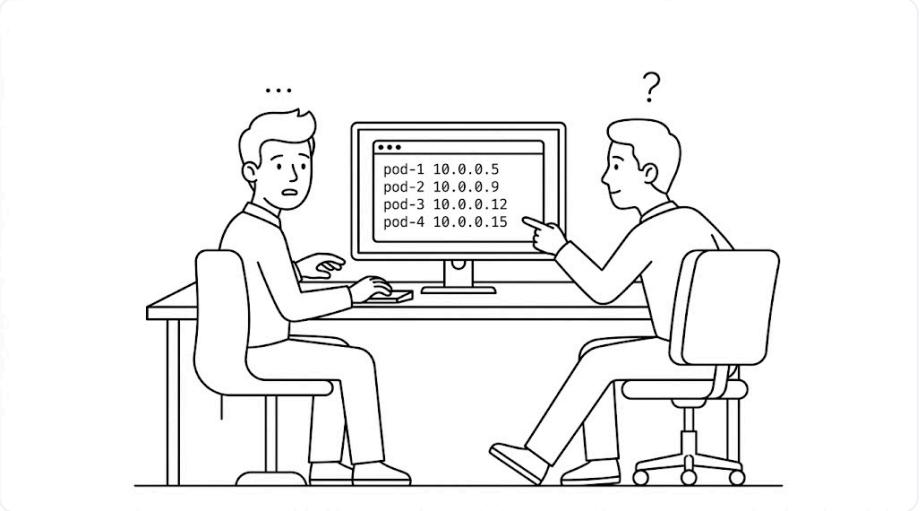


그림 5-1. Pod는 떠 있는데 접근할 주소가 없던 아침

게다가 Pod가 재생성될 때마다 IP가 바뀌니, IP로는 접근할 수도 없었습니다.

'쿠버네티스 환경에서는 외부에서 Pod로 어떻게 접속해야 하지?'

막막해진 오픈이가 선배에게 묻자, 선배가 화면을 보며 설명했습니다.

선배 : "외부 요청이 실제 Pod까지 도달하려면 두 가지가 필요해요. 들어온 요청을 도메인이나 경로에 따라 알맞은 곳으로 보내 주는 **Ingress**, 그리고 수시로 바뀌는 Pod를 묶어 고정된 진입점을 제공하는 **Service**죠. 이 두 가지를 한번 알아봐요."

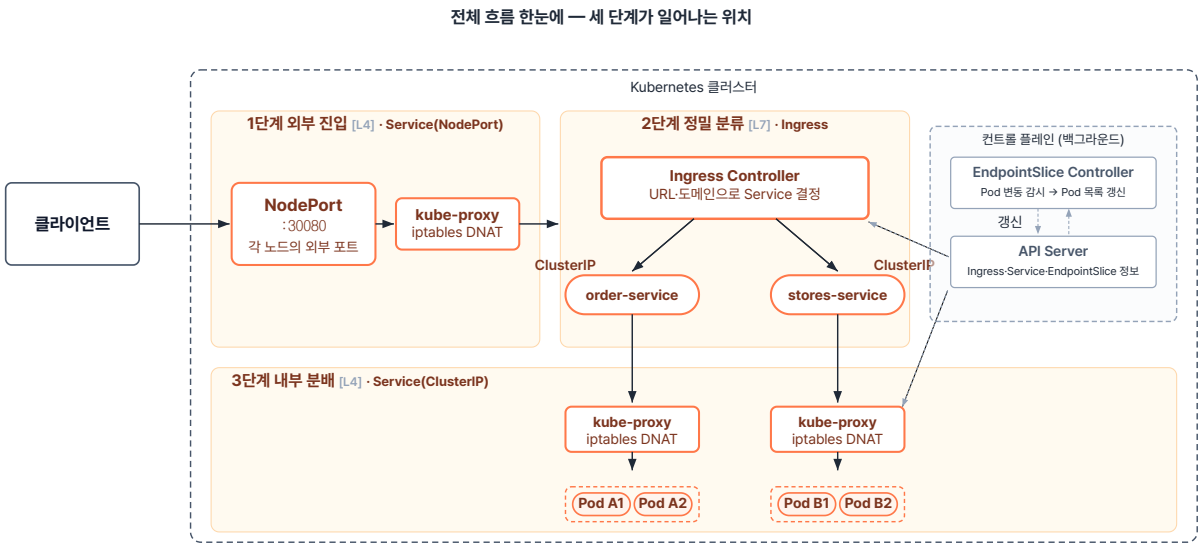


그림 5-2. 챕터 5 한눈에 보기 - 요청이 Pod까지 도달하는 전체 흐름

실습 준비. 예제 코드

이 책의 실습 코드는 GitHub 레포 하나에 챕터별 폴더로 담겨 있습니다. 아래 명령으로 레포를 한 번 git clone 합니다. 이후 챕터마다 해당 폴더로 이동해 실습합니다.

터미널 예제 코드 클론

```
git clone https://github.com/metacoding-10-linux-docker/start
```

이번 챕터는 `ex11` ~ `ex12` 폴더를 사용합니다.

완성된 코드는 아래 주소에서 확인하세요.

터미널 완성 코드

```
https://github.com/metacoding-10-linux-docker/final
```

5.1 Service - Pod의 고정 주소

5.1.1 Service가 필요한 이유

Pod의 IP는 고정된 값이 아닙니다. Pod는 종료되거나 새 버전으로 교체될 때마다 새로 만들어지고, 그때마다 IP가 바뀝니다. 게다가 같은 역할을 하는 Pod를 여러 개 띄워 두면, 요청을 그중 어디로 보낼지도 정해야 합니다.

Service는 이 문제를 해결합니다. Pod IP가 바뀌어도 변하지 않는 **고정 진입점**을 만들고, 들어온 요청을 **연결된 여러 Pod**에 **분배**하는 리소스입니다.

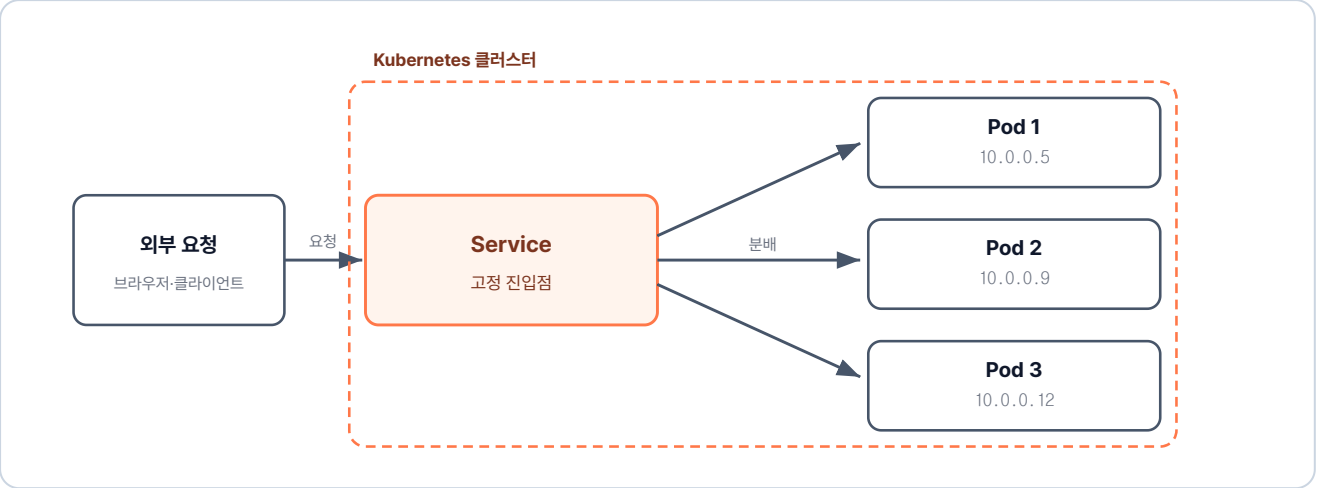


그림 5-3. Service는 Pod IP가 바뀌어도 변하지 않는 고정 주소를 제공

5.1.2 Service 생성

이제 Pod에 연결할 Service를 직접 만들어 보겠습니다.

실습 코드는 `ex11` 폴더에 있습니다. 이전 챕터에서 사용한 `deploy-ex02.yml`을 사용합니다.

```
ex11/
├── deploy-ex02.yml    # nginx Pod 4개를 유지하는 Deployment 정의
├── service-ex01.yml   # Pod 앞에 붙이는 NodePort Service 정의
└── README.md          # 실습 안내
```

다음은 Service를 정의한 YAML입니다.

실습 1 **ex11/service-ex01.yml. NodePort Service 정의**

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort        # 노드 IP와 포트로 외부 접근이 가능한 타입
  selector:
    app: nginx          # 이 라벨을 가진 Pod들을 뒤에 연결
  ports:
    - port: 80          # 클러스터 내부에서 Service로 진입하는 포트
      targetPort: 80    # Service가 Pod로 요청을 전달하는 포트
      nodePort: 30080   # 외부에서 노드로 진입 시 사용하는 포트 (기본 허용 범위 30000~32767)
```

Service는 **Deployment**와 마찬가지로 **selector**에 지정한 라벨과 일치하는 Pod를 골라 연결합니다.

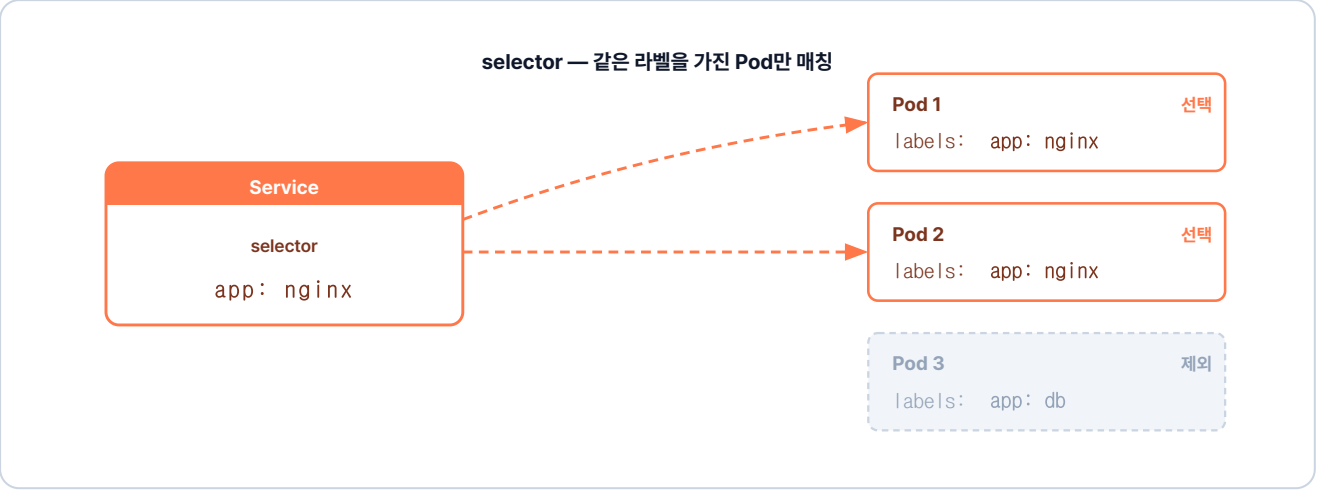


그림 5-4. selector가 지정한 라벨(app: nginx)을 가진 Pod만 매칭하고 다른 라벨은 제외

5.1.3 Service 타입과 접근 범위

앞서 작성한 Service YAML에는 `type: NodePort` 설정이 있습니다. Service는 접근 범위에 따라 타입이 **ClusterIP**, **NodePort**, **LoadBalancer**로 나뉩니다.

ClusterIP

ClusterIP는 사내에서만 연결되는 **내선 번호**입니다. **클러스터 안의 Pod끼리는 서로 연결되지만, 외부에서는 접근할 수 없습니다.** DB처럼 외부에 노출할 필요가 없는 구성에 적합하며, 서비스 타입의 기본값입니다.

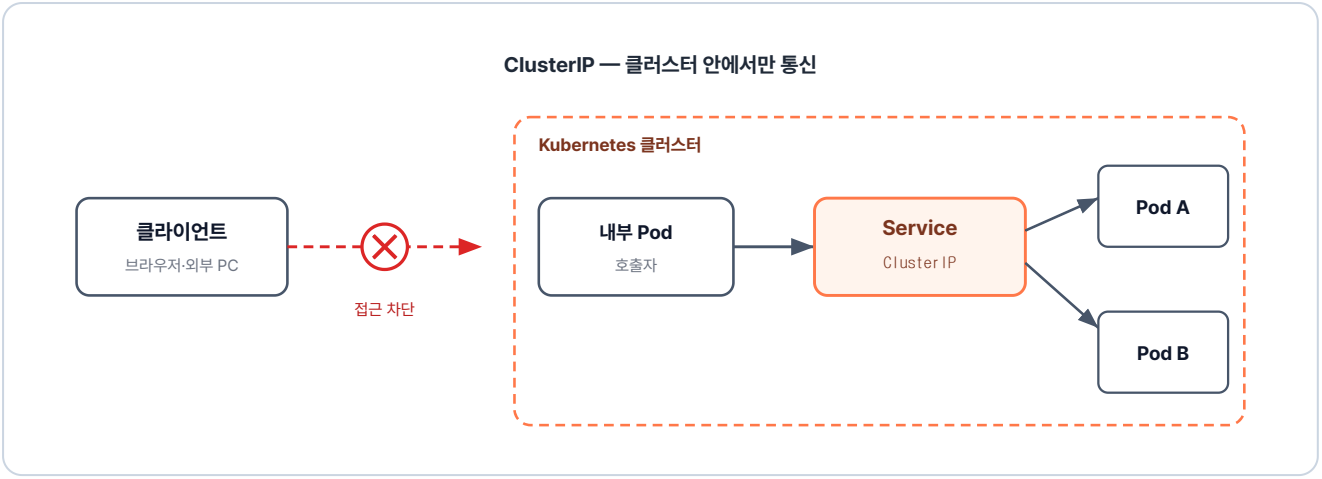


그림 5-5. ClusterIP는 외부 요청은 닿지 못하고 내부 Pod끼리만 통신

NodePort

NodePort는 외부에서도 연결할 수 있는 **직통 번호**입니다. Service를 NodePort로 지정하면, **노드(서버)에 외부와 통하는 포트가 열립니다.** 외부에서 그 노드의 IP와 포트로 접속하면, 요청이 클러스터 안의 Service를 거쳐 Pod로 전달됩니다.

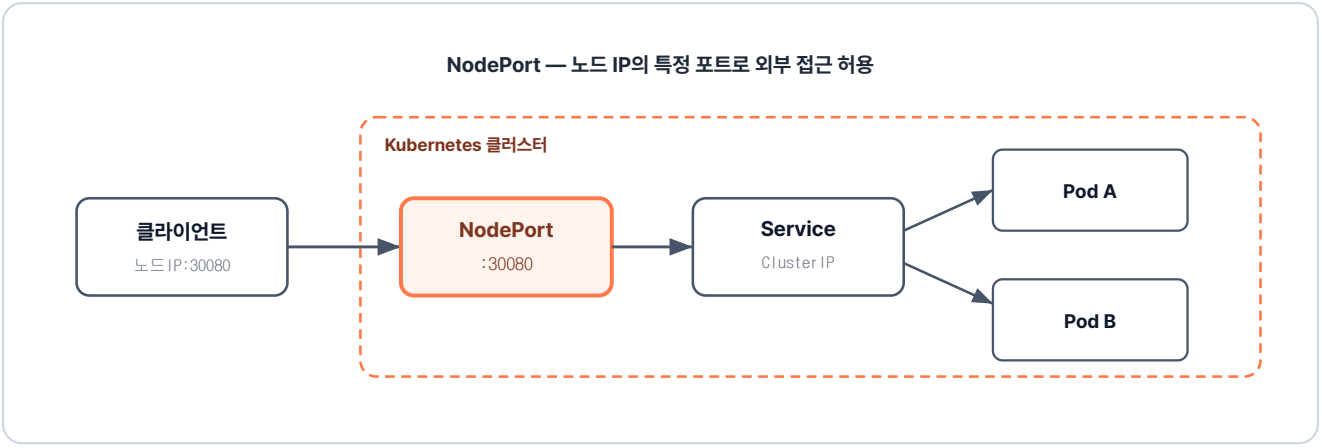


그림 5-6. NodePort는 노드의 특정 포트로 외부 접근을 허용

LoadBalancer

LoadBalancer는 외부 요청을 한곳에서 받아 연결하는 **대표 번호**입니다. Service를 LoadBalancer로 지정하면, AWS·GCP 같은 클라우드 플랫폼이 **공인 IP**를 발급합니다. **그 IP로 들어온 모든 요청을 받아, 뒤에 있는 여러 노드에 분산합니다.** 실제 운영에서 가장 흔히 사용합니다.

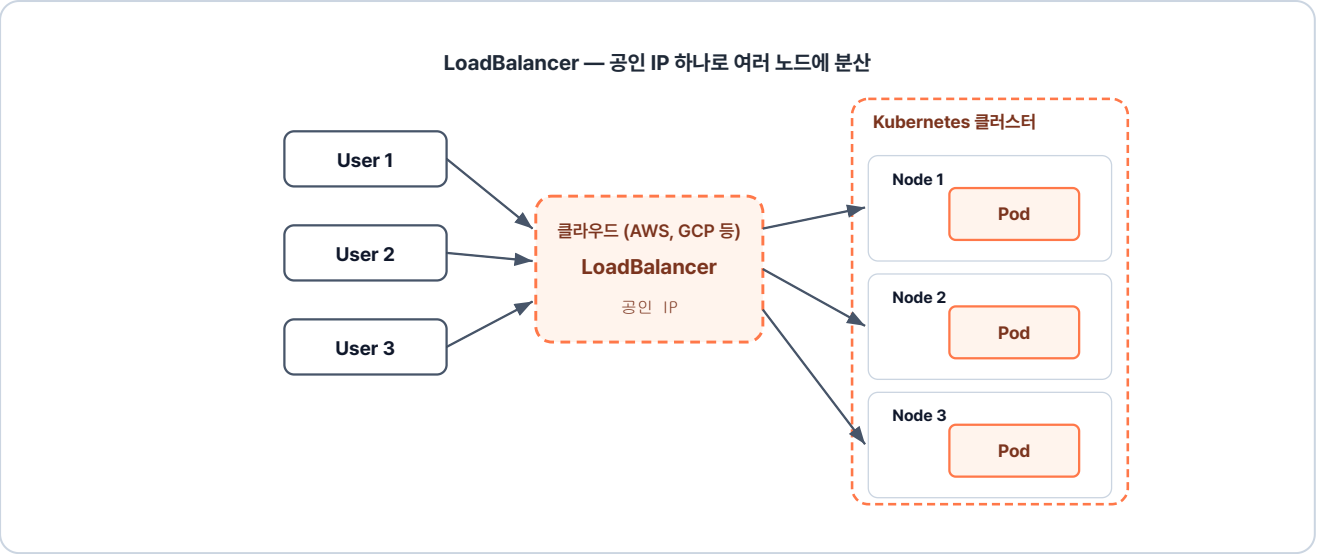


그림 5-7. LoadBalancer는 클라우드가 공인 IP를 발급해 여러 노드에 분산

LoadBalancer 타입은 ClusterIP와 NodePort를 함께 포함합니다. 그래서 Service 하나로 내부 통신과 외부 접근이 모두 처리됩니다.

5.1.4 포트 흐름

Service YAML의 포트 설정에는 `port`, `targetPort`, `nodePort` 세 종류가 있습니다. 각 포트는 외부에서 들어온 요청을 Pod까지 단계별로 전달합니다.

외부 사용자가 노드의 `nodePort`로 접속하면, 요청은 `port`로 열려 있는 Service에 도달하고, Service는 이를 Pod의 `targetPort`로 전달합니다.

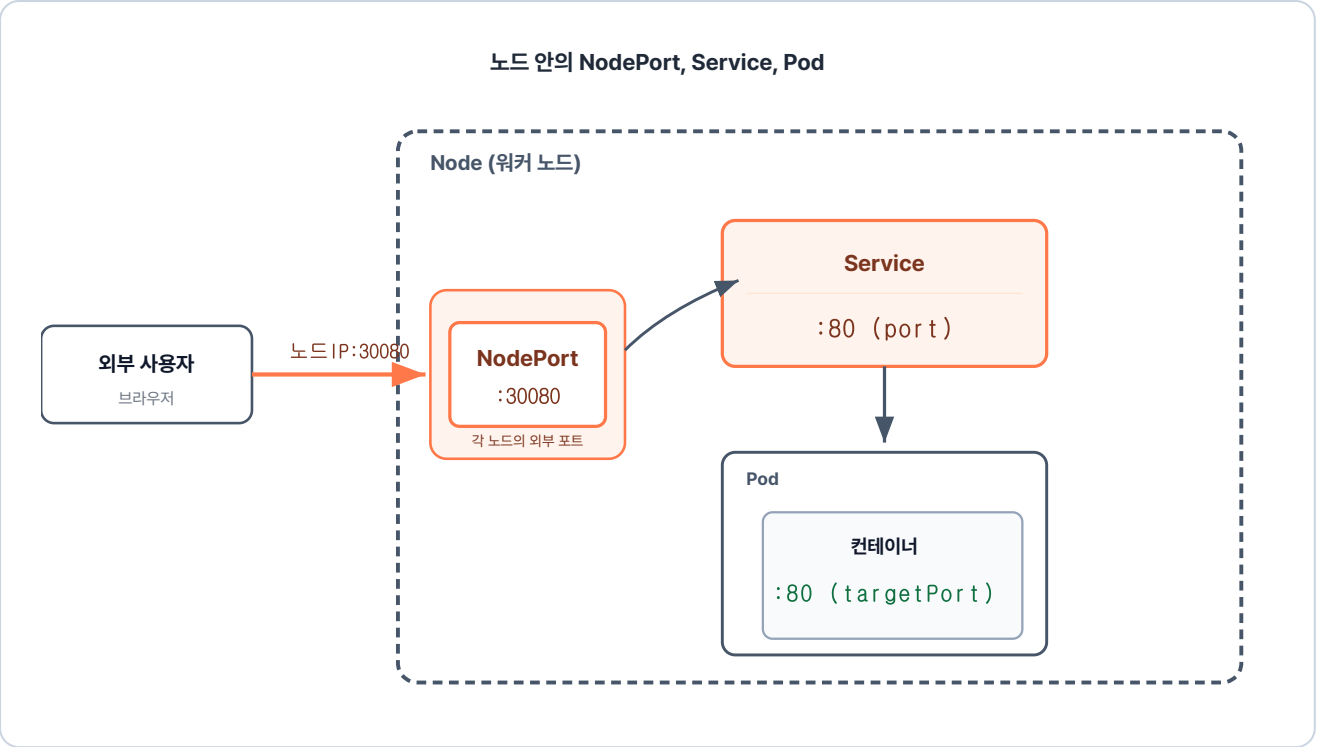


그림 5-8. NodePort에서 Service를 거쳐 Pod 컨테이너까지 이어지는 포트 흐름

포트 종류	위치	역할	생략 시
`nodePort`	워커 노드	외부 사용자가 노드 IP로 클러스터 내부에 접근할 때 열리는 포트	30000~32767 범위에서 자동 할당. 같은 범위로 직접 지정도 가능
`port`	Service	클러스터 내부의 다른 Pod가 이 Service를 호출할 때 쓰는 포트	생략 불가 (필수)
`targetPort`	Pod(컨테이너)	Service가 받은 트래픽을 전달할 컨테이너의 수신 포트 (컨테이너 포트와 일치)	생략 시 `port`와 같은 값

5.1.5 외부에서 Service 접속해 보기

이제 실습을 해 보겠습니다. 아래 명령어를 실행해 클러스터에 적용한 뒤, 외부에서 실제로 접속되는지 확인합니다.

터미널 **Pod와 Service 생성**

```
kubectl apply -f ex11/deploy-ex02.yml    # Pod 4개 생성
kubectl apply -f ex11/service-ex01.yml   # Service YAML 적용
```

실행결과

```
$ kubectl apply -f ex11/deploy-ex02.yml
deployment.apps/nginx-replica created
$ kubectl apply -f ex11/service-ex01.yml
service/nginx-service created
```

그림 5-9. Pod와 Service 생성 결과

브라우저를 열고 NodePort로 설정한 30080 포트(`localhost:30080`)로 접속합니다. 그런데 연결할 수 없다는 화면이 뜹니다.

'분명 30080 포트를 열었는데 왜 안 들어가지.'

> **NodePort로 접속이 안 되는 이유**
> 미니큐브는 내 컴퓨터 안에 작은 가상 환경을 만들고, 그 안에서 클러스터를 실행합니다. 이 가상 환경은 별개의 PC처럼 동작해서, 내 컴퓨터와 다른 네트워크를 가집니다. 그래서 NodePort를 열어도 `localhost` 주소로는 접근할 수 없습니다.

이 문제를 해결하기 위해, 미니큐브에는 내 컴퓨터와 클러스터를 연결하는 별도의 명령어가 있습니다.

방법	명령어	설명
URL 생성	`minikube service <서비스이름> --url`	**Service 한 개에 접근할 임시 URL 생성**. NodePort 접근에 사용
터널 개방	`minikube tunnel`	**LoadBalancer·Ingress에 외부 IP 부여**. LoadBalancer·Ingress 접근에 사용

NodePort에 접근할 때는 임시 URL을 만드는 `minikube service`를 사용합니다. 명령을 실행하면 내 컴퓨터에서 접속할 수 있는 URL이 출력됩니다.

터미널 **Service 접근 URL 생성**

```
minikube service nginx-service --url    # Service 접근 URL 생성
```

실행결과

```
$ minikube service nginx-service --url
http://127.0.0.1:10345
! windows 에서 Docker 드라이버를 사용하고 있기 때문에, 터미널을 열어야 실행할 수 있습니다.
```

그림 5-10. minikube service URL 생성 결과

URL이 출력되고 커서는 그대로 멈춰 있습니다. 이 명령은 연결을 유지하는 동안 터미널에서 계속 실행됩니다. 출력된 주소를 브라우저에 붙여 넣으면 NGINX 환영 페이지가 나타납니다.



그림 5-11. 브라우저에서 nginx 접속 확인

다음으로 Pod가 사라지고 IP가 바뀌어도 Service가 새 Pod로 자동 연결되는지 직접 확인합니다. Ctrl+C로 터미널을 빠져나온 후 Pod를 전부 지웁니다.

터미널 **Pod 재생성 후 같은 주소로 재접속**

```
kubectl delete pod --all                # 모든 Pod 삭제 (Deployment가 자동 재생성)
minikube service nginx-service --url    # 다시 접속 URL 확인
```

실행결과

```
$ kubectl delete pod --all
pod "nginx-replica-756b46b54c-7qztn" deleted
pod "nginx-replica-756b46b54c-cb592" deleted
pod "nginx-replica-756b46b54c-ff5dt" deleted
pod "nginx-replica-756b46b54c-hpnzm" deleted
$ minikube service nginx-service --url
http://127.0.0.1:3082
! windows 에서 Docker 드라이버를 사용하고 있기 때문에, 터미널을 열어야 실행할 수 있습니다.
```

그림 5-12. Pod 삭제 후 Service 접속 확인

새 URL로 접속해도 NGINX 페이지가 그대로 나옵니다.

'Pod 앞에 Service를 두니까, Service로 요청만 보내면 Pod까지 전달되는구나.'

5.2 Ingress - 도메인 라우팅

Service는 Pod 그룹마다 고정 주소를 제공합니다. 하지만 Service는 IP와 포트만으로 요청을 전달할 뿐, 같은 도메인 안에서 **URL 경로에 따라 요청을 나눠 보내지는 못합니다**. 이 한계를 해결하는 리소스가 Ingress입니다.

5.2.1 Ingress의 역할

Ingress는 외부 도메인 하나로 들어온 요청을 **URL 경로에 따라 알맞은 Service로 보냅니다**. 예를 들어 `/order` 경로와 `/stores` 경로를 각각의 Service로 보내도록 규칙을 적어 두면, Ingress가 요청 경로를 읽어 알맞은 Service로 전달합니다.

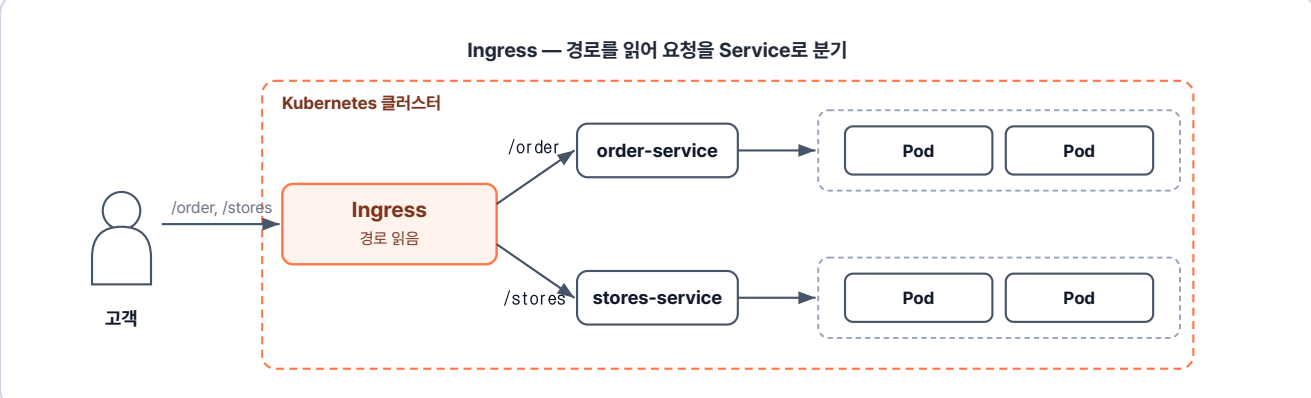


그림 5-13. Ingress가 도메인과 경로를 읽어 요청을 적절한 Service로 연결하는 구조

5.2.2 Ingress 컨트롤러

Ingress는 **Ingress 리소스(YAML)** 와 **Ingress 컨트롤러** 두 가지로 구성됩니다.

Ingress 리소스에는 어떤 경로를 어느 Service로 보낼지 **라우팅 규칙을 적어 둡니다**. 그리고 외부 요청이 들어오면 Ingress 컨트롤러가 **이 규칙대로 알맞은 Service로 전달합니다**.

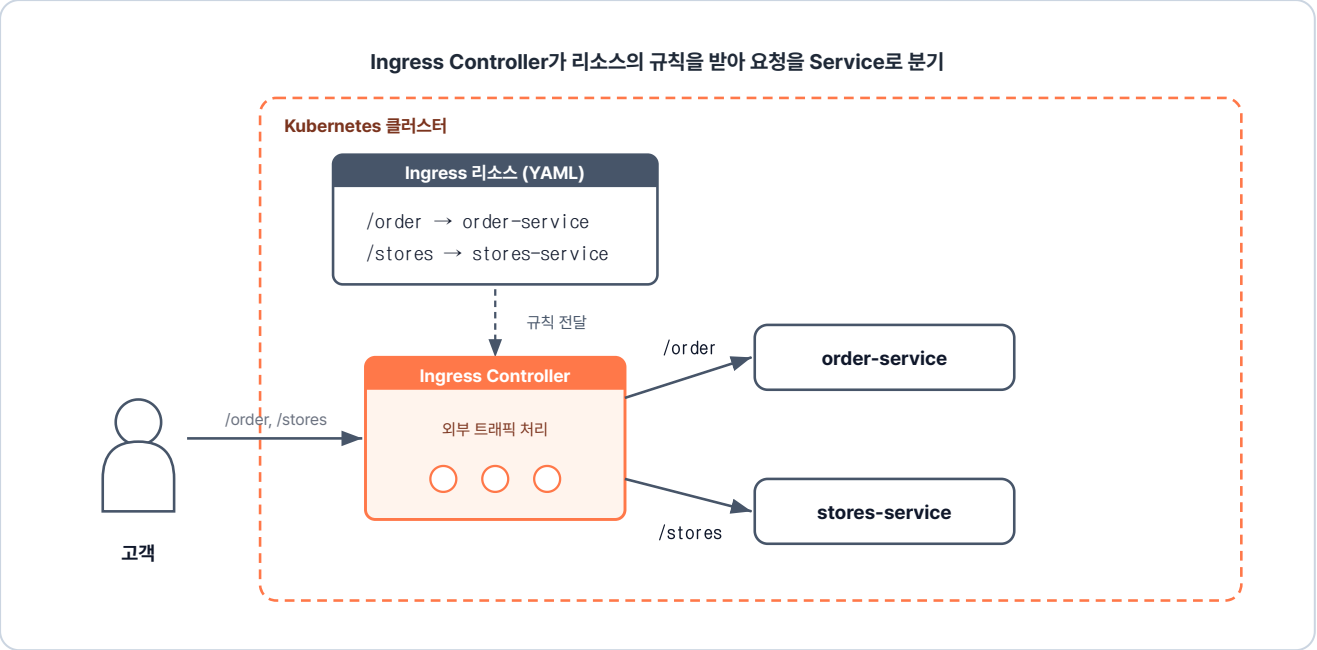


그림 5-14. Ingress 리소스(YAML)와 Ingress 컨트롤러

5.2.3 Ingress 적용하기

실습을 위해 미니큐브에 인그레스를 활성화해 보겠습니다. 미니큐브는 Ingress 컨트롤러를 애드온 형태로 한 번에 배포할 수 있는 명령어를 제공합니다.

터미널 **인그레스 애드온 활성화**

```
minikube addons enable ingress          # 인그레스 애드온 활성화
```

실행 결과

```
$ minikube addons enable ingress
* ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at:
https://github.com/kubernetes/minikube/blob/master/OWNERS
* After the addon is enabled, please run "minikube tunnel" and your ingress resources would
be available at "127.0.0.1"
   - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
   - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
   - Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
* Verifying ingress addon...
* The 'ingress' addon is enabled
```

그림 5-15. minikube에서 ingress 애드온 활성화 과정

Ingress 컨트롤러가 실제로 떠 있는지 확인합니다.

터미널 **Ingress 컨트롤러 확인**

```
kubectl get pods -n ingress-nginx        # Ingress 컨트롤러 확인
```

실행 결과

```
$ kubectl get pods -n ingress-nginx
NAME READY STATUS RESTARTS AGE
ingress-nginx-admission-create-zg9hh 0/1 Completed 0 40s
ingress-nginx-admission-patch-bvxlw 0/1 Completed 1 40s
ingress-nginx-controller-9cc49f96f-qrpm7 1/1 Running 0 35s
```

그림 5-16. Ingress 컨트롤러가 Running 상태임을 확인

실제 환경에서는 어떻게 띄울까

실제 서비스 환경에서는 Ingress 컨트롤러를 클러스터에 직접 설치해야 합니다. 그런 다음 외부 사용자가 접속할 수 있도록 클라우드의 공인 IP와 연결하는 작업이 필요합니다. 앞서 사용한 미니큐브 애드온은 이 두 단계를 명령어 하나로 처리해 줍니다.

`ex12` 폴더에는 Ingress 규칙과 두 Service, 그리고 각 Service가 가리키는 Deployment가 함께 들어 있습니다.

```
ex12/
├── ingress-ex01.yml   # /order·/stores 경로별 라우팅 규칙
├── order-deploy.yml    # 주문 응답 Pod Deployment
├── order-service.yml   # 주문 Pod 앞 ClusterIP Service
├── stores-deploy.yml   # 매장 응답 Pod Deployment
├── stores-service.yml  # 매장 Pod 앞 ClusterIP Service
└── README.md           # 실습 안내
```

이제 외부 요청을 주문 Service와 매장 Service로 나눠 보낼 **Ingress 규칙**을 만들 차례입니다. 두 Service는 **ClusterIP** 타입이라 클러스터 바깥에서는 직접 접근할 수 없습니다.

다음은 `/order`와 `/stores` 두 경로를 각각의 Service로 분기하는 Ingress YAML입니다.

실습 2 **ex12/ingress-ex01.yml. 경로별 라우팅 규칙**

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ex12-ingress
spec:
  ingressClassName: nginx           # 어느 Ingress 컨트롤러가 이 규칙을 집행할지 지정
  rules:
    - http:
        paths:
          - path: /order            # 주문 경로 → order-service
            pathType: Prefix        # /order로 시작하는 하위 경로까지 매칭
            backend:
              service:
                name: order-service
                port:
                  number: 5678
          - path: /stores           # 매장 경로 → stores-service
            pathType: Prefix        # /stores로 시작하는 하위 경로까지 매칭
            backend:
              service:
                name: stores-service
                port:
                  number: 5678
```

이제 `ex12` 폴더의 모든 파일을 한 번에 적용합니다.

터미널 **Ingress 규칙과 두 Service 일괄 적용**

```
kubectl apply -f ex12/                    # ex12 폴더 내 모든 yaml 일괄 적용
```

실행결과

```
$ kubectl apply -f ex12/
ingress.networking.k8s.io/ex12-ingress created
deployment.apps/order-deploy created
service/order-service created
deployment.apps/stores-deploy created
service/stores-service created
```

그림 5-17. Ingress 리소스 등록 확인

5.2.4 외부에서 Ingress 접속해 보기

클러스터를 호스트와 연결하기 위해, 외부 IP를 부여하는 `minikube tunnel`을 별도 터미널에서 실행합니다.

터미널 **외부 터널 개방**

```
minikube tunnel                           # 별도 터미널에서 실행
```

실행결과

```
$ minikube tunnel
* Tunnel successfully started
* NOTE: Please do not close this terminal as this process must stay alive for the tunnel to
be accessible ...
* Starting tunnel for service ex12-ingress.
```

그림 5-18. minikube tunnel 실행 화면

이제 브라우저에서 두 경로를 차례로 접속해 보겠습니다.

먼저 `http://localhost/order`를 열면 "주문 접수 완료"가 나옵니다.

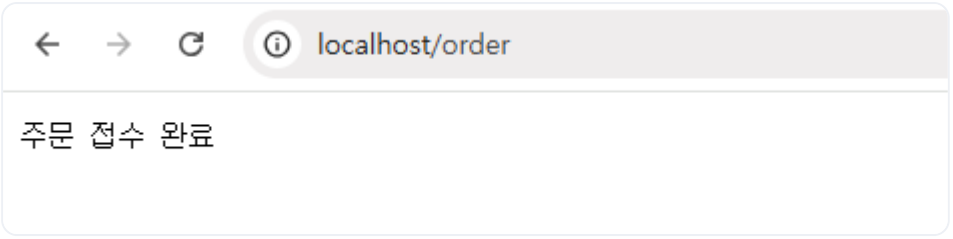


그림 5-19. /order 접속 결과

이어서 `http://localhost/stores`를 열면 "매장 선택"이 나옵니다.

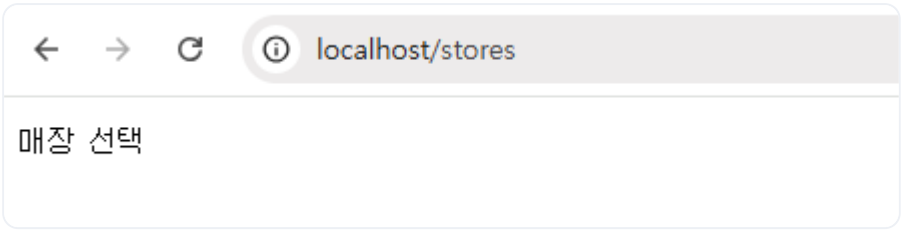


그림 5-20. /stores 접속 결과

도메인은 하나인데 경로마다 다른 응답이 돌아왔습니다. Ingress가 경로를 보고 알맞은 **Service**로 요청을 넘기고, Service가 다시 그 뒤의 **Pod**로 요청을 전달해 응답이 만들어진 결과입니다.

그럼 한 요청 안에서 Ingress와 Service는 각각 어떤 정보를 보고 처리하는 걸까?

5.2.5 두 리소스의 계층 분담

두 리소스가 같은 요청에서 보는 정보가 다른 모습은 우편 봉투의 송장과 닮았습니다.

우편 송장에는 두 가지 정보가 함께 적혀 있습니다. 물류 센터가 어느 지역으로 보낼지 정할 때 찍는 **송장 바코드**, 그리고 그 지역 우체국에서 **집배원**이 직접 눈으로 확인하는 **받는 곳 주소**입니다.



그림 5-21. 우편 위 송장에 바코드와 받는 곳 주소가 함께 있고, 물류 센터는 바코드까지, 집배원은 받는 곳 주소만 읽습니다

이 우편 송장의 구조를 네트워크 **패킷**에 그대로 옮겨 보겠습니다. 패킷은 **클라이언트와 서버가 네트워크로 데이터를 주고받는 기본 단위**입니다.

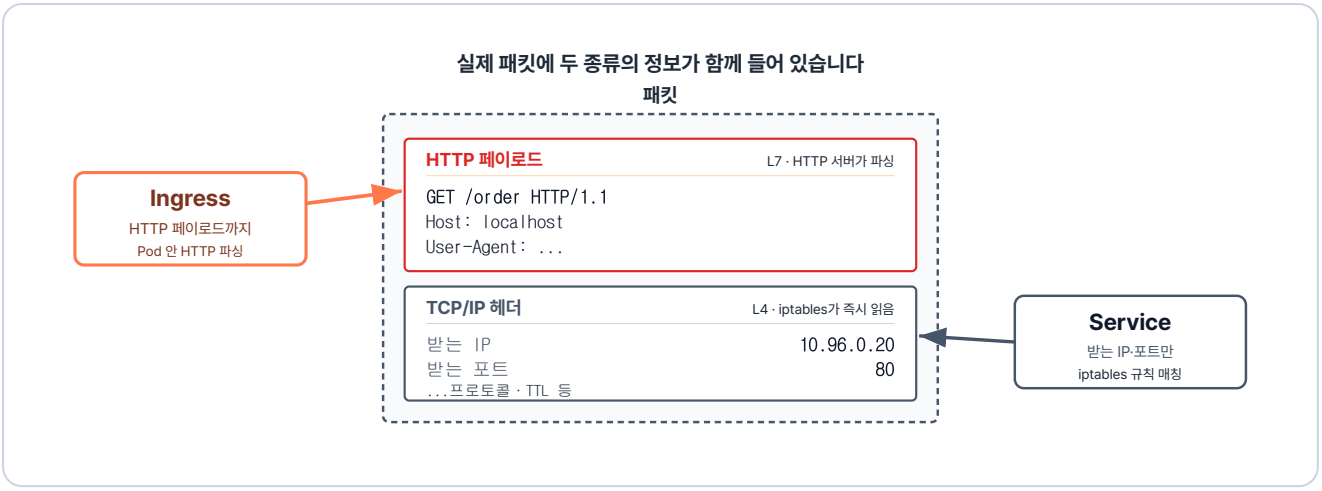


그림 5-22. 실제 패킷에 HTTP 페이로드와 TCP/IP 헤더가 함께 들어 있고, Ingress는 페이로드까지, Service는 헤더만 읽습니다

패킷 안에는 우편 송장처럼 역할이 다른 여러 종류의 데이터가 섞여 있습니다. 이 정보들은 각자의 역할에 따라 **전송 계층**(Layer 4)과 **응용 계층**(Layer 7)에서 각각 처리됩니다.

구분	전송 계층 (L4)	응용 계층 (L7)
역할	데이터를 어느 컴퓨터의 어느 프로세스로 보낼지 안내	패킷 내부의 메시지·콘텐츠를 해석
보는 정보	패킷 겉면(헤더)의 받는 IP·포트	패킷 내부의 URL·도메인
담당 리소스	**Service**	**Ingress**
동작	지정된 IP·포트로 들어온 요청을 백엔드 Pod로 분산·전달	세부 경로(`/order`, `/stores`)에 따라 알맞은 Service로 전달

5.3 브라우저에서 Pod까지의 경로

이전 챕터에서는 개발자의 명령으로 Pod를 띄우고 관리하는 흐름을 다뤘습니다. 이번에는 사용자가 브라우저에서 요청했을 때 Pod에 도달하는 흐름을, 우체국에 우편을 보내는 예시를 통해 단계별로 알아보겠습니다.

5.3.1 NodePort로 클러스터에 들어오기

먼저 발신자가 **중앙 우체국** 창구에 우편을 맡깁니다. 우편을 받은 **창구 직원**은 다음 거점인 **물류 센터**로 보냅니다.

브라우저가 보낸 요청도 비슷하게 흘러갑니다. 요청이 노드에 열린 **NodePort**로 클러스터에 들어오면, 같은 노드의 **kube-proxy**가 패킷의 목적지를 다음 거점인 **Ingress 컨트롤러**의 IP로 바꿉니다. 그러면 패킷은 Ingress 컨트롤러로 전달됩니다.

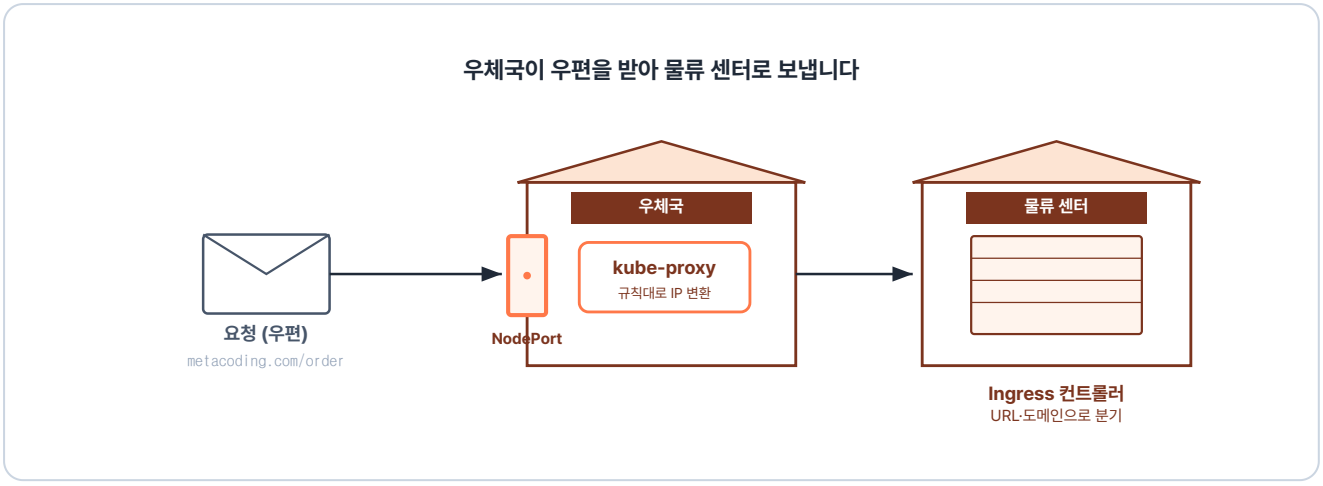


그림 5-23. 요청이 NodePort로 들어와 Ingress 컨트롤러로 향합니다

컴포넌트	이 단계에서 하는 일
NodePort	노드에 포트를 열어 외부 요청을 클러스터 안으로 받는 입구
kube-proxy	패킷의 목적지를 노드 IP에서 Ingress 컨트롤러의 IP로 바꿈

5.3.2 Ingress가 URL·도메인으로 Service 분기하기

우편이 **물류 센터**에 도착합니다. 물류 센터에서는 바코드를 스캐너로 찍어 등록된 **분류 규칙**에 따라 어느 지역 우체국으로 갈지 정합니다.

Ingress 컨트롤러도 **Ingress 리소스**에 적힌 규칙으로 요청을 보낼 곳을 정합니다. 요청의 **URL 경로와 도메인**을 그 규칙과 비교해 맞는 **Service**를 고릅니다.

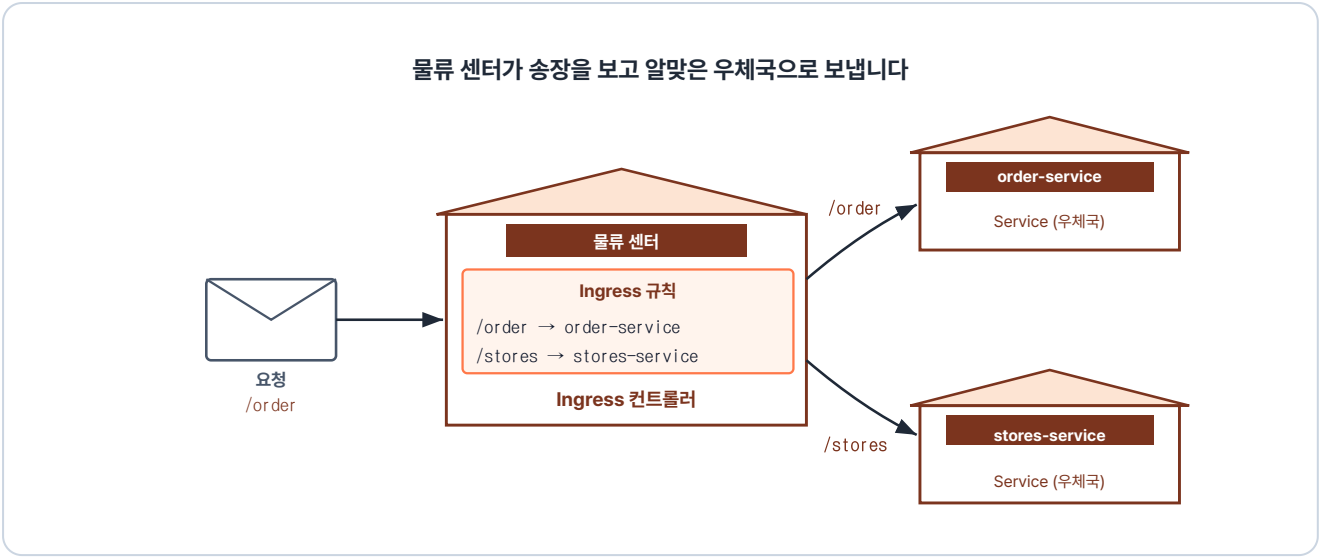


그림 5-24. Ingress 컨트롤러가 경로에 따라 알맞은 Service로 보냅니다

컴포넌트	이 단계에서 하는 일
Ingress 컨트롤러	요청을 받아 URL 경로·도메인에 맞는 Service로 전달
Ingress 규칙	어떤 경로·도메인을 어느 Service로 보낼지 적어 둔 설정

5.3.3 ClusterIP가 Pod로 전달하기

물류 센터에서 우체국으로 도착한 우편은 이제 배송이 시작됩니다. **집배원**은 **배송 목록**에 적힌 **우편함**에 우편을 넣습니다.

마지막 단계는 **kube-proxy**가 담당합니다. kube-proxy는 도착한 패킷의 IP를 **EndpointSlice**에서 살아있는 **Pod** 중 하나의 IP로 바꿉니다. 그러면 패킷은 그 Pod로 전달됩니다.

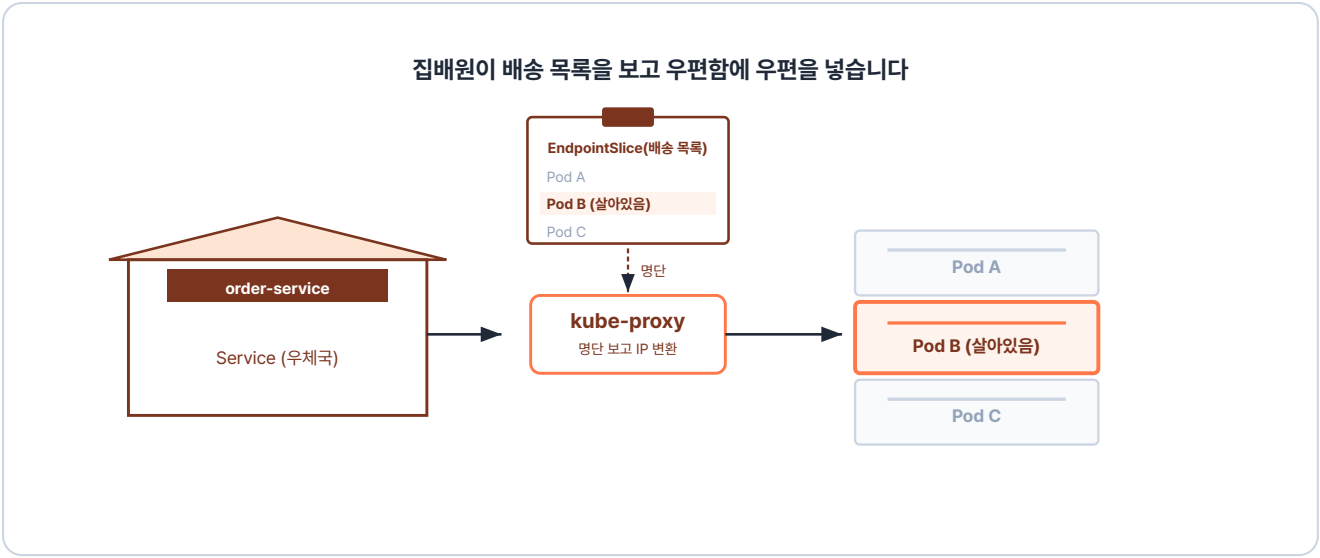


그림 5-25. kube-proxy가 살아있는 Pod로 목적지를 바꿉니다

컴포넌트	이 단계에서 하는 일
Service (ClusterIP)	여러 Pod를 대표하는 고정 IP로, 요청이 이 주소로 들어옴
EndpointSlice	Service에 연결된, 지금 살아있는 Pod들의 IP 목록
kube-proxy	요청의 목적지를 ClusterIP에서 살아있는 Pod의 IP로 바꿈

Ingress 컨트롤러와 kube-proxy가 설정을 받는 방법

클러스터에 등록된 모든 설정은 **API Server**를 거쳐 상태 저장소인 **etcd**에 보관됩니다. 각 컴포넌트는 API Server로부터 자신에게 필요한 설정을 전달받아 적용합니다. 이때 Ingress 컨트롤러는 YAML로 작성된 **Ingress 리소스**를 받아 라우팅 규칙으로 사용하고, kube-proxy는 살아있는 Pod 목록인 **EndpointSlice**를 받아 IP 변환에 반영합니다.

이번 챕터에서는 외부에서 들어온 요청이 클러스터 안의 Pod까지 도달하는 흐름을 살펴봤습니다. 하나의 요청이 들어오면 Ingress는 도메인과 URL 경로를, Service는 IP와 포트를 확인해 목적지를 결정합니다.

'이제 요청이 클러스터 안에서 Pod까지 닿는 구조를 알았으니, 내 프로젝트에도 직접 적용해 볼 수 있겠다.'

다음 챕터에서는 실제 운영 환경에 필요한 환경 변수와 데이터 저장소를 다뤄 보겠습니다.

이것만은 기억하자

- **Service는 Pod의 고정 진입점입니다.** Pod는 소모품이라 IP가 수시로 바뀌지만, Service는 변하지 않는 주소를 제공하며 트래픽을 분산합니다.
- **Service는 접근 범위에 따라 세 종류로 나뉩니다. ClusterIP**는 클러스터 안에서만 통신합니다. **NodePort**는 노드의 IP와 포트로 외부에 열립니다. **LoadBalancer**는 클라우드가 발급한 공인 IP로 외부에 열립니다.
- **Ingress는 외부 요청을 URL 경로로 분기합니다.** 숫자(IP·포트)만 보는 Service와 달리, 도메인과 URL 경로를 읽어 알맞은 Service로 연결합니다. 라우팅 규칙을 정의하는 **리소스(YAML)** 와 실제 요청을 처리하는 **Ingress 컨트롤러(Pod)** 가 함께 동작합니다.

CHAPTER

6

Kubernetes 운영하기

설정과 데이터를 따로 관리하다

이번 챕터가 끝나면

- 설정값과 비밀번호를 이미지에 넣지 않고 따로 관리합니다.
- 데이터를 Pod 외부의 저장 공간에 두어, Pod가 삭제돼도 사라지지 않게 합니다.
- 앞서 만든 프론트엔드·백엔드·DB를 쿠버네티스 위에 한 번에 배포합니다.

챕터 6. Kubernetes 운영하기

며칠 뒤, 오픈이는 진행 중인 프로젝트를 배포하기 전 마지막으로 한 번 더 코드 리뷰를 하고 있었습니다. 그러던 중 Dockerfile의 환경 변수 코드가 눈에 띄었습니다.

```
ENV MYSQL_PASSWORD=metacoding1234
```

데이터베이스 비밀번호가 코드에 그대로 노출되어 있었습니다.

'잠깐. 이대로 빌드하면 비밀번호가 이미지에 그대로 남는데.'



그림 6-1. Dockerfile에 적힌 비밀번호가 이미지에 그대로 담겨 배포된다

오픈이가 옆자리 선배에게 묻자, 선배가 의자를 돌려 오픈이를 봤습니다.

선배 : "테스트 환경에서는 괜찮지만, 실제 운영 환경에 배포할 때는 두 가지를 주의해야 해요. 먼저 비밀번호 같은 설정값은 자주 바뀔 수 있으니, Dockerfile에 직접 적지 말고 **이미지 밖으로 분리**해서 따로 관리해야 해요.
그리고 Pod는 언제든 종료될 수 있어서, Pod 안에 데이터를 저장하면 안 돼요. 보관해야 할 데이터는 **Pod 외부의 저장소**에 따로 저장해야 해요. 설정과 데이터 모두 컨테이너 밖으로 안전하게 빼 두는 게 핵심이죠."

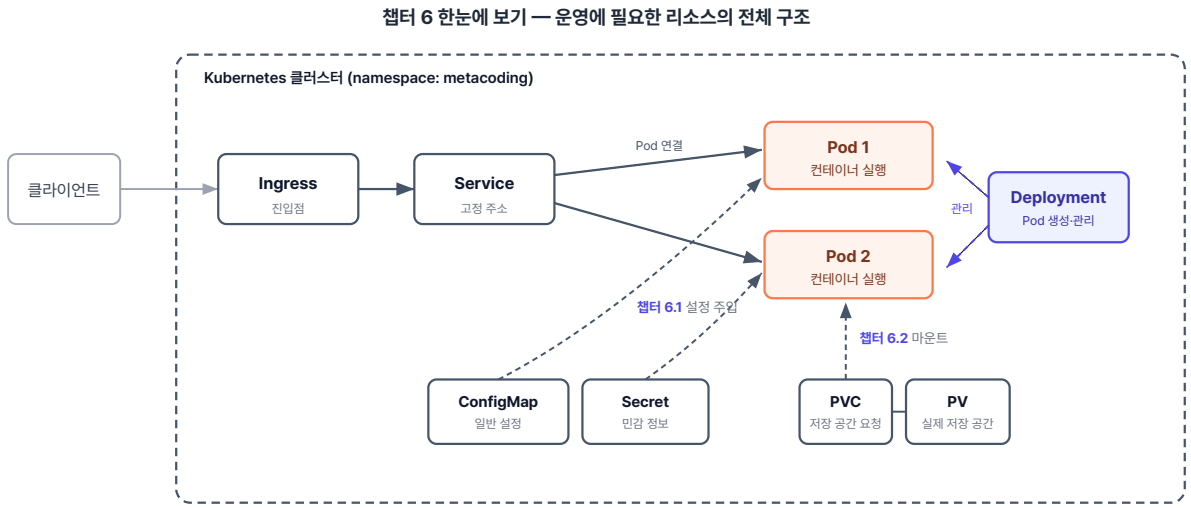


그림 6-2. 챕터 6 한눈에 보기 - 운영에 필요한 리소스의 전체 구조

실습 준비. 예제 코드

이 책의 실습 코드는 GitHub 레포 하나에 챕터별 폴더로 담겨 있습니다. 아래 명령으로 레포를 한 번 git clone 합니다. 이후 챕터마다 해당 폴더로 이동해 실습합니다.

터미널 **예제 코드 클론**

```
git clone https://github.com/metacoding-10-linux-docker/start
```

이번 챕터는 `ex13` ~ `ex15` 폴더를 사용합니다.

완성된 코드는 아래 주소에서 확인하세요.

터미널 **완성 코드**

```
https://github.com/metacoding-10-linux-docker/final
```

6.1 ConfigMap·Secret - 설정과 비밀번호를 이미지 외부로

컨테이너 이미지에 데이터베이스 주소나 비밀번호를 직접 적어 두면, 설정값이 바뀔 때마다 이미지를 새로 빌드해야 합니다. 운영 환경에서 설정 하나를 바꾸기 위해 매번 이미지를 다시 빌드하고 배포하는 것은 비효율적입니다.

그래서 쿠버네티스에는 컨테이너 이미지와 환경 변수를 분리하기 위한 **ConfigMap**과 **Secret**이라는 리소스가 있습니다. ConfigMap에는 일반 설정값을, Secret에는 민감한 정보를 저장합니다.

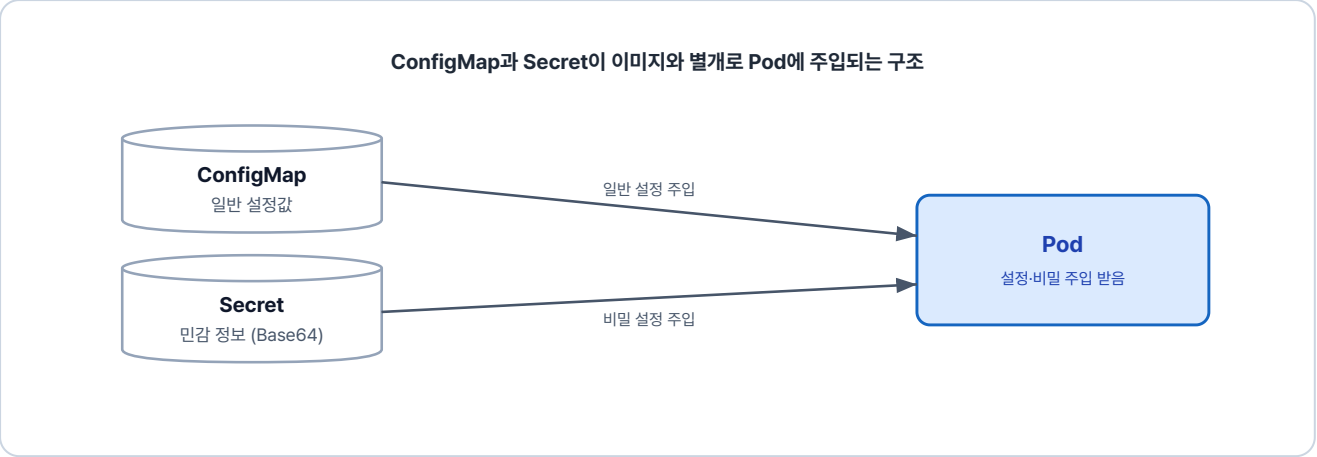


그림 6-3. ConfigMap과 Secret이 이미지와 별개로 Pod에 설정과 민감 정보를 주입

6.1.1 ConfigMap

ConfigMap은 환경에 따라 달라지는 설정값을 Pod에 환경 변수로 주입하는 리소스입니다.

이번 예제의 실습 파일은 `ex13` 폴더에 있습니다.

```
ex13/
├── configmap-conn.yml    # 설정값을 담는 ConfigMap 정의
├── secret-password.yml   # 비밀번호를 담는 Secret 정의
├── deploy-ex03.yml       # ConfigMap·Secret을 주입받는 Deployment
└── README.md             # 실습 안내
```

다음은 설정값 두 개를 정의한 ConfigMap입니다.

실습 1 **ex13/configmap-conn.yml. ConfigMap 정의**

```
apiVersion: v1
kind: ConfigMap
```

```yaml
metadata:
  name: configmap-conn                # ConfigMap 이름 지정
data:                                 # 설정값 넣는 영역
  conn_info: "localhost:80"
  conn_url: "config.test"
```

이어서 Deployment에 이 ConfigMap을 연결합니다. `envFrom.configMapRef`를 쓰면 ConfigMap의 값을 환경 변수로 주입받을 수 있습니다.

실습 2 **ex13/deploy-ex03.yml. ConfigMap을 Pod에 주입**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-config-secret           # Deployment 이름 (재시작 시 이 이름으로 지목)
spec:
  replicas: 1                         # pod 수 지정
  selector:
    matchLabels:
      app: nginx                      # 이 라벨을 가진 Pod를 관리 대상으로
  template:
    metadata:
      labels:
        app: nginx                    # pod에 붙일 라벨
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.20
          envFrom:
            - configMapRef:
                name: configmap-conn  # ConfigMap 연결
```

두 YAML을 적용한 뒤 Pod 안의 환경 변수를 확인합니다.

터미널 **ConfigMap·Deployment 적용과 환경 변수 확인**

```
kubectl apply -f ex13/configmap-conn.yml
kubectl apply -f ex13/deploy-ex03.yml
kubectl get pod                                                  # Pod 이름 확인
kubectl exec -it nginx-config-secret-794499d5d4-c2xmw -- env  # Pod 환경 변수 조회
```

실행결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-794499d5d4-c2xmw 1/1 Running 0 11s
$ kubectl exec -it nginx-config-secret-794499d5d4-c2xmw -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-794499d5d4-c2xmw
conn_info=localhost:80
conn_url=config.test
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT=tcp://10.96.0.1:443
```

그림 6-4. Pod 안의 환경 변수 목록에 ConfigMap의 값이 보이는 모습

ConfigMap에 적어 둔 conn_info와 conn_url이 Pod의 환경 변수로 주입됐습니다.

6.1.2 Secret

앞에서 본 ConfigMap이 일반 설정값을 위한 리소스라면, **Secret**은 비밀번호·토큰·키처럼 민감한 정보를 위한 리소스입니다.

다음은 비밀번호를 정의한 Secret YAML입니다. `stringData`에 평문을 적으면 쿠버네티스가 저장할 때 Base64로 인코딩합니다.

실습 3 **ex13/secret-password.yml. Secret 정의**

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-password
stringData:
  password: metacoding1234
```

Secret을 클러스터에 적용한 뒤 저장된 값을 확인합니다. 아래 명령어를 실행하면 저장된 Secret의 전체 내용이 YAML 형태로 출력됩니다.

터미널 **Secret 적용 후 저장값 확인**

```
kubectl apply -f ex13/secret-password.yml      # Secret YAML 적용
kubectl get secret secret-password -o yaml     # 저장된 Secret 내용을 YAML로 조회
```

실행결과

```
$ kubectl get secret secret-password -o yaml
apiVersion: v1
data:
  password: bWV0YWNvZGluZzEyMzQ=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":{},"name":"secret-
password","namespace":"defaul...
  creationTimestamp: "2026-03-15T06:30:27Z"
  name: secret-password
  namespace: default
  resourceVersion: "1387"
```

그림 6-5. Secret 내부를 보면 비밀번호가 Base64로 인코딩된 상태

저장된 결과를 보니 `password` 값이 Base64로 인코딩되어 알아볼 수 없는 문자열로 바뀌어 있습니다.

Pod에 주입하는 방식은 ConfigMap과 동일합니다. `deploy-ex03.yml`의 `envFrom` 아래에 `secretRef`를 추가하면, 쿠버네티스가 실행 시점에 Base64를 디코딩해 환경 변수로 주입합니다.

실습 4 **ex13/deploy-ex03.yml. Secret 연결 (envFrom 부분)**

```
envFrom:
  - configMapRef:
      name: configmap-conn
  - secretRef:
      name: secret-password   # Secret 연결
```

변경한 Deployment를 다시 적용한 뒤 환경 변수를 확인합니다.

터미널 **변경한 Deployment 적용과 환경 변수 확인**

```
kubectl apply -f ex13/deploy-ex03.yml                          # 변경한 Deployment 적용
kubectl get pod                                                # Pod 이름 확인
kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env   # Pod 환경 변수 조회
```

실행결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-7fbccb65f5-zq8nz 1/1 Running 0 51s
$ kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-7fbccb65f5-zq8nz
TERM=xterm
conn_url=config.test
password=metacoding1234
conn_info=localhost:80
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
```

그림 6-6. 환경 변수 목록에서 확인한 Secret의 값

6.1.3 ConfigMap 변경

기본 동작을 확인했다면 다음은 ConfigMap을 변경해 보겠습니다. `configmap-conn.yml`의 `conn_info` 포트를 80에서 90으로 변경합니다.

실습 5 **ex13/configmap-conn.yml. 포트 변경**

```
# ... 생략
data:                                  # 설정값 넣는 영역
  conn_info: "localhost:90"            # 환경변수 변경
  conn_url: "config.test"
```

변경한 ConfigMap을 클러스터에 적용합니다.

터미널 **변경된 ConfigMap 적용 후 환경 변수 확인**

```
kubectl apply -f ex13/configmap-conn.yml                       # 변경된 ConfigMap 적용
```

```
kubectl get pod                                                    # Pod 이름 확인
kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env  # Pod 환경 변수 조회
```

실행 결과

```
$ kubectl apply -f ex13/configmap-conn.yml
configmap/configmap-conn configured
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-7fbccb65f5-zq8nz 1/1 Running 0 6m
$ kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-7fbccb65f5-zq8nz
conn_info=localhost:80
conn_url=config.test
password=metacoding1234
KUBERNETES_SERVICE_HOST=10.96.0.1
```

그림 6-7. ConfigMap 적용 직후 기존 Pod의 환경 변수 조회 결과

ConfigMap은 갱신됐지만, 이미 실행 중인 Pod의 환경 변수는 여전히 80입니다.

'어? 분명히 바꿨는데 왜 그대로지?'

리눅스에서 환경 변수는 **프로세스가 시작될 때 한 번 주입되는 값**입니다. 그래서 ConfigMap이 갱신되어도 이미 실행 중인 Pod의 프로세스에는 처음 주입된 값이 그대로 남아 있습니다. **새 값을 반영하려면 Pod를 다시 실행해야 합니다.**

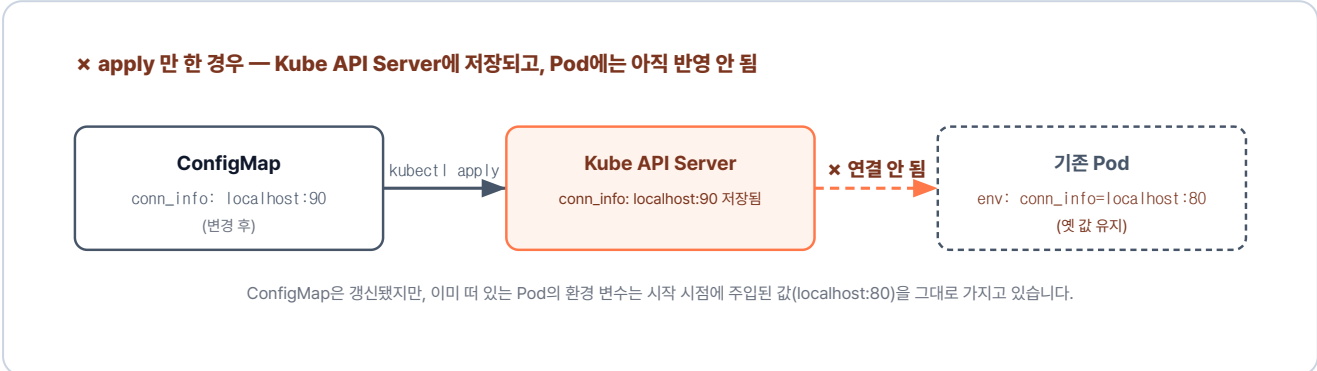


그림 6-8. apply 만 한 경우 - ConfigMap 변경이 기존 Pod에는 반영되지 않음



그림 6-9. rollout restart 로 Pod를 새로 실행하면 ConfigMap 새 값이 환경 변수로 반영됨

`kubectl rollout restart`는 Pod를 새로 교체해 새 값을 반영하는 명령입니다. 명령어를 실행해 Deployment를 재시작합니다.

터미널 **Pod 재시작 후 환경 변수 확인**

```
kubectl rollout restart deployment nginx-config-secret       # Pod 재시작
kubectl get pod                                              # 새 Pod 이름 확인
kubectl exec -it nginx-config-secret-8c5d4f9b6-r2t4p -- env  # Pod 환경 변수 조회
```

실행결과

```
$ kubectl rollout restart deployment nginx-config-secret
deployment.apps/nginx-config-secret restarted
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-8c5d4f9b6-r2t4p 1/1 Running 0 12s
$ kubectl exec -it nginx-config-secret-8c5d4f9b6-r2t4p -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-8c5d4f9b6-r2t4p
conn_url=config.test
conn_info=localhost:90
password=metacoding1234
```

그림 6-10. Pod 재시작 후 환경 변수에 새 포트 90이 반영된 모습

재시작 후 환경 변수가 적용된 것을 확인할 수 있습니다.

설정은 Pod가 새로 만들어질 때 반영된다

쿠버네티스에서 Pod는 한 번 생성되면 그 설정 내용을 중간에 수정할 수 없는 불변성을 가집니다. 따라서 **kubectl apply**로 ConfigMap이나 Secret의 값만 변경하더라도, 이미 실행 중인 기존 Pod에는 새 값이 자동으로 반영되지 않습니다.

반면, Deployment 내의 Pod 템플릿(template)을 수정하여 적용하면 쿠버네티스는 이를 새로운 배포로 인식합니다. 즉시 교체 작업이 시작되어 기존 Pod를 종료하고 바뀐 템플릿으로 새로운 Pod를 생성하므로 변경 내용이 반영됩니다.

6.2 Volume - 데이터의 영속성 확보

6.2.1 Pod의 휘발성 문제

Deployment의 자동 복구가 데이터베이스에서는 다른 문제를 만듭니다. Pod는 기본적으로 **휘발성**이라, 안에서 만든 파일은 Pod 수명과 함께 사라집니다. 데이터가 매일 쌓이는 DB가 이대로 휘발되면 운영할 수 없습니다.

이 문제는 데이터를 Pod 외부에 두어 해결할 수 있습니다. Docker에서 본 마운트처럼, 쿠버네티스에서도 별도의 저장 공간에 데이터를 두는 기능을 **Volume**이라고 합니다.

Volume에는 여러 종류가 있습니다.

종류	설명	데이터 유지
`emptyDir`	Pod 생성 시 만들어지는 임시 저장 공간	Pod 삭제 시 함께 삭제
`hostPath`	워커 노드(호스트)의 특정 경로를 Pod에 마운트	노드에 남지만, Pod가 다른 노드로 이동하면 접근 불가
`PV / PVC`	클러스터가 관리하는 영구 저장소를 만들고 요청서(PVC)로 Pod에 연결	Pod가 삭제되어도 유지

이 중 가장 보편적으로 사용하는 **PV(PersistentVolume)** 와 **PVC(PersistentVolumeClaim)** 를 알아보겠습니다.

6.2.2 PV와 PVC

쿠버네티스는 저장소(PV)와 사용 요청(PVC)을 분리합니다. **PV는 용량과 권한 등 실제 저장 공간의 사양을 정의**하고, **PVC는 필요한 공간을 요구하는 신청서**입니다. PVC로 "1Gi 크기의 읽기·쓰기가 가능한 공간"을 요청하면, 쿠버네티스가 조건에 맞는 PV를 찾아 자동으로 연결합니다.

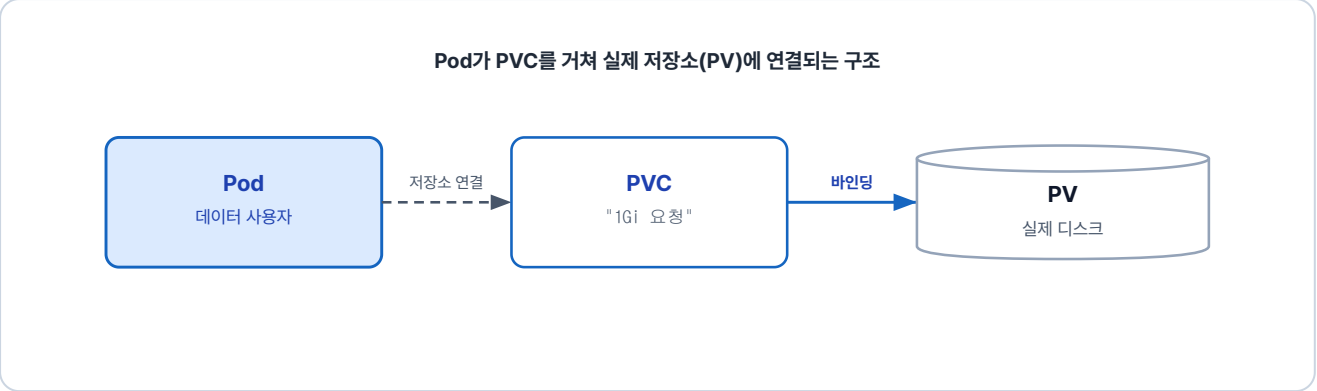


그림 6-11. PV는 실제 저장 공간, PVC는 그 공간을 요청하는 신청서

Pod는 PV를 직접 참조하지 않고 PVC만 연결해 사용합니다. 덕분에 실제 저장소가 무엇인지 몰라도 됩니다.

6.2.3 PV 만들기

PV를 먼저 만들어 보겠습니다. 이번 실습에서는 PV의 저장 위치로 Minikube 내부의 경로(`/mnt/data`)를 지정합니다.

이번 예제의 실습 파일은 `ex14` 폴더에 있습니다.

```
ex14/
├── volume-pv.yml      # 저장 공간을 제공하는 PersistentVolume 정의
├── volume-pvc.yml     # 저장 공간을 요청하는 PersistentVolumeClaim 정의
├── volume-pod.yml     # PVC를 마운트해 데이터를 쓰는 Pod 정의
└── README.md          # 실습 안내
```

실습 6 **ex14/volume-pv.yml. PersistentVolume 정의**

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: volume-pv                    # PVC가 volumeName으로 참조할 이름
spec:
  capacity:
```

```
    storage: 1Gi                        # 저장 용량
  accessModes:
    - ReadWriteOnce                     # 단일 노드 읽기·쓰기 모드
  storageClassName: ""                  # 자동 StorageClass 비활성 (정적 바인딩)
  hostPath:
    path: /mnt/data                     # Minikube 내부의 실제 저장 경로
    type: DirectoryOrCreate             # 경로가 없으면 자동 생성
```

> **StorageClass란**
> **StorageClass**는 PVC가 요청한 조건에 맞는 PV를 자동으로 만들어 주는 템플릿입니다. AWS·GCP·Minikube 같은 환경마다 기본 StorageClass가 따로 준비되어 있어 평소에는 PVC만 작성해도 PV가 자동 발급됩니다. 하지만 이번 실습은 PV를 직접 만들어 보기 위해 **storageClassName: ""** 로 자동 생성을 비활성화했습니다.

6.2.4 PVC 만들기

다음으로 앞서 만든 PV에 연결할 PVC를 만들어 보겠습니다.

실습 7 **ex14/volume-pvc.yml. PersistentVolumeClaim 정의**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: volume-pvc                      # PVC 이름 (Pod가 claimName으로 참조)
spec:
  accessModes:
    - ReadWriteOnce                     # PV와 일치해야 바인딩
  storageClassName: ""                  # PV와 일치해야 바인딩
  resources:
    requests:
      storage: 1Gi                      # 신청 용량 (PV capacity 이하)
  volumeName: volume-pv                 # 바인딩할 PV 이름을 수동 지정
```

PVC를 생성하면 PV와 연결됩니다. 다만 둘이 연결되려면 서로 맞아야 할 조건이 있습니다.

> **PVC와 PV가 연결되는 조건**
> PVC와 PV가 연결되려면 읽기·쓰기 방식(`accessModes`)과 스토리지 종류(`storageClassName`)가 일치해야 하며, PV의 용량이 PVC가 요청한 크기 이상이어야 합니다. 이 조건 중 하나라도 어긋나면 PVC는 짝을 찾지 못하고 **Pending** 상태에 머무릅니다.
> 단, `volumeName`으로 특정 PV를 직접 지정했다면 다른 조건이 모두 맞더라도 이름이 일치하는 PV와만 연결됩니다.

6.2.5 Pod에 마운트

이번 실습은 편의를 위해 Deployment 대신 Pod를 직접 생성합니다. 앞서 만든 PVC를 Pod에 연결하면, 컨테이너가 저장소를 쓸 수 있습니다.

실습 8 **ex14/volume-pod.yml. PVC를 마운트한 Pod**

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: volume-pod                    # Pod 이름
spec:
  containers:
  - name: nginx-volume                # 컨테이너 이름
    image: nginx                      # 사용할 이미지
    volumeMounts:
    - name: storage                   # 아래 volumes에서 정의한 이름과 일치
      mountPath: /data                # 컨테이너가 데이터를 읽고 쓰는 경로
  volumes:
  - name: storage                     # volumeMounts에서 참조할 이름
    persistentVolumeClaim:
      claimName: volume-pvc           # 연결할 PVC 이름
```

컨테이너가 보는 마운트 경로는 `/data`이고, Minikube 내부의 실제 저장 경로는 `/mnt/data`입니다. 컨테이너가 `/data`에 파일을 생성하면 PVC와 PV를 거쳐 최종적으로 Minikube의 `/mnt/data`에 저장됩니다.

아래 명령어를 실행해 클러스터에 적용합니다.

터미널 **PV·PVC·Pod 일괄 생성과 Bound 확인**

```bash
kubectl apply -f ex14/volume-pv.yml      # PV(저장소) 생성
kubectl apply -f ex14/volume-pvc.yml     # PVC(저장소 요청) 생성
kubectl apply -f ex14/volume-pod.yml     # PVC를 마운트한 Pod 생성
kubectl get pv,pvc -o wide               # PV·PVC가 Bound 됐는지 확인
```

실행결과

```
$ kubectl get pv,pvc -o wide
NAME CAPACITY ACCESS MODES RECLAIM POLICY STATUS CLAIM S...
persistentvolume/volume-pv 1Gi RWO Retain Bound default/volume-pvc <unset>
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCl...
persistentvolumeclaim/volume-pvc Bound volume-pv 1Gi RWO <unset> 9s
```

그림 6-12. PV와 PVC가 Bound 상태로 연결된 결과

STATUS가 Bound 상태라면 PV와 PVC가 정상적으로 연결됐다는 뜻입니다.

확인을 위해 Pod 내부에서 파일을 하나 만든 뒤, Pod를 삭제하고 다시 실행해 보겠습니다.

터미널 **파일 생성 후 Pod 재생성으로 데이터 보존 확인**

```
kubectl exec -it volume-pod -- /bin/bash      # Pod 안 bash 접속
touch /data/c.txt                              # 마운트 경로에 빈 파일 생성
exit                                           # Pod 셸 종료

kubectl delete pod volume-pod                  # 기존 Pod 삭제
kubectl apply -f ex14/volume-pod.yml           # 같은 PVC를 쓰는 Pod 재생성
kubectl exec -it volume-pod -- ls /data        # 새 Pod에서 c.txt가 남아 있는지 확인
```

실행결과

```
$ kubectl exec -it volume-pod -- /bin/bash
root@volume-pod:/# touch /data/c.txt
root@volume-pod:/# exit
exit
$ kubectl delete pod volume-pod
pod "volume-pod" deleted
$ kubectl apply -f ex14/volume-pod.yml
pod/volume-pod created
$ kubectl exec -it volume-pod -- ls /data
c.txt
```

그림 6-13. Pod가 새로 생성됐는데도 c.txt가 그대로 남아 있는 모습

새 Pod에서도 `c.txt`가 그대로 보입니다. Pod는 삭제·재생성되어도 PV 안 파일은 그대로 남습니다.

6.3 통합 실습 - 쿠버네티스 위에 웹사이트 올리기

마지막으로 지금까지 익힌 리소스를 모아 하나의 프로젝트로 연결해 보겠습니다. 앞서 Docker Compose로 실행했던 프로젝트를 쿠버네티스 위에 다시 구축합니다.

6.3.1 전체 그림

이번 실습의 구성도에는 프론트엔드, 백엔드, DB, 그리고 방문 횟수를 기록하는 Redis까지 서비스 네 개가 들어 있습니다. 앞서 Docker Compose로 실행했던 세 서비스에 Redis가 더해진 구조입니다.

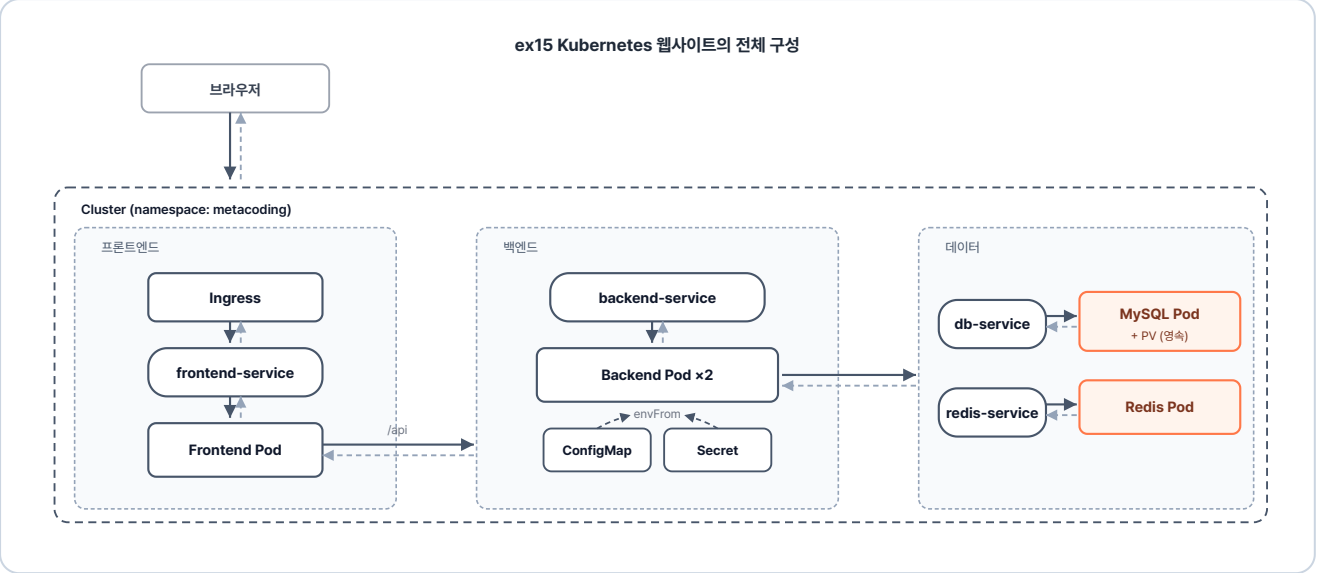


그림 6-14. ex15 Kubernetes 웹사이트의 전체 구성

6.3.2 폴더 구조와 진행 방식

ex15는 이미지를 만드는 폴더들과 쿠버네티스 설정(YAML) 폴더로 나뉘어 있습니다.

```
ex15/
├── backend/                    # Spring Boot 백엔드 이미지
│   ├── Dockerfile              # JDK 이미지 + entrypoint.sh 복사
│   └── entrypoint.sh           # Git clone + Gradle 빌드 + JAR 실행
├── db/                         # MySQL 이미지
│   ├── Dockerfile              # MySQL 이미지 + init.sql 복사
│   └── init.sql                # 테이블·초기 데이터 생성 스크립트
├── frontend/                   # NGINX + HTML 이미지
│   ├── Dockerfile              # nginx 이미지 + index.html·nginx.conf 복사
│   ├── index.html              # 로그인/게시판 UI (방문 카운터 표시)
│   └── nginx.conf              # /api 경로를 backend-service로 프록시
├── redis/                      # Redis 이미지
```

```
│   └── Dockerfile                    # redis 공식 이미지 기반
├── k8s/                              # 쿠버네티스 리소스 YAML
│   ├── namespace.yml                 # ex15 네임스페이스 정의
│   ├── backend/
│   │   ├── backend-configmap.yml     # 비밀이 아닌 설정값
│   │   ├── backend-deploy.yml        # 백엔드 Deployment
│   │   ├── backend-secret.yml        # DB 비밀번호 등 민감 정보
│   │   └── backend-service.yml       # 내부용 ClusterIP Service
│   ├── db/
│   │   ├── db-deploy.yml             # MySQL Deployment
│   │   ├── db-pv.yml                 # PersistentVolume (노드 로컬 저장소)
│   │   ├── db-pvc.yml                # PersistentVolumeClaim (볼륨 요청)
│   │   ├── db-secret.yml             # MySQL 계정 정보
│   │   └── db-service.yml            # 내부용 ClusterIP Service
│   ├── frontend/
│   │   ├── frontend-deploy.yml       # 프론트 Deployment
│   │   ├── frontend-ingress.yml      # 외부 진입점 (Ingress)
│   │   └── frontend-service.yml      # 내부용 ClusterIP Service
│   └── redis/
│       ├── redis-deploy.yml          # Redis Deployment
│       └── redis-service.yml         # 내부용 ClusterIP Service
└── README.md                         # 실습 안내
```

6.3.3 리소스 살펴보기

Namespace - 논리적 분리

지금까지 만든 리소스는 별도 공간을 지정하지 않았기 때문에 모두 **default라는 기본 공간**에 생성되었습니다. 혼자 실습할 때는 문제가 없지만, 여러 팀이 같은 클러스터를 함께 사용하면 리소스 이름이 겹쳐서 충돌할 수 있습니다.

하나의 회사 안에서 팀마다 사무실을 나눠 쓰듯이, 쿠버네티스도 리소스 공간을 분리할 수 있습니다. 이렇게 하나의 클러스터를 논리적으로 나눠주는 가상 공간을 **네임스페이스**(Namespace)라고 합니다.

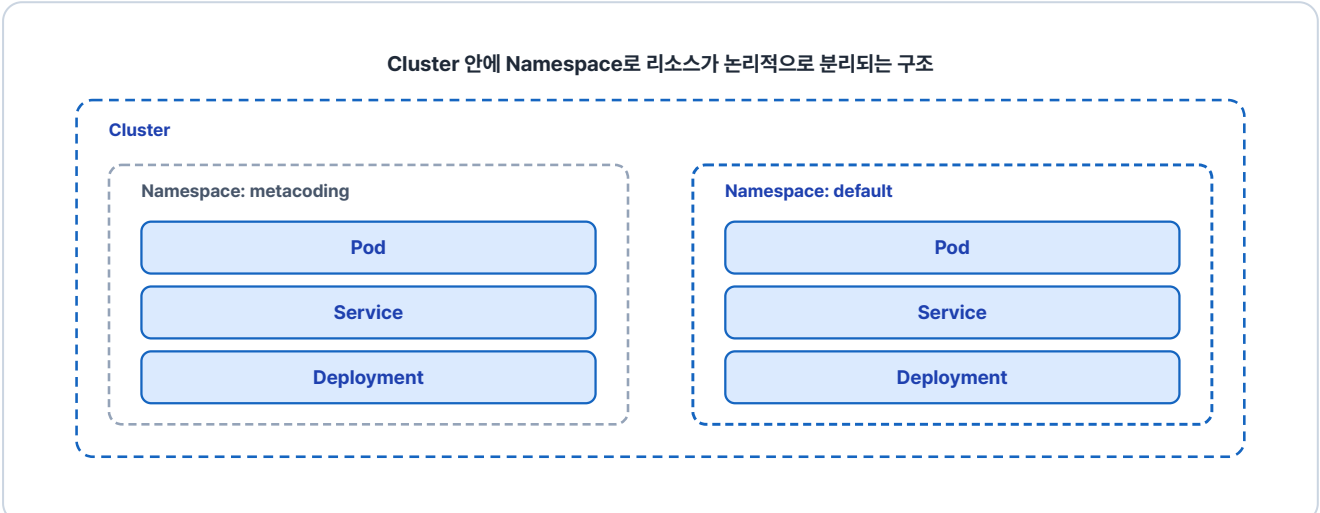


그림 6-15. 같은 클러스터 안에서 Namespace가 리소스를 논리적으로 분리

이번 실습에서는 `metacoding`이라는 전용 네임스페이스를 만들어 모든 리소스를 그 안에서 관리합니다.

실습 9 **ex15/k8s/namespace.yml. 네임스페이스 정의**

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: metacoding
```

이후 만들어지는 모든 리소스는 metacoding이라는 독립된 영역에 들어갑니다.

Frontend - 외부 진입

frontend 폴더는 외부에서 들어오는 요청이 처음 도달하는 입구를 정의합니다. 그 입구에서 요청을 어디로 보낼지는 `frontend-ingress.yml`이 정합니다.

실습 10 **ex15/k8s/frontend/frontend-ingress.yml. 외부 진입 Ingress**

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-ingress
  namespace: metacoding
spec:
  ingressClassName: nginx       # 어느 Ingress 컨트롤러가 이 규칙을 집행할지 지정
  rules:
    - http:
        paths:
          - path: /                 # 모든 경로를 잡음
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

frontend-ingress.yml은 `/` 경로로 들어오는 모든 요청을 frontend-service로 넘깁니다.

같은 폴더의 나머지 두 파일은 다음과 같습니다.

파일	역할
`frontend-deploy.yml`	웹 화면을 보여주는 프론트엔드를 실행합니다
`frontend-service.yml`	Ingress가 프론트엔드 Pod로 들어가는 진입점입니다

Backend - API 처리

backend 폴더의 핵심 YAML은 `backend-deploy.yml` 입니다. Pod를 2개 생성하고, DB 접속 정보는 ConfigMap·Secret에서 가져옵니다.

실습 11 **ex15/k8s/backend/backend-deploy.yml. ConfigMap·Secret 주입**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deploy
  namespace: metacoding              # 모든 리소스를 metacoding 네임스페이스에 배치
spec:
  replicas: 2                        # Pod 두 개로 트래픽 분산
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend-server
          image: metacoding/backend:1
          ports:
            - containerPort: 8080    # Tomcat 기본 포트
          envFrom:
            - configMapRef:
                name: backend-configmap   # DB 주소·JDBC URL·Redis 호스트 등 일반 설정
            - secretRef:
                name: backend-secret      # DB 계정·비밀번호
```

같은 폴더의 다른 세 파일은 다음과 같습니다.

파일	역할
`backend-service.yml`	다른 Pod가 백엔드 Pod로 들어가는 진입점입니다

파일	역할
`backend-configmap.yml`	환경에 따라 달라지는 설정값(DB·Redis 주소 등)을 담습니다
`backend-secret.yml`	비밀번호 같은 민감 정보를 담습니다

DB - 영속 저장소

db 폴더의 핵심 YAML은 `db-deploy.yml`입니다. Pod 1개를 생성하고, 데이터는 PV에 저장합니다.

실습 12 **ex15/k8s/db/db-deploy.yml. PVC 마운트 MySQL Deployment**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: db-deploy
  namespace: metacoding
spec:
  replicas: 1                        # DB는 단일 인스턴스로 운영
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
        - name: db-server
          image: metacoding/db:1
          ports:
            - containerPort: 3306    # MySQL 기본 포트
          envFrom:
            - secretRef:
                name: db-secret      # MySQL 계정·비밀번호
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql   # MySQL이 데이터를 쓰는 경로를 PV로
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: db-pvc
```

같은 폴더의 나머지 네 파일은 다음과 같습니다.

파일	역할
`db-service.yml`	백엔드가 DB Pod로 들어가는 진입점입니다
`db-secret.yml`	DB 접속 계정 정보를 담습니다
`db-pv.yml`	노드의 실제 저장 공간입니다
`db-pvc.yml`	DB가 저장 공간을 신청하는 요청서입니다

Redis - 인메모리 캐시

redis 폴더는 백엔드가 방문 횟수를 기록하기 위해 호출하는 인메모리 저장소이므로 다른 설정 파일 없이 두 파일로 구성됩니다.

파일	역할
`redis-deploy.yml`	방문 횟수를 기록하는 인메모리 저장소를 실행합니다
`redis-service.yml`	백엔드가 Redis Pod로 들어가는 진입점입니다

6.3.4 공통 준비

코드를 살펴봤으니 배포할 차례입니다. 실습을 시작하기 전에 Minikube와 Ingress 컨트롤러가 켜져 있지 않다면 아래 명령어를 실행합니다. 그런 다음 서비스별 이미지를 빌드합니다.

터미널 **Minikube와 Ingress 컨트롤러 시작**

```
minikube start
minikube addons enable ingress
```

실행결과

```
$ minikube addons enable ingress
* ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at:
https://github.com/kubernetes/minikube/blob/master/OWNERS
* After the addon is enabled, please run "minikube tunnel" and your ingress resources would
be available at "127.0.0.1"
   - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
   - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
   - Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
* Verifying ingress addon...
* The 'ingress' addon is enabled
```

그림 6-16. Nginx Ingress 컨트롤러 애드온 설치

Minikube는 독립적인 가상 환경에서 작동하는 클러스터라, 로컬 호스트에서 빌드한 이미지를 곧장 인식하지 못합니다. 그래서 별도의 레지스트리를 두지 않고 `minikube image build` 명령으로 Minikube 자체에 이미지를 직접 만들어 둡니다.

터미널 **네 이미지 한 번에 빌드**

```
minikube image build -t metacoding/db:1 ex15/db              # DB 이미지 빌드
minikube image build -t metacoding/backend:1 ex15/backend    # 백엔드 이미지 빌드
minikube image build -t metacoding/frontend:1 ex15/frontend  # 프론트엔드 이미지 빌드
minikube image build -t metacoding/redis:1 ex15/redis        # Redis 이미지 빌드
```

6.3.5 한 번에 배포하고 결과 확인

리소스 구경을 마쳤으니 이제 `ex15/k8s/` 폴더를 한꺼번에 배포합니다.

터미널 **전체 리소스 일괄 배포**

```
kubectl apply -f ex15/k8s/namespace.yml      # Namespace 먼저
kubectl apply -f ex15/k8s/ --recursive       # k8s 이하 폴더의 리소스 전부 일괄 배포
```

명령을 치면 모든 리소스가 한꺼번에 생성됩니다. 이어서 상태를 조회합니다.

터미널 **배포 결과 조회**

```
kubectl get deploy,pod,service,ingress -n metacoding   # 리소스 상태 일괄 조회
```

프론트엔드와 데이터베이스 Pod는 빠르게 Running 상태로 바뀝니다. 백엔드 Pod만 Gradle 빌드와 의존성 설치 때문에 시간이 더 걸립니다.

모든 Pod가 Running 상태가 되면, `minikube tunnel`을 실행합니다. 그러면 Ingress 입구가 `127.0.0.1`로 열립니다.

터미널 **외부 터널 개방**

```
minikube tunnel                          # 새 터미널에서 실행 (관리자 권한 요구)
```

브라우저에서 `http://127.0.0.1`로 접속해 결과를 확인합니다.



그림 6-17. Ingress를 거쳐 웹사이트가 화면에 응답

화면에는 데이터베이스에서 불러온 정보와 함께 방문 횟수가 표시됩니다. 새로고침을 할 때마다 숫자가 하나씩 올라가는데, 방문 횟수가 **Redis에 저장되어 공유**되기 때문에 **백엔드 Pod가 두 개여도 카운터가 함께 증가**합니다.

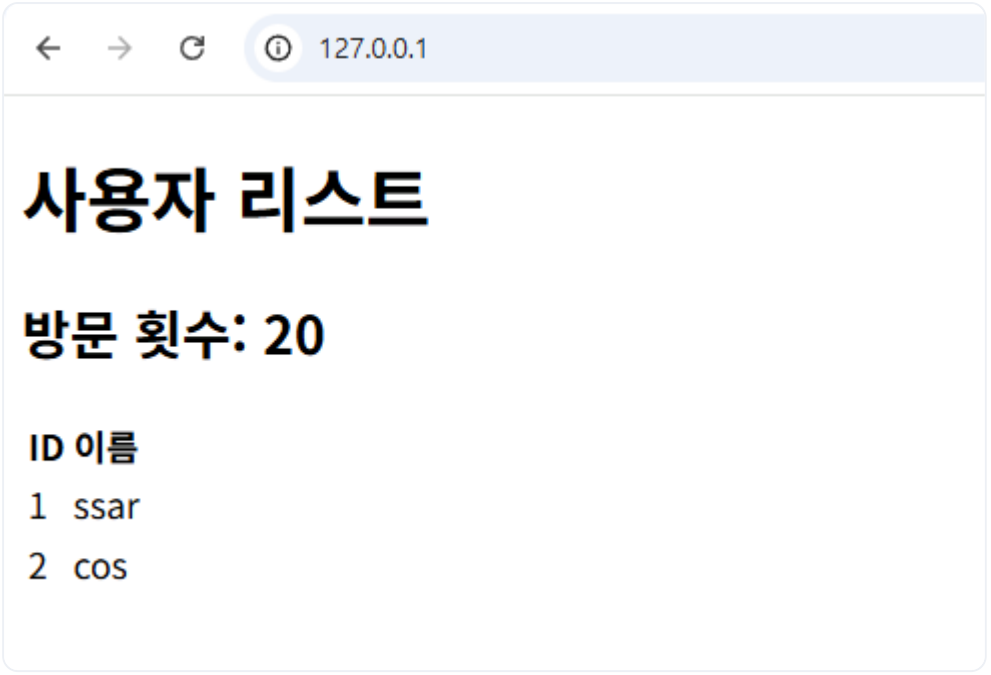


그림 6-18. 새로고침 시 방문 횟수가 증가

오픈이는 화면을 새로고침했습니다. 방문 횟수가 오르고, 사이트는 문제없이 응답합니다. 여러 서비스가 하나의 클러스터 안에서 각자 제 역할을 하며 돌아갑니다.

개발과 운영 환경의 차이를 극복하기 위해 시작된 여정은 자동화된 시스템 운영으로 완성됩니다. 코드를 환경째 컨테이너에 담아 어디서든 동일하게 실행할 수 있게 되면서, 인프라를 다루는 방식 자체가 바뀝니다. 장애가 난 컨테이너를 재시작하고, 부하에 맞춰 그 수를 늘리고, 중단 없이 배포하는 일까지 이제 온전히 클러스터의 몫이 됩니다.

이것만은 기억하자

- **설정과 비밀번호는 이미지 외부에 둡니다.** ConfigMap에는 환경마다 달라지는 일반 설정값을 두고, Secret에는 비밀번호·토큰 같은 민감 정보를 보관합니다.
- **ConfigMap을 바꾼 뒤에는 Pod를 새로 만듭니다.** 환경 변수는 프로세스 시작 시점에 한 번 꽂히는 값이라 `apply`만으로는 갱신이 반영되지 않습니다. `kubectl rollout restart`로 Pod를 교체해야 새 값이 박힙니다.
- **데이터는 PV·PVC로 Pod 외부에 두어 보존합니다.** Pod는 휘발성이라 안에서 만든 파일은 함께 사라지지만, PVC가 가리키는 PV에 데이터를 두면 Pod가 새로 생성돼도 그대로 남습니다.

에필로그

그로부터 6개월이 지났습니다.

예전에 오픈이를 가장 끈질기게 괴롭혔던 문제는 개발 노트북과 실제 서버의 환경이 다르다는 점이었습니다. 자기 PC에서는 완벽하게 돌아가던 코드가 운영 서버에만 올라가면 알 수 없는 에러를 뱉어내기 일쑤였습니다. 그럴 때면 원인이 코드에 있는지, 서버에 설치된 소프트웨어 버전에 있는지 찾느라 며칠 밤낮을 허비해야 했습니다.

하지만 이제 사무실에서 "제 컴퓨터에서는 분명히 됐는데..."라는 당혹스러운 변명은 사라졌습니다. 프로그램과 그것이 실행되는 데 필요한 모든 환경을 컨테이너에 담아 통째로 배포하기 때문입니다. 자기 노트북에서 잘 돌아갔다면, 운영 서버에서도 똑같이 돌아간다는 확실한 믿음이 생겼습니다.

배포를 마친 날 밤의 풍경도 완전히 달라졌습니다. 예전에는 배포한 뒤 컨테이너가 멈추거나 새 버전에 문제가 생기면 새벽에라도 직접 손을 봐야 했기에, 휴대폰을 쥐고 선잠을 잤습니다. 그러나 이제는 컨테이너가 멈춰도 클러스터가 몇 초 만에 새 컨테이너로 빈자리를 채우고, 새 버전도 서비스 중단 없이 배포됩니다. 어젯밤 새로운 버전을 배포했지만, 오픈이는 평소처럼 깊고 편안하게 잠을 잤습니다.

다음 날 아침, 오픈이는 출근하자마자 모니터링 로그를 확인했습니다. 밤사이 컨테이너 하나가 오류로 멈췄지만, 클러스터가 새 컨테이너로 교체한 뒤라 서비스는 한 번도 끊기지 않았습니다. 서버 환경과 씨름하며 에너지를 쏟던 시간은 이제 온전히 다음 기능을 개발하는 시간으로 바뀌었습니다. 오픈이는 가벼운 마음으로 오늘 작성할 코드를 열며 하루를 시작합니다.

마치며

이 책은 로컬 환경과 운영 서버의 차이에서 오는 답답함에서 출발했습니다. 코드를 실행 환경과 함께 컨테이너에 담아 어디서든 똑같이 동작하게 만들고, 늘어난 컨테이너를 쿠버네티스 클러스터 위에 올려 스스로 유지되도록 구성하는 과정까지 차근차근 살펴보았습니다.

이제 책을 덮고 현업으로 돌아가면, 여기서 다룬 예제보다 훨씬 거대하고 복잡한 시스템을 마주하게 됩니다. 수많은 Pod와 낯선 설정 파일이 얽혀 있는 모습에 다시금 막막함이 앞설 수도 있습니다.

하지만 겉보기에 규모가 커졌을 뿐, 본질은 우리가 함께 구성해 본 구조와 다르지 않습니다. 아무리 복잡한 시스템이라도 '어떤 컨테이너를 올렸고, 클러스터가 어떤 상태를 유지하는가'라는 기준을 잡으면 충분히 전체 그림을 짚어낼 수 있습니다.

우리가 복잡한 인프라 기술을 익히는 이유는, 역설적이게도 더 이상 인프라에 얽매이지 않기 위해서입니다. 환경 설정과 배포에 쏟던 에너지를 덜어내고, 여러분이 정말 만들고자 하는 서비스 본연의 가치와 코드에 온전히 집중할 수 있기를 바랍니다.

이 책에서 함께 그려 본 밑그림이, 앞으로 마주할 문제 앞에서 기준점이 되기를 기대합니다.